
Entwicklung einer Ontologie-basierten Pipeline zur semantischen Reaktionsklassifikation und Datenintegration

Development of an Ontology-Based Pipeline for Semantic Reaction Classification and Data
Integration

Frederick Gabriel Kitel

Matrikelnummer: 226017

Dortmund, 23. April 2025

Erstprüfer: Prof. Dr.-Ing. Norbert Kockmann

Zweitprüfer: M.Sc. Hendrik Borgelt

Eidesstattliche Versicherung

(Affidavit)

Kitel, Frederick

226017

Name, Vorname
(surname, first name)

Matrikelnummer
(student ID number)

☒ Bachelorarbeit
(Bachelor's thesis)

☐ Masterarbeit
(Master's thesis)

Titel
(Title)

Development of an Ontology-Based Pipeline for Semantic Reaction Classification and Data Integration

Entwicklung einer Ontologie-basierten Pipeline zur semantischen Reaktionsklassifikation und Datenintegration

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem oben genannten Titel selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

I declare in lieu of oath that I have completed the present thesis with the above-mentioned title independently and without any unauthorized assistance. I have not used any other sources or aids than the ones listed and have documented quotations and paraphrases as such. The thesis in its current or similar version has not been submitted to an auditing institution before.

Hattingen, 23.04.2025

Frederick Kitel

Ort, Datum
(place, date)

Unterschrift
(signature)

Belehrung:

Wer vorsätzlich gegen eine die Täuschung über Prüfungsleistungen betreffende Regelung einer Hochschulprüfungsordnung verstößt, handelt ordnungswidrig. Die Ordnungswidrigkeit kann mit einer Geldbuße von bis zu 50.000,00 € geahndet werden. Zuständige Verwaltungsbehörde für die Verfolgung und Ahndung von Ordnungswidrigkeiten ist der Kanzler/die Kanzlerin der Technischen Universität Dortmund. Im Falle eines mehrfachen oder sonstigen schwerwiegenden Täuschungsversuches kann der Prüfling zudem exmatrikuliert werden. (§ 63 Abs. 5 Hochschulgesetz - HG -).

Die Abgabe einer falschen Versicherung an Eides statt wird mit Freiheitsstrafe bis zu 3 Jahren oder mit Geldstrafe bestraft.

Die Technische Universität Dortmund wird ggf. elektronische Vergleichswerkzeuge (wie z.B. die Software „turnitin“) zur Überprüfung von Ordnungswidrigkeiten in Prüfungsverfahren nutzen.

Die oben stehende Belehrung habe ich zur Kenntnis genommen:

Official notification:

Any person who intentionally breaches any regulation of university examination regulations relating to deception in examination performance is acting improperly. This offense can be punished with a fine of up to EUR 50,000.00. The competent administrative authority for the pursuit and prosecution of offenses of this type is the Chancellor of TU Dortmund University. In the case of multiple or other serious attempts at deception, the examinee can also be unenrolled, Section 63 (5) North Rhine-Westphalia Higher Education Act (*Hochschulgesetz, HG*).

The submission of a false affidavit will be punished with a prison sentence of up to three years or a fine.

As may be necessary, TU Dortmund University will make use of electronic plagiarism-prevention tools (e.g. the "turnitin" service) in order to monitor violations during the examination procedures.

I have taken note of the above official notification:*

Hattingen, 23.04.2025

Frederick Kitel

Ort, Datum
(place, date)

Unterschrift
(signature)

***Please be aware that solely the German version of the affidavit ("Eidesstattliche Versicherung") for the Bachelor's/ Master's thesis is the official and legally binding version.**

Abkürzungsverzeichnis

BFO	Basic Formal Ontology
ChEBI	Chemical Entities of Biological Interest
FAIR	Findability, Accessibility, Interoperability, and Reuse
IAO	Information Artifact Ontology
IRI	International Ressource Identifier
IUPAC	International Union of Pure and Applied Chemistry
MOP	Molecular Process Ontology
NC-IUBMB	Nomenclature Committee of the International Union of Biochemistry and Molecular Biology
OWL2	Web Ontology Language
RDF	Resource Description Framework
RXNO	Name Reaction Ontology
SHACL	Shapes Constraint Language
SPARQL	SPARQL Protocol And RDF Query Language
URI	Uniform Resource Identifiers
W3C	World Wide Web Consortium
RDKit	Rational Discovery Kit
SMILES	Simplified Molecular Input Line Entry System
SMARTS	SMILES Arbitrary Target Specification
InChI	International Chemical Identifier
OWA	Open World Assumption
CWA	Closed World Assumption
DL	Description Logic
BGP	Basic Graph Pattern
KEGG	Kyoto Encyclopedia of Genes and Genomes
NCBI	National Center for Biotechnology Informa- tion
NLM	National Library of Medicine
IUPAC	International Union of Pure and Applied Chemistry
EBI	European Bioinformatics Institute
OBO	Open Biological and Biomedical Ontologies

RSC	Royal Society of Chemistry
Regex	Regular Expressions

Abbildungsverzeichnis

2.1	RDF-Triple von Reaktion B, welche den Reaktanden Substanz A enthält	5
2.2	Grundstruktur einer SPARQL-Abfrage	12
2.3	Datenvalidierung via SHACL	14
3.1	InChI-Code und Strukturformel von Essigsäure [44]	21
4.1	Workflow der Datenintegration und -analyse	25
4.2	RXNO Superklassen [50]	28
4.3	Beispielreaktion „Basische Hydrolyse von Triacetin“ für die repräsentative Modellierung von Reaktionen in Ontologien	28
4.4	Klassenhierarchie der Klasse der Alkohole in RXNO	29
4.5	Modifizierte Klassenhierarchie der Klasse der Alkohole in RXNO	30
4.6	Modifizierte Klassenhierarchie der Klasse der Ester	31
4.7	Modifizierte Klassenhierarchie für anorganische Komponenten	32
4.8	Modifizierte Klassenhierarchie für funktionelle Gruppen	33
4.9	Klassenhierarchie und -restriktionen der Reaktionsklassen	34
4.10	Import-Pipelines von exp. sowie kuratierten bzw. idealisierten Reaktionsdaten aus Literatur und Datenbanken in den Wissensgraphen	36
4.11	Exemplarischer Datenbankeintrag für eine Reaktion aus der KEGG	41
4.12	Klassenrestriktionen für Glycerin und Triacetin	47
5.1	GUI-Eingabefenster für verschiedene Import-Optionen	52
5.2	OntoGraf für importierte Beispielreaktionen	53
5.3	Substrukturanalyse-Ergebnisse und Verknüpfung der Instanzen	55
5.4	Klassenzuordnung der Substanzen durch Reasoning	56
5.5	Ergebnisse für durch HermiT Reasoner neu inferierte Klassen	57
5.6	KEGG-Reaktion R12516: Hydrolyse von Isoamylacetat [64]	58
5.7	Experimentelle Durchführung der Isoamylacetat-Hydrolyse mit zusätzlichen Produkten	58
5.8	Reaktionsinstanzen und Relationen innerhalb des KG nach Import von Reaktionsdaten	59
5.9	Ergebnis der SPARQL-Ähnlichkeitsanalyse	60

A.1	Exemplarische Darstellung von relevanten EnzymeML-Tabellenblätter für Reaktionen und Reaktanden	XVI
A.2	Basische Esterhydrolyse von Essigsäurebutylester	XVI

Tabellenverzeichnis

2.1	RDF Schema Konstrukte und deren Beschreibung [6]	6
2.2	Eigenschaftsmerkmale und deren Bedeutung	10
2.3	SPARQL Abfrage-Klauseln und deren Beschreibung	13
3.1	SMILES-Spezifikationen [45]	22
3.2	SMARTS-Spezifikationen [46]	23

Inhaltsverzeichnis

Abkürzungsverzeichnis	II
Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
1 Einleitung	1
2 Theoretischer Hintergrund	3
2.1 FAIR Data	3
2.1.1 Experimentelle FAIR Daten	3
2.2 Semantische Artefakte	4
2.2.1 RDF	4
2.2.2 OWL2	6
2.2.3 SPARQL	12
2.2.4 SHACL	14
3 Aktueller Stand der Technik	15
3.1 Datenbanken & Ontologien	15
3.1.1 PubChem	16
3.1.2 ChEBI	16
3.1.3 KEGG	17
3.1.4 RHEA	17
3.1.5 BFO	17
3.1.6 MOP & RXNO	18
3.1.7 EnzymeML & pyEnzyme	18
3.2 Software	18
3.2.1 Protégé	19
3.2.2 Python Erweiterungen	19
3.2.3 InChI	21
3.2.4 SMILES	22
3.2.5 SMARTS	23

3.2.6	pyEnzyme	23
4	Methodischer Ansatz	25
4.1	Auswahl und Modifikation der Basis Ontologie	27
4.1.1	Wahl der Basis-Ontologie	27
4.1.2	Modifikation von RXNO	27
4.1.3	Einführung der Beispielreaktion: Verseifung von Triacetin	28
4.2	Import und Erkennung funktioneller Gruppen	35
4.2.1	Import-Pipelines	36
4.2.2	Substrukturanalyse mittels RDKit	42
4.2.3	SHACL-SPARQL-Abfrage	47
5	Ergebnisse	52
5.0.1	Aufzeigen des Imports	52
5.0.2	Ergebnisse der Substrukturanalyse	54
5.0.3	Reasoningergebnisse	56
5.0.4	SHACL-SPARQL-Abfrage	57
6	Diskussion	61
6.1	Bewertung des entwickelten Systems und der Ontologie	61
6.2	Herausforderungen, Limitationen und Modellierungsentscheidungen	62
6.2.1	Modellierungsentscheidungen und ihre Grenzen	62
6.2.2	Herausforderungen der chemischen Repräsentation und Analyse	63
6.2.3	Grenzen des ontologischen Modells & Datenqualität	64
7	Ausblick	66
	Referenzen	XII
	Anhang	XIII

1 Einleitung

Die effiziente Nutzung chemischer Reaktionsdaten aus heterogenen Quellen wie experimentellen Protokollen (z. B. via EnzymeML) und kuratierten Datenbanken (z. B. KEGG) wird durch mangelnde Standardisierung und fehlende semantische Verknüpfungen erschwert. Einfache, aber essentielle Wertschöpfungen wie die automatische Klassifizierung von Reaktionen anhand ihrer strukturellen und funktionellen Eigenschaften sind jedoch ein treibender Faktor für eine einheitliche Digitalisierung in der chemischen Forschung. Diese Arbeit stellt die Entwicklung einer automatisierten Datenverarbeitungspipeline vor, die dieses Problem adressiert, indem sie Reaktionsdaten in einen semantisch angereicherten Wissensgraphen transformiert und dessen Analyse ermöglicht.

Den methodischen Kern der Arbeit bildet diese Datenverarbeitungskette: Zunächst erfolgt die Datenerfassung und Standardisierung, bei der Reaktionsinformationen aus verschiedenen Quellen (EnzymeML, KEGG-API, manuelle Eingabe) aufgenommen und mit Hilfe der PubChem-API auf einheitliche Identifikatoren (IRIs) und Strukturdarstellungen (SMILES) abgebildet werden. Darauf aufbauend beinhaltet die Pipeline einen Schritt zur semantischen Anreicherung, in dem mittels RDKit eine Substrukturanalyse der SMILES-Codes durchgeführt wird, um funktionelle Gruppen zu identifizieren. Anschließend werden die standardisierten und angereicherten Daten mittels Owlready2 in eine erweiterte Ontologie (basierend auf RXNO) integriert, wodurch der Wissensgraph als zentrales Ergebnis der Pipeline entsteht. Dieser Graph bildet die Grundlage für die abschließende Stufe der automatisierten Analyse und Klassifikation: Es wird gezeigt, wie mittels OWL2-Reasoning (HermiT) sowie gezielter SPARQL-Abfragen und SHACL-Validierungen eine automatische Zuordnung zu Reaktionsklassen und weiterführende Analysen auf Basis der integrierten Daten und Ontologie-Axiome möglich werden.

Dazu werden in der vorliegenden Arbeit zunächst die theoretischen Grundlagen semantischer Technologien sowie der relevante Stand der Technik bei Antwortdatenbanken und Ontologien dargestellt. Darauf aufbauend wird die methodische Entwicklung der Ontologieverweiterungen und der Datenverarbeitungspipeline detailliert beschrieben; dies umfasst u. a. die Implementierung der Datenerfassung aus verschiedenen Quellen sowie die Substrukturanalyse mittels RDKit. Im Ergebnisteil wird die Funktionalität der Pipeline anhand von Beispielreaktionen demonstriert, wobei insbesondere die Ergebnisse der automatisierten Klassifikation durch Reasoning und der semantischen Anfragen mittels SPARQL/SHACL im Vordergrund stehen. Die Nutzbarmachung der Pipeline

für Forschende erfolgte durch die Entwicklung einer grafischen Benutzeroberfläche (GUI), welche ebenfalls vorgestellt wird. Eine kritische Diskussion der Stärken und Limitationen des entwickelten Ansatzes sowie ein Ausblick auf zukünftige Erweiterungsmöglichkeiten schließen die Arbeit ab.

2 Theoretischer Hintergrund

Um einen tieferen Einblick in die zugrunde liegenden Prinzipien zu ermöglichen, werden im Folgenden die für das Thema relevanten semantischen Instrumente beschrieben. Zunächst werden im Kapitel die semantischen Artefakte Resource Description Framework (RDF), Web Ontology Language (OWL2), SPARQL Protocol And RDF Query Language (SPARQL) sowie Shapes Constraint Language (SHACL) vorgestellt. Weiterhin wird einleitend die Datenverarbeitung und -analyse nach den FAIR-Richtlinien (Findability, Accessibility, Interoperability, and Reuse (FAIR)) dargelegt und dessen Relevanz anschließend im Kontext experimenteller Daten erläutert.

2.1 FAIR Data

Die FAIR-Prinzipien fungieren als Richtlinien für das Management und die Bereitstellung wissenschaftlicher Daten und stehen für die Aspekte „Findable“ (auffindbar), „Accessible“ (zugänglich), „Interoperable“ (interoperabel) und „Reusable“ (wiederverwendbar) [1]. Ihr Ziel besteht darin, zu gewährleisten, dass Forschungsdaten nicht nur für Menschen, sondern auch für Maschinen effizient nutzbar sind. Durch die Verwendung eindeutiger Identifikatoren, standardisierter Metadaten und offener Schnittstellen erleichtern die FAIR-Prinzipien die Vernetzung und Wiederverwendung von Daten über verschiedene Disziplinen hinweg. Ziel ist es, den wissenschaftlichen Fortschritt durch verbesserte Datenintegration, Transparenz und Nachnutzbarkeit zu fördern, insbesondere im Kontext von Open Science und digitaler Forschung [1]. Im Rahmen der vorliegenden Arbeit erfolgt die Berücksichtigung der FAIR-Prinzipien durch den Einsatz bestehender standardisierter Ontologien sowie gängiger Datenbanken. Die zunehmende Verbreitung von FAIR-Prinzipien in der wissenschaftlichen Community begünstigt die Entwicklung von Schnittstellen, die den Austausch und die Analyse experimenteller Reaktionsdaten über etablierte Formate wie EnzymeML ermöglichen. Dadurch lassen sich Daten strukturiert in wissensbasierte Systeme integrieren.

2.1.1 Experimentelle FAIR Daten

Mit dem zunehmenden Grad der Digitalisierung eröffnen sich immer mehr Möglichkeiten, experimentelle Forschungsdaten strukturiert zu verwalten und zu nutzen. In interdisziplinären Forschungsfeldern entstehen zunehmend gemeinsame Datenstandards und einheitliche Dokumentationsprakti-

ken, um die Vergleichbarkeit und Wiederverwendbarkeit von Daten zu verbessern. Während in den Laboren weiterhin individuelle Arbeitsabläufe und maßgeschneiderte Messtechnik eingesetzt werden, rücken interoperable Datenformate und standardisierte Erfassungsmethoden stärker in den Fokus. Insbesondere die Maschinenlesbarkeit gewinnt an Bedeutung, da immer mehr Initiativen darauf abzielen, experimentelle Details in strukturierten, maschinenverarbeitbaren Formaten bereitzustellen, wodurch Interoperabilität und Automatisierung zunehmend erleichtert werden [2].

2.2 Semantische Artefakte

Als semantische Artefakte werden digitale Instrumente bezeichnet, die dazu dienen, die Relationen zwischen verschiedenen Datenpunkten in einer maschinen- und menschenlesbaren Weise darzustellen [3]. Ontologien sind eines der wichtigsten semantischen Artefakte im Zusammenhang mit FAIR-Data [4, 5]. Ontologien sind komplexe Wissensgraphen und ermöglicht es neben der Darstellung von Daten, auch semantische Beziehungen abzubilden, indem Individuen mit Eigenschaften verschiedenen Typs durch Hierarchie- und Eigenschaftsbeziehungen zu einem logischen Wissensnetz verknüpft werden [3]. Die Maschinenlesbarkeit der semantischen Zusammenhänge ermöglicht eine digitale Verarbeitung der Daten und legt damit den Grundstein für eine Steigerung des Automatisierungsgrades. Darüber hinaus sind Ontologien und die darin enthaltenen Elemente in der Regel domänenspezifisch, weswegen verschiedene Ontologien aus unterschiedlichen Fachdomänen die gleichen Individuen mit unterschiedlichen, vom Kontext abhängigen Definitionen haben können [5]. Zu den im Rahmen dieser Arbeit relevanten semantischen Artefakten gehören unter anderem RDF und OWL2, welche im Folgenden vorgestellt werden.

2.2.1 RDF

Das Resource Description Framework (RDF) ist ein Framework zur Beschreibung von Informationen im Web, welches vom World Wide Web Consortium (W3C) entwickelt wurde. Die zu beschreibenden Ressourcen („resources“) können eine Vielzahl von Entitäten umfassen, darunter Dokumente, Personen, physische Objekte aber auch abstrakte Konzepte [6]. RDF dient der Darstellung von Daten in Form von sogenannten Tripeln, die aus den drei Einzelkomponenten Subjekt, Prädikat und Objekt zusammengesetzt sind. Das Framework bietet eine Vielzahl an Vorteilen, insbesondere wenn es um die strukturierte Darstellung, Verknüpfung und den Austausch von Daten geht. Ein zentraler Aspekt von RDF ist die Möglichkeit, maschinenlesbare Informationen direkt in Webseiten einzubinden. Darüber hinaus können diese maschinenlesbaren Daten von weiteren externen Software-Tools verarbeitet werden, wodurch sich neue Nutzungsmöglichkeiten ergeben, beispielsweise für personalisierte Dienste oder automatisierte Datenverarbeitung. Ein weiterer Vorteil von RDF ist die Möglichkeit, bestehende Datensätze mit externen Quellen zu verknüpfen. Dadurch erhält man nicht nur Zugang zu einer größeren Menge an Kontextinformationen über die Reaktionen

selbst, sondern auch zu weiterführenden, verknüpften Ressourcen. Dies erleichtert die Arbeit mit großen Datenmengen und ermöglicht es, neue Zusammenhänge zwischen Datensätzen herzustellen.

Das Subjekt beschreibt das Individuum oder die Ressource, über die eine Aussage getroffen wird. Das Prädikat beschreibt die Natur der Eigenschaftsbeziehung zwischen Subjekt und Objekt, während das Objekt den Wert der Eigenschaft oder ein weiteres Individuum beschreibt, welches mit dem Subjekt in einer Relation steht [7].

Während Subjekt und Objekt als sogenannte „*nodes*“ (Knoten) bezeichnet werden, wird das Prädikat auch als „*arc*“ (Bogen) genannt. Innerhalb der „*nodes*“ wird nochmals zwischen International Resource Identifier (IRI), literals und „*blank nodes*“ (Leere Knoten) unterschieden. Bekannte Individuen sind mit einer IRI hinterlegt, die als eindeutiger Identifikator für Individuen und Entitäten dient und eine Erweiterung der Uniform Resource Identifiers (URI) darstellt. Diese global eindeutigen IRIs ermöglichen unter anderem das ontologieübergreifende Auffinden und Integrieren der Individuen. Die „*literal nodes*“ hingegen geben lediglich eine Zeichenkette („*String*“) oder einen Zahlenwert an, dem zeitgleich ein bestimmter Datentyp zugewiesen ist (z.B integer, decimal etc.). Demzufolge handelt es sich bei literals innerhalb von RDF-Tripeln immer um Objekte, die dem Subjekt einen gewissen Wert zuweisen. Leere Knoten („*blank nodes*“) repräsentieren Ressourcen, die mit keiner eindeutigen IRI hinterlegt werden können und somit global auch nicht auffindbar sind [6, 7].

Im Folgenden wird eine durch RDF-Triple ausgedrückte Eigenschaftsbeziehung anhand eines Beispiels im Kontext chemischer Reaktionen erläutert. Wir betrachten beispielhaft in Abbildung 2.1 einen Reaktanden A, welcher an der Reaktion B als Edukt teilnimmt.



Abbildung 2.1: RDF-Triple von Reaktion B, welche den Reaktanden Substanz A enthält

In diesem Beispiel handelt es sich sowohl beim Subjekt, als auch beim Objekt um durch IRIs, eindeutig zu identifizierende Ressourcen, welche durch das Prädikat **hatReaktanden** in Beziehung gebracht werden.

Literal nodes ermöglichen es außerdem, Subjekte über Eigenschaften mit bestimmten Zahlenwerten oder Zeichenketten in Beziehung zu setzen. In Kontext chemischer Reaktionen könnte so beispiels-

weise die Stöchiometrie abgebildet werden.

Neben der theoretisch unendlichen Anzahl an verschiedenen Prädikaten, beinhaltet RDF noch einige Standardkonstrukte, die die Hierarchie, Klassifizierung und weitere gängige Eigenschaftsbeziehungen zwischen verschiedenen Elementen beschreiben. Diese sind in Tabelle 2.1 dargestellt.

Tabelle 2.1: RDF Schema Konstrukte und deren Beschreibung [6]

Konstrukte	Beschreibung
Klasse (<i>Klasse</i>)	C ist eine RDF-Klasse
Eigenschaft (<i>Klasse</i>)	P (<i>Ressource</i>) ist eine RDF-Eigenschaft
type (<i>Eigenschaft</i>)	I (<i>Ressource</i>) ist eine Instanz von C (<i>Klasse</i>)
subClassOf (<i>Eigenschaft</i>)	C1 (<i>Klasse</i>) ist eine Subklasse von C2 (<i>Klasse</i>)
subPropertyOf (<i>Eigenschaft</i>)	P1 (<i>Eigenschaft</i>) ist eine Sub-Eigenschaft von P2 (<i>Eigenschaft</i>)
Domäne (<i>Eigenschaft</i>)	Domäne von P (<i>Eigenschaft</i>) ist C (<i>Klasse</i>)
Reichweite (range) (<i>Eigenschaft</i>)	Reichweite von P (<i>Eigenschaft</i>) ist C (<i>Klasse</i>)

Die in 2.1 dargestellten RDF-Konstrukte bilden außerdem das Fundament für die in OWL2 genutzten Konstrukte, welche im Kapitel 2.2.2 weiterführend behandelt werden.

2.2.2 OWL2

OWL2 ist eine semantische Beschreibungssprache für das Web, die auf dem RDF basiert und dieses um eine zusätzliche semantische Schicht erweitert. Die Entwicklung von OWL erfolgte, genau wie bei RDF und SPARQL, durch das World Wide Web Consortium (W3C). OWL spielt eine zentrale Rolle im Semantic Web, da es die formale Strukturierung, Verknüpfung und maschinelle Interpretierbarkeit von Wissen ermöglicht [8].

Die Sprache bietet eine präzise Modellierung von Wissen durch Klassen, Individuen (Instanzen) und Eigenschaften. Es existieren mehrere Versionen von OWL, wobei OWL2, spezifiziert in der W3C-Empfehlung von 2012, die aktuellste und umfassendste Variante darstellt [8]. Die Erweiterung von RDF um zusätzliche semantische Konzepte und die Möglichkeit logischer Schlussfolgerungen (Reasoning) ermöglicht es digitalen Maschinen, aus bestehenden Daten neue Informationen abzuleiten. RDF selbst stellt lediglich eine Struktur bereit, mit der Daten in Subjekt-Prädikat-Objekt-Form dargestellt werden können. Durch zusätzliche semantische Einschränkungen und logische Ableitungen, auf die im weiteren Verlauf näher eingegangen wird, geht OWL2 über die Möglichkeiten von RDF hinaus [6, 7, 9].

Die Websprache OWL2 folgt der sogenannten Open World Assumption (OWA), die davon ausgeht, dass das Fehlen einer Information nicht zwangsläufig als Beweis für deren Falsifizierung angesehen werden kann. Demnach wird Unwissenheit nicht als Fehlbarkeit interpretiert, sondern als eine Form der Unkenntnis. Demgegenüber steht die Closed World Assumption (CWA), die die Prämisse vertritt, dass alle Aussagen, die nicht als wahr klassifiziert werden können, als falsch zu betrachten sind. Diese Annahme findet häufig in Systemen Anwendung, die mit vollständigen Informationen als Basis operieren, wie es beispielsweise in einer Vielzahl von Datenbankanwendungen der Fall ist [9].

Im Folgenden werden zentrale Funktionalitäten und Elemente der semantischen Struktur von OWL2 erläutert. Dazu gehören vor allem die Klassen, Individuen (Instanzen) sowie Anmerkungen (annotations) und Eigenschaftszuweisungen (properties) verschiedener Art [10].

Klassen

OWL-Klassen sind Mengen, die Individuen bzw. Instanzen enthalten und durch formale mathematische Beschreibungen definiert werden. Diese Beschreibungen legen exakt fest, welche Bedingungen für die Zugehörigkeit zu einer Klasse erfüllt sein müssen. Klassen sind in der Regel in einer Hierarchie aus Ober- und Unterklassen organisiert, welche auch als Taxonomie bekannt ist. Innerhalb dieser Taxonomie werden Eigenschaften von Superklassen an die jeweiligen Subklassen übertragen, ein Vorgang, der als „Vererbung“ bezeichnet wird. Eine Vererbung in umgekehrter Richtung, von Subklasse zu Superklasse, ist jedoch nicht mit der OWL-Logik vereinbar. Zur Veranschaulichung sei die Klassenhierarchie zwischen der Subklasse `einwertiger Alkohol` und der Superklasse `Alkohol` herangezogen: `einwertiger Alkohol` weist alle Eigenschaften eines Alkohols auf, wobei jedoch nicht jeder Alkohol die gleichen Eigenschaften wie ein einwertiger Alkohol aufweist. Im Gegensatz zu herkömmlichen Baumstrukturen erlaubt OWL2 das Zuordnen einer Subklasse zu mehreren Superklassen. Dadurch wird eine sogenannte Mehrfachvererbung ermöglicht, das heißt die Vererbung von Eigenschaften aus unterschiedlichen Superklassen. Dies erlaubt eine intuitive Modellierung komplexer Zusammenhänge. Im chemischen Kontext ermöglicht dies beispielsweise einerseits die Zuordnung von Natronlauge (NaOH) zur Klasse der Basen, aber zeitgleich auch zur Klasse der Elektrolyte, ohne dabei in einen Widerspruch zu geraten bzw. einen Informationsverlust durch einseitige Vererbung zu erleiden. Unterklassen spezialisieren ihre Oberklassen, wodurch alle Angehörigen einer Unterklasse auch Mitglieder ihrer Oberklasse sind. Diese sogenannten Subsumptionsbeziehungen können automatisch von einem Reasoner berechnet werden, dessen Funktionsweise im folgenden Kapitel 2.2.2 noch erläutert wird. OWL und die Software Protégé nutzen hierfür eine formale Sprache namens Description Logic (DL), die solche logischen Schlussfolgerungen ermöglicht [10].

Weiterhin wird unter den Klassen zwischen undefinierten (primitiven) und definierten Klassen differenziert. Primitive Klassen beinhalten lediglich notwendige Bedingungen, sodass eine Instanz nicht

automatisch als Mitglied dieser Klasse identifiziert wird, selbst wenn sie die Bedingungen erfüllt. Hier findet also die OWA Anwendung. Es wird davon ausgegangen, dass noch Informationen fehlen könnten, weswegen keine automatisierte Zuordnung erfolgen kann. Definierte Klassen hingegen beinhalten in jedem Fall notwendige und optional hinreichende Bedingungen. Erfüllt eine Instanz alle Bedingungen einer definierten Klasse, wird sie automatisch als Mitglied dieser Klasse klassifiziert. Definierte Klassen unterliegen also nicht länger der OWA. Die Umwandlung einer Klasse von primitiv zu definiert erfolgt durch die Angabe der Bedingungen unter **Equivalent to** statt unter **SubClassOf**, was zudem eine komplexere Beschreibung der Klassenrestriktionen ermöglicht. Diese Vorgehensweise ermöglicht es dem Reasoner (s. Abschnitt 2.2.2), automatisch neue Klassenzugehörigkeiten abzuleiten [8, 10].

Individuen/Instanzen

Während Klassen eine Menge beschreiben, handelt es sich bei Individuen um die einzelnen Elemente innerhalb dieser Menge. Individuen fungieren als Repräsentanten von Objekten im spezifischen Anwendungsbereich. Ein wesentlicher Unterschied zwischen OWL und den meisten Programmiersprachen bzw. Wissensrepräsentationssprachen besteht darin, dass OWL die Unique Name Assumption (UNA) nicht nutzt. Die Prämisse, dass verschiedene Namen verschiedene Individuen bezeichnen, wird in OWL nicht a priori angenommen [8]. Diese Eigenschaft erweist sich insbesondere in Bezug auf Semantic-Web-Anwendungen als vorteilhaft. In solchen Kontexten ist es möglich, dass unterschiedliche Organisationen unbeabsichtigt dieselbe Entität unter verschiedenen Namen erfassen. Um dies zu verhindern, ist es erforderlich, eine explizite Ausnahme zu definieren, die eindeutig festlegt, dass zwei Namen nicht dasselbe Individuum repräsentieren [8, 10].

Des Weiteren ist es auch möglich, dass verschiedene Elemente innerhalb der Ontologie den selben Namen zugewiesen bekommen, trotz dessen aber verschiedene Elemente darstellen. Ein Beispiel hierfür wäre eine Instanz **Glycerin**, welche zur gleichnamigen Klasse **Glycerin** gehört. Die Unterscheidung zwischen den verschiedenen Elementen erfolgt durch die Angabe der individuellen IRI. Diese dient als eindeutiger Identifikator und verhindert so Verwechslungen [8, 11]. Wie im vorigen Abschnitt 2.2.2 schon angeklungen ist, können Instanzen über Eigenschaften mit anderen Instanzen bzw. Klassen in binäre Beziehungen gebracht werden.

Eigenschaften

Im Rahmen von OWL2 ermöglichen Eigenschaften (properties) den Aufbau von binären Relationen zwischen Instanzen bzw. zwischen Klassen und Instanzen. Wie bereits im RDF-Kapitel 2.2.1 eingeführt, basieren die Property-Beziehungen in OWL auf RDF-Konstrukten und verknüpfen somit immer ein Subjekt über eine Eigenschaft (Prädikat) mit einem Objekt, so dass auch hier Tripel auftreten. In Analogie zu den Relationen innerhalb der RDF-Tripel, bei denen vom Subjekt ausge-

hend entweder eine Relation zu einem bekannten, mit einer IRI versehenen Objekt oder zu einem „literal node“, also einem String oder Zahlenwert, aufgebaut werden kann, können in OWL ebenso verschiedene Arten von Eigenschaftsbeziehungen aufgebaut werden. Im Rahmen der Web Ontology Language wird zwischen Objekteigenschaften (object properties), Dateneigenschaften (data properties) und Annotationseigenschaften (annotation properties) unterschieden. Die Modellierung der Relationen innerhalb von Instanzen oder Klassen respektive zwischen Klassen und Instanzen wird durch Objekteigenschaften vorgenommen [10]. Demnach handelt es sich bei der exemplarischen Eigenschaft `hatReaktanden` aus dem Beispiel in Abbildung 2.1 um eine Objekteigenschaft. Dateneigenschaften fungieren demgegenüber als Mittel, um Relationen zwischen Instanzen und konkreten Datenwerten, wie beispielsweise Zeichenketten oder numerischen Werten, zu etablieren und bilden daher das direkte Analogon zu RDF-Tripeln, bei denen das Objekt ein „literal node“ ist [8, 10]. Als Beispiel lässt sich hier die Eigenschaft `hasMolarMass` anführen, welche das molare Gewicht einer chemischen Substanz in Form eines Zahlenwerts repräsentiert. Im Kontext chemischer Reaktionen ermöglicht diese Funktionalität unter anderem auch die Modellierung stöchiometrischer Verhältnisse. Annotationseigenschaften hingegen dienen der Anreicherung von Ontologie-Elementen mit Metadaten, wie etwa der Vergabe von Bezeichnungen, Definitionen oder Beschreibungen, werden aber nicht von der Logik betrachtet und können so also auch nicht vom Reasoner zum Schlussfolgern herangezogen werden. Eine häufig verwendete Annotationseigenschaft ist in diesem Zusammenhang `rdfs:label`, mittels derer menschenlesbare Namen für Entitäten hinterlegt werden können. Im chemischen Kontext sind zusätzliche Angaben wie Synonyme, Stoffdaten, Strukturformel und die SMILES-Darstellung gängige Annotationseigenschaften, welche die Auffindbarkeit und Interoperabilität der Ontologie erhöhen. Eine differenzierte Kategorisierung ermöglicht OWL eine präzise semantische Modellierung und erleichtert die automatische Verarbeitung und Schlussfolgerung innerhalb von Ontologien.

Genau wie die OWL-Klassen, können auch Properties bestimmten Axiomen unterstehen. Dazu gehört beispielsweise die `domain` welche angibt aus welcher Superklasse das Subjekt kommt, die `range`, die die Zielklasse, also die Klasse möglicher Objekte, definiert. Des Weiteren lassen sich für Eigenschaften auch Invers- sowie Subeigenschaften definieren [10]. In Tabelle 2.2 sind die verschiedenen Charakteristiken für Eigenschaften dargestellt:

Axiome & Restriktionen

In der Ontologie-Sprache OWL2 stellen Axiome die grundlegenden Aussagen dar, aus denen eine Ontologie besteht. Axiome repräsentieren die elementaren Wissensbausteine und definieren, welche Fakten oder Regeln innerhalb der Ontologie als wahr angenommen werden. Die Idee von OWL2 besteht darin, Wissen als eine Sammlung solcher Aussagen zu formulieren, wobei eine Ontologie genau diese Axiome enthält und ihre Gültigkeit beansprucht. Die im Abschnitt 2.2.2 erläuterte Beziehung `subclass Of` und die in Abschnitt 2.2.2 definierten Charakteristiken gehören beispielsweise zu diesen

Tabelle 2.2: Eigenschaftsmerkmale und deren Bedeutung

Merkmal	Bedeutung
Functional	Eine functional-Eigenschaft bedeutet, dass jedes Individuum über diese Eigenschaft mit höchstens einem anderen Individuum verknüpft sein kann.
Inverse functional	Eine inverse functional-Eigenschaft bedeutet, dass die inverse dieser Eigenschaft eine functional-Eigenschaft ist, also ebenfalls nur eine eindeutige Verbindung zulässt.
Transitive	Eine transitive Eigenschaft bedeutet, dass wenn ein Individuum mit einem zweiten über diese Eigenschaft verbunden ist und das zweite mit einem dritten, dann ist auch das erste mit dem dritten über dieselbe Eigenschaft verbunden.
Symmetric	Eine symmetrische Eigenschaft bedeutet, dass wenn ein Individuum mit einem zweiten verbunden ist, das zweite automatisch auch mit dem ersten über dieselbe Eigenschaft verbunden ist.
Asymmetric	Eine asymmetrische Eigenschaft bedeutet, dass wenn ein Individuum mit einem zweiten verbunden ist, das zweite nicht mit dem ersten über dieselbe Eigenschaft verbunden sein darf.
Reflexive	Eine reflexive Eigenschaft bedeutet, dass jedes Individuum über diese Eigenschaft mit sich selbst verbunden ist.
Irreflexive	Eine irreflexive Eigenschaft bedeutet, dass kein Individuum über diese Eigenschaft mit sich selbst verbunden ist.

Axiomen. Im Kontext von OWL-Klassen lassen sich weiterhin noch Axiome bezüglich der Beziehung von Klassen untereinander formulieren. So lassen sich u. a. Klassen als disjunkt definieren. Disjunkte, also „einander ausschließende“, Klassen können niemals ein gemeinsames Individuum enthalten. So wäre es beispielsweise sinnvoll, Klassen für einwertige und zweiwertige Alkohole als disjunkt zu betrachten, da immer nur einer dieser Zustände wahr sein kann. Genau so können disjunkte Klassen weder zu einer Vereinigung gebildet, noch als Komplementärklassen definiert werden. Komplementärklassen hierbei beschreiben die logische Negation einer anderen Klasse [8].

Eine der fundamentalen Axiome im Rahmen von OWL2 ist das sogenannte **Equivalent to-Axiom**, welches für die Erstellung definierter Klassen essenziell ist. Dieses Axiom gestattet die Definition von Klassenrestriktionen in einer Form, die bei Erfüllung aller Bedingungen eine erfolgreiche Zuordnung durch den Reasoner ermöglicht, ohne dass dies in Konflikt mit der OWA gerät. Die Klassenrestriktionen stellen Anforderungen dar, die ein Individuum erfüllen muss, um der Klasse zugeordnet zu werden. Diese Anforderungen sind als Eigenschaften formuliert, die das entsprechende Individuum aufweisen muss. Kardinalitätsbeschränkungen ermöglichen hierbei die Modellierung komplexerer Zusammenhänge, welche im nächsten Unterkapitel diskutiert werden.

Kardinalitätsrestriktionen

In OWL2 besteht die Möglichkeit, die Anzahl an Beziehungen, die eine Klasse bzw. Instanz hat, genauer zu definieren. Für die Formulierung einer weniger rigorosen Restriktion, reicht es aus, die Eigenschaftsbeziehung mit „some“ festzulegen, für den Fall, dass nicht die Häufigkeit, sondern einzig und allein das Vorhandensein dieser Eigenschaft von Relevanz ist. Mittels der zusätzlichen Operatoren „exactly“, „min“ und „max“ kann die Anzahl an Beziehungen limitiert bzw. auf einen exakten Wert festgelegt werden [8]. Ein Beispiel hierfür wäre die Klassenrestriktionen für die Klasse der **dreiwertigen Alkohole**. Bei dieser könnte z. B. gefordert werden, dass alle Teilnehmer genau 3 Hydroxygruppen enthalten (**hat_funktionalen_anteil exactly 3 hydroxy_gruppe**). Dadurch wird die Definition von ähnlichen Klassen auf einer präzisen Ebene ermöglicht, ohne dass eine Vielzahl einzelner Eigenschaftsbeziehungen definiert werden muss.

Automatisches Schlussfolgern/Automated Reasoning

Wie bereits mehrfach erwähnt, ist eine zentrale Funktion von OWL2 das sogenannte Reasoning bzw. das automatische Schlussfolgern. Auf Basis der definierten Klassenrestriktionen, -axiome und -eigenschaften ist der Reasoner in der Lage, die maschinenlesbaren Informationen zu verarbeiten und darauf aufbauend neue Relationen und Klassenzugehörigkeiten zu inferieren. Darüber hinaus überprüft ein Reasoner die Ontologie auch auf Inkonsistenzen und logische Fehler innerhalb der Modellierung der Axiome und Restriktionen. In der OWL2 DL gehören „Fact++“, „Hermit“ sowie „Pellet“ zu den häufig eingesetzten Reasonern [8].

2.2.3 SPARQL

Bei der SPARQL Protocol And RDF Query Language (SPARQL) handelt es sich um eine vom W3C standardisierte Abfragesprache für RDF-Daten. Sie ermöglicht die Abfrage von Informationen aus unterschiedlichen Datenquellen, unabhängig davon, ob die Daten nativ als RDF gespeichert sind oder über eine Vermittlungssoftware (Middleware) als RDF dargestellt werden. Bei Nutzung von SPARQL können semantische Beziehungen mithilfe von verkürzter Präfix-Schreibweise dargestellt werden, was gerade bei komplexeren Abfragen und beim Zusammenführen mehrerer Datenbanken den Umfang der Abfrage deutlich verringern kann [12, 13]. Die Funktionsweise von SPARQL basiert auf dem Matching von Basic Graph Pattern (BGP), welche eine Menge von Triple Patterns beinhalten. Als Triple Patterns bezeichnet man Musterstrukturen mit RDF-Triple-ähnlichem Aufbau der Art Subjekt-Prädikat-Objekt. Dabei können die einzelnen Elemente innerhalb der Triple entweder explizit angegeben werden, oder aber auch generischer mithilfe von Variablen formuliert werden. Die Variablen werden innerhalb der Abfrage durch den Präfix „?“ markiert und können sowohl Klassen, Individuen als auch Eigenschaften beinhalten. Ein BGP resultiert also aus der Abfolge mehrerer Triple Patterns. Erst bei erfolgreichem Matching aller im BGP enthaltenen Triple Patterns ist die SPARQL-Query erfolgreich. Während des Matching Prozesses werden die BGP mit den vorhandenen RDF-Daten abgeglichen und die darin enthaltenen Variablen so angepasst, dass die Musterstruktur mit einem Subgraphen der RDF-Daten übereinstimmt. Die relevanten Daten, die zum erfolgreichen Matching geführt haben, werden daraufhin in diesen Variablen gespeichert. Zur Formulierung komplexerer Abfragen können Group Pattern Matches formuliert werden, die BGPs um zusätzliche logische Regeln und Bedingungen erweitern [12, 13].

Die Ergebnisse von SPARQL-Abfragen können entweder Ergebnismengen oder RDF-Graphen sein. Die folgende Abbildung zeigt die grundlegende Struktur einer SPARQL-Abfrage:

Prefix Area	{ PREFIX obo: <http://purl.obolibrary.org/obo/> PREFIX ex: <http://example.org/chemicals#>
Query Header	{ SELECT DISTINCT ?variable_1 ?variable_2
Query pattern	{ WHERE { ?variable_1 a obo:MOP_0000543 ; obo:BFO_0000078 ?object_1 ; obo:BFO_0000077 ?object_2 . FILTER NOT EXISTS { ?variable_2 ex:isRelevant true } }
Query modifiers	{ ORDER BY ?variable_1

Abbildung 2.2: Grundstruktur einer SPARQL-Abfrage

Wie der Abbildung 2.2 zu entnehmen ist, beginnt jede SPARQL-Abfrage mit der Angabe der

Präfixe. Die Präfixe erlauben es, die IRIs innerhalb der Abfrage abzukürzen und erhöhen so die Übersichtlichkeit. Nach den Präfixen folgt der Query-Header, der die Art der SPARQL-Abfrage angibt und festlegt, welche Variablen innerhalb der Abfrage genutzt werden. Mögliche Abfragetypen sind unter anderem **CONSTRUCT**, **DESCRIBE**, **SELECT** und **ASK**. Der Kern der Abfrage ist das Query-Pattern. Innerhalb der **WHERE**-Klausel werden diejenigen Tripelpaare definiert, die innerhalb der Abfrage aufgefunden werden sollen. Durch zusätzliche Klauseln kann die Abfrage weiter spezifiziert werden. So können z. B. nur Ergebnisse, die einem bestimmten Muster entsprechen, durch Klauseln wie **FILTER EXISTS** oder **FILTER NOT EXISTS** ausgegeben bzw. aus dem Ergebnisgraphen entfernt werden. Weitere SPARQL-Klauseln, die im Rahmen dieser Arbeit Anwendung finden, sind in der nachfolgenden Tabelle 2.3 dargestellt.

Tabelle 2.3: SPARQL Abfrage-Klauseln und deren Beschreibung

Klausel	Beschreibung
SELECT	Die SELECT-Klausel wird im Abfrage-Header verwendet und gibt alle oder eine Teilmenge der Variablen in Tabellenform zurück, die durch ein Query Pattern eingebunden sind.
CONSTRUCT	Die CONSTRUCT-Klausel wird im Abfrage-Header verwendet und liefert einen RDF-Graphen zurück, der durch Ersetzen von Variablen in einer Reihe von Triple-Vorlagen erstellt wird.
FILTER	Die FILTER-Klausel wird im Abfrage-Musterbereich verwendet und schränkt die Ergebnisse auf jene Lösungen ein, für die der Filterausdruck als wahr ausgewertet wird.
FILTER EXISTS	Die FILTER EXISTS-Klausel wird im Abfrage-Musterbereich verwendet und entfernt Ergebnisse, die einem bestimmten Muster entsprechen, aus der Ergebnisliste.
FILTER NOT EXISTS	Die FILTER NOT EXISTS-Klausel wird im Abfrage-Musterbereich verwendet und filtert die Ergebnisse so, dass nur solche zurückgegeben werden, bei denen ein bestimmtes Muster nicht im Ergebnisgraphen vorhanden ist.
DISTINCT	Die DISTINCT-Klausel ist ein Abfragemodifikator, der doppelte Ergebnisse entfernt und nur eindeutige Einträge zurückliefert.

2.2.4 SHACL

Ähnlich wie OWL2 und SPARQL ist auch SHACL eine vom W3C standardisierte Websprache im Umgang mit RDF-Daten. Im Gegensatz zu SPARQL ist die primär angedachte Funktion bei SHACL nicht die gezielte Abfrage einzelner Informationen, sondern die Überprüfung der strukturellen Korrektheit eines RDF-Datenmodells. SHACL liefert in der Regel keinen Ergebnisgraphen, sondern einen Validierungsbericht, der aufzeigt, ob bestimmte Anforderungen erfüllt wurden oder nicht. Zur Formulierung dieser Anforderungen bedient sich SHACL sogenannter „Shapes“, welche selbst als RDF-Graphen dargestellt werden. Ein SHACL-Validator überprüft, ob ein gegebener RDF-Daten-Graph (Data Graph) den im Shapes-Graph definierten Bedingungen entspricht. Dies gewährleistet die Datenintegrität und erleichtert die Interoperabilität zwischen verschiedenen Systemen [14]. Der Prozess der Datenvalidierung via SHACL ist in Abbildung 2.3 veranschaulicht:

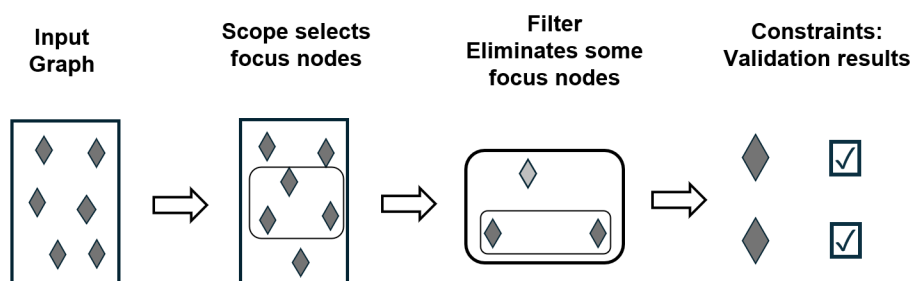


Abbildung 2.3: Datenvalidierung via SHACL

Wie der Abbildung 2.3 zu entnehmen ist, liegt zu Beginn die Gesamtheit der RDF-Daten vor und es wird dabei, anders als in der OWL, von einer CWA ausgegangen [14]. In einem ersten Schritt werden durch Definition der „Shapes“ die im Geltungsbereich („Scope“) liegenden Datenpunkte identifiziert. Dabei unterscheidet man zwischen den sog. „Node-Shapes“ und den „Property-Shapes“. Dabei prüfen Node Shapes die Korrektheit der Klassen, während Property Shapes die Korrektheit und Einschränkungen von Relationen zwischen Individuen bzw. zwischen Klassen und deren Eigenschaften validieren. Innerhalb von Node Shapes wird beispielsweise definiert, welcher Zielklasse die zu betrachtenden Knotenpunkte entstammen. Im einem nächsten Schritt kommen die Property Shapes hinzu. Mithilfe von „Property-Shapes“ können darüber hinaus die Eigenschaftsbeziehungen weiter spezifiziert werden. Beispielsweise können Einschränkungen definiert werden, die die Anzahl an Relationen überprüft. So könnten also im Kontext chemischer Reaktionen z. B. nur Datenpunkte herausgefiltert werden, die jeweils mindestens eine Relation zu einem Edukt bzw. Produkt aufweisen, um sicherzustellen, dass es sich um valide Reaktionen handelt. Zuletzt werden die übrigen Datenpunkte auf die vordefinierten Bedingungen (Constraints) geprüft und das Ergebnis der Prüfung ausgegeben [14].

3 Aktueller Stand der Technik

In der Fachdomäne der Chemie sind der Austausch und die Zugänglichkeit von chemischem Wissen zentrale Herausforderungen, denen mit der Entwicklung von Methoden zur Verbesserung der Auffindbarkeit (Findability) und Interoperabilität (Interoperabilität) chemischer Daten begegnet werden soll. In diesem Kapitel werden die derzeit existierenden Ansätze vorgestellt, die zur Lösung dieser Herausforderungen beitragen.

Zunächst werden öffentliche chemische Datenbanken wie PubChem, ChEBI und KEGG vorgestellt, die als zentrale Wissensquellen für chemische Strukturen und Reaktionsmechanismen dienen. Dabei wird insbesondere ChEBI als Brücke zwischen Datenbanken und ontologischen Systemen betrachtet, da es strukturelle Informationen mit semantischer Modellierung verbindet. Ein wesentlicher Aspekt ist hier die ontologische Darstellung chemischer Konzepte, wie sie in Name Reaction Ontology (RXNO) oder Molecular Process Ontology (MOP) angewendet wird. Im weiteren Verlauf werden bestehende Methoden zur Steigerung der Interoperabilität chemischer Daten diskutiert. Dazu gehören die Nutzung von Programmierschnittstellen (APIs) wie die PubChem API, die eine automatisierte Abfrage und Analyse chemischer Daten im großen Maßstab ermöglicht. Standards wie EnzymeML sorgen für eine einheitliche Formatierung von experimentellen Daten und erleichtern deren Austausch in der wissenschaftlichen Community. Darüber hinaus spielen maschinenlesbare chemische Strukturformate wie SMILES, SMARTS und InChI eine entscheidende Rolle, um chemische Informationen eindeutig darzustellen und sowohl für Menschen als auch für Maschinen verständlich zu machen.

Um die Potenziale und Grenzen bestehender Methoden zu erfassen, ist ein fundiertes Verständnis derselben unabdingbar. Aufbauend auf diesen Erkenntnissen werden im weiteren Verlauf dieser Arbeit neue Ansätze entwickelt, um bestehende Einschränkungen zu überwinden und die chemische Wissensrepräsentation weiterzuentwickeln.

3.1 Datenbanken & Ontologien

Im Zuge der Optimierung des Austauschs von chemischem Wissen wurden umfangreiche Datenbanken sowie Ontologien konzipiert. Datenbanken dienen der Speicherung chemischer Strukturen, Reaktionen und Eigenschaften, während Ontologien eine semantische Modellierung und logische

Verknüpfung dieser Informationen ermöglichen. Ontologien definieren formale Konzepte und deren Beziehungen zueinander, wodurch Wissen strukturiert und für Maschinen interpretierbar wird [15]. Die nachfolgenden Abschnitte präsentieren zentrale chemische Datenbanken sowie relevante Ontologien in detaillierter Form.

3.1.1 PubChem

PubChem ist eine chemische Datenbank, welche vom National Center for Biotechnology Information (NCBI) der U.S. National Library of Medicine (NLM) betrieben wird. Ihre Entwicklung zielt auf die Schaffung einer zentralen Plattform für chemische Informationen ab, welche Forschenden umfassende Daten zu chemischen Verbindungen, ihren Eigenschaften, biologischen Aktivitäten und Sicherheitsinformationen bietet. Die Datenbank umfasst Millionen von Einträgen zu kleinen Molekülen, Proteinen und biochemischen Reaktionen und wird kontinuierlich durch Beiträge aus wissenschaftlichen Publikationen, Patenten und anderen Datenquellen erweitert. Neben einer benutzerfreundlichen Weboberfläche stellt PubChem auch eine API bereit, die eine automatisierte Abfrage und Analyse chemischer Daten ermöglicht. Die Verwendung standardisierter Formate wie Simplified Molecular Input Line Entry System (SMILES) und International Chemical Identifier (InChI) trägt zur verbesserten Interoperabilität und Findability chemischer Daten in der wissenschaftlichen Forschung bei [16].

3.1.2 ChEBI

Chemical Entities of Biological Interest (ChEBI) ist eine frei zugängliche Datenbank biologisch relevanter chemischer Verbindungen, die vom European Bioinformatics Institute (EBI) betrieben wird. Sie enthält detaillierte Informationen über molekulare Einheiten, einschließlich ihrer strukturellen Eigenschaften, chemischen Rollen und biologischen Funktionen. Im Gegensatz zu allgemeinen chemischen Datenbanken legt ChEBI besonderen Wert auf die standardisierte Beschreibung und Klassifizierung von Molekülen, wobei die Nomenklatur der International Union of Pure and Applied Chemistry (IUPAC) und der Nomenclature Committee of the International Union of Biochemistry and Molecular Biology (NC-IUBMB) verwendet wird. Zusätzlich ist die ChEBI mit einer Vielzahl weiterer Datenbanken verknüpft und enthält Referenzen zu weiteren Erwähnungen in anderer Fachliteratur. Eine Besonderheit der ChEBI ist seine ontologische Struktur, die chemische Entitäten hierarchisch ordnet und semantische Beziehungen zwischen ihnen definiert. Dadurch können Verbindungen nicht nur nach ihrer chemischen Struktur, sondern auch nach funktionellen und biologischen Kriterien kategorisiert werden. Die ChEBI enthält mehr als 220.000 Klassen und 47 Eigenschaftsbeziehungen [17].

3.1.3 KEGG

Bei der Kyoto Encyclopedia of Genes and Genomes (KEGG) handelt es sich ebenfalls um eine Datenbank mit Fokus auf biochemische Prozesse. KEGG ist heutzutage eine der wichtigsten öffentlichen Datenbanken für Systembiologie und Biochemie. Eine Besonderheit der KEGG liegt in ihrem modularen Aufbau, der verschiedene spezialisierte Datenbanken miteinander verknüpft. Darunter beispielsweise KEGG Pathway, KEGG Compound und KEGG Reaction. Die im Rahmen dieser Arbeit vorrangig relevanten Bibliotheken „KEGG Reaction“ und „KEGG Compound“ umfassen 12.212 biochemische, hauptsächlich enzymatische Reaktionen und 19.437 chemische Substanzen [18].

3.1.4 RHEA

Auch die RHEA enthält hauptsächlich biochemische Reaktionen. Die Einträge werden händisch kuratiert. Eine weitere Besonderheit ist, dass alle Rhea-Reaktionen hinsichtlich Masse und Ladung ausgeglichen sind. Die Rhea nutzt zur Beschreibung der an der Reaktionen teilnehmenden Komponenten außerdem ausschließlich die ChEBI und enthält weiterhin viele Kreuzreferenzen zu weiteren biochemischen Bibliotheken, darunter auch die zuvor genannte KEGG [19]. Derzeit umfasst das Rhea-Verzeichnis 17.422 Reaktionen sowie 14.335 chemische Verbindungen [20].

3.1.5 BFO

Die Basic Formal Ontology (BFO) stellt eine Top-Level-Ontologie dar, die mit der Intention entwickelt wurde, die Interoperabilität zwischen verschiedenen Domänenontologien zu fördern. Entwickelt wurde BFO u. a. von Barry Smith und der Open Biological and Biomedical Ontologies (OBO)-Foundry. Sie dient als Grundlage für zahlreiche Ontologieprojekte, insbesondere im biomedizinischen Bereich. Im Jahr 2021 erfolgte die Anerkennung von BFO als internationaler Standard gemäß ISO/IEC 21838-2 [21]. Der Fokus der OBO-Foundry zielt auf die Schaffung einer Familie interoperabler Ontologien ab, die sowohl logische Konsistenz als auch wissenschaftliche Präzision aufweisen. Um dieses Ziel zu erreichen, verpflichteten sich die Teilnehmer der OBO Foundry zur Einhaltung einer Reihe von Prinzipien und tragen aktiv zu deren Weiterentwicklung bei. Dazu gehören u. a. die offene Nutzung, die kollaborative Entwicklung, die Trennung von Inhalten sowie die Verwendung gemeinsamer Syntax und Relationen zur semantischen Beschreibung [22]. Die BFO beinhaltet 35 Klassen sowie 22 Eigenschaftsbeziehungen [23]. Zur OBO-Ontologiefamilie gehören unter anderem auch die im nachfolgenden Kapitel besprochenen Ontologien RXNO und MOP [24, 25].

3.1.6 MOP & RXNO

Die MOP stellt eine Ontologie dar, die zur semantischen Strukturierung von molekularen Prozessen sowie Wechselwirkungen entwickelt wurde. Die Ontologie umfasst die molekularen Prozesse, die der Reaktionsontologie RXNO zugrunde liegen, wie beispielsweise Cyclisierung, Methylierung und Demethylierung. Derzeit enthält die MOP etwa 3686 Klassen sowie 37 Eigenschaften [24]. Die RXNO ist eine von der Royal Society of Chemistry (RSC) entwickelte Ontologie, die zur Klassifikation chemischer Reaktionen dient und so die präzise Zuordnung von Reaktionen anhand von Mechanismen, Reaktionsklassen und experimentellen Bedingungen ermöglicht. Die RXNO umfasst 1019 Klassen und 14 Objekteigenschaften, die eine Hierarchie zwischen chemischen Substanzen und Reaktionen sowie chemischen Gruppen aufbauen [25]. Der Fokus innerhalb von RXNO liegt auf organischen Synthesereaktionen und Namensreaktionen, wie z. B. der Diels-Alder- oder Grignard-Reaktion [26, 27]. Die Ontologie basiert auf etablierten Klassifikationssystemen wie dem Merck Index und der Hierarchie von Carey, Laffan, Thomson und Williams [26].

3.1.7 EnzymeML & pyEnzyme

EnzymeML stellt ein standardisiertes, maschinenlesbares Datenformat zur Speicherung und Verarbeitung enzymkinetischer Experimente dar. Das Format beinhaltet Informationen über Reaktionen, Edukte, Produkte, Enzyme sowie experimentelle Messwerte und Metadaten zu Bedingungen wie Temperatur oder pH-Wert. Basierend auf den Formaten XML und JSON ermöglicht EnzymeML eine strukturierte und interoperable Speicherung, wodurch der Austausch zwischen verschiedenen Softwareanwendungen erleichtert wird. Dies verbessert die automatisierte Analyse und Modellierung enzymatischer Prozesse und fördert die Interoperabilität mit bestehenden Datenbanken. Trotz dieser Vorteile wird EnzymeML noch nicht flächendeckend genutzt, da die Integration in bestehende Systeme und die korrekte Annotation der Daten zusätzliche Herausforderungen darstellen. Dennoch leistet EnzymeML einen Beitrag zur Umsetzung der FAIR-Prinzipien und bildet eine vielversprechende Grundlage für die semantische Modellierung enzymatischer Reaktionen [28, 29]. Zur Arbeit mit EnzymeML ist die Python-Erweiterung pyEnzyme im Rahmen dieser Thesis ein unerlässliches Tool. Der Zweck dieser Erweiterung wird im weiteren Verlauf im Abschnitt 3.2.6 erläutert.

3.2 Software

Im Rahmen der Entwicklung, Analyse und Validierung chemischer Ontologien sowie der maschinenlesbaren Verarbeitung chemischer Strukturen finden diverse Softwarelösungen und Programmbibliotheken Anwendung. Die eingesetzten Tools unterstützen dabei die Modifikation von Ontologien, die Datenvalidierung und die automatisierte Analyse chemischer Informationen.

Protégé, eine weit verbreitete Software zur Bearbeitung von OWL- und RDF-Ontologien, ermöglicht

eine visuelle Modifikation von Ontologieklassen und Relationen und stellt daher einen zentralen Bestandteil dieser Arbeit dar. Ergänzend dazu werden Python-basierte Bibliotheken wie Owlready2 und pySHACL genutzt, um Ontologien mithilfe von Python zu modifizieren und SHACL-basierte Validierungen und Abfragen durchzuführen. Für die Verarbeitung chemischer Strukturen und die Erkennung funktioneller Gruppen wird RDKit eingesetzt, das als Grundlage für weiterführende Erweiterungen wie Extended Functional Groups (EFGs) dient.

Im Folgenden werden die eingesetzten Softwarelösungen und Bibliotheken näher erläutert.

3.2.1 Protégé

Protégé ist eine weit verbreitete, Open-Source-Entwicklungsumgebung, die zur Modellierung und Bearbeitung von Ontologien in den Formaten OWL (Web Ontology Language) und RDF verwendet wird. Entwickelt am Stanford Center for Biomedical Informatics Research, zeichnet sich Protégé durch eine grafische Benutzeroberfläche aus, die eine intuitive Erstellung, Strukturierung und Verwaltung von Ontologien ermöglicht [30].

Ein zentraler Vorteil von Protégé liegt in seiner Modularität und Erweiterbarkeit. Durch zahlreiche Plug-ins und Schnittstellen kann es flexibel an unterschiedliche Anforderungen angepasst werden und ermöglicht so beispielsweise auch die Nutzung von SHACL und SPARQL innerhalb der Protégé-Umgebung [30, 31]. Protégé unterstützt zudem semantische Modellierungen durch logische Inferenzen und Reasoning-Mechanismen, wodurch sich neue Beziehungen zwischen Ontologieklassen ableiten lassen. Dies ist besonders relevant für die Klassifikation chemischer Reaktionen anhand funktioneller Gruppen, die in dieser Arbeit untersucht werden [30, 31].

3.2.2 Python Erweiterungen

Python ist eine weit verbreitete, dynamisch typisierte und interpretierte Programmiersprache, die sich durch ihre einfache Syntax und hohe Lesbarkeit auszeichnet. Ihre Anwendung findet sich in zahlreichen wissenschaftlichen und industriellen Kontexten, insbesondere in den Bereichen Datenanalyse, maschinelles Lernen, Automatisierung und semantische Webtechnologien [32]. Sie unterstützt verschiedene Programmierparadigmen, darunter objektorientierte, prozedurale und funktionale Programmierung, und verfügt über eine umfangreiche Standardbibliothek, die zahlreiche Module für unterschiedliche Anwendungsbereiche bereitstellt [32–34]. Darunter zählen auch die im Rahmen dieser Arbeit eingesetzten Erweiterungen Rational Discovery Kit (RDKit), Owlready2 und pySHACL, welche die effektive Verarbeitung chemischer und ontologischer Daten ermöglichen [35–37].

owlready2

Owlready2 ist eine leistungsfähige Python-Bibliothek zur Manipulation und Verarbeitung von OWL 2.0-Ontologien, welche eine nahtlose Integration von Ontologien in Python ermöglicht. Mithilfe dieser Bibliothek können Ontologien geladen, bearbeitet und gespeichert sowie Klassen, Individuen und Relationen innerhalb einer Python-Umgebung erstellt und modifiziert werden. Darüber hinaus bietet Owlready2 Funktionen für logisches Schließen (Reasoning) mit dem HermiT-Reasoner sowie die Möglichkeit, SPARQL-Abfragen auf Ontologien auszuführen. Obwohl Owlready2 theoretisch auch SHACL-Validierungen unterstützen könnte, wurde diese Funktionalität im Rahmen dieser Arbeit durch die externe Bibliothek pySHACL realisiert [35].

pySHACL

pySHACL ist eine Python-Bibliothek, die zur Validierung von RDF-Daten unter Verwendung des SHACL-Standards (Shapes Constraint Language) dient. Das Ziel dieser Bibliothek ist es, semantische Daten auf Konsistenz und Struktur zu prüfen, indem definierte Einschränkungen auf RDF-Graphen angewendet werden. pySHACL erweist sich hierbei insbesondere für Ontologie-Workflows als vorteilhaft, da es dazu beiträgt, die Datenintegrität zu gewährleisten. Darüber hinaus unterstützt pySHACL SHACL-SPARQL-Abfragen, wodurch benutzerdefinierte Validierungsregeln direkt über SPARQL-Abfragen formuliert und ausgeführt werden können. Diese Funktion erweitert die Möglichkeiten der Datenvalidierung erheblich und erlaubt komplexere semantische Analysen. Durch die Unterstützung von OWL-RL und RDFS Reasoning kann pySHACL auch logische Schlussfolgerungen ziehen und semantische Beziehungen validieren. Infolgedessen kommt pySHACL eine Schlüsselrolle bei der Qualitätssicherung von Wissensgraphen und Ontologien zu [37].

Regular Expressions

Reguläre Ausdrücke (Regular Expressions (Regex)) definieren eine kompakte, formale Sprache zur Beschreibung von Mustern in Zeichenketten, die über eine einfache Textsuche hinausgeht. Mittels spezieller Metazeichen (z. B. „.“ für Ziffern, „*“ für Wiederholungen, „[]“ für Zeichenklassen) ermöglichen sie die flexible Suche, Extraktion und Validierung von Textdaten [38]. Für die Implementierung von Regex in Python steht das standardmäßig eingebundene re-Modul zur Verfügung [39]. Eine Anwendung von Regex zur Extraktion relevanter Textsequenzen innerhalb von Datenbankeneinträgen wird im weiteren Verlauf in Abschnitt 4.2.1 beispielhaft präsentiert.

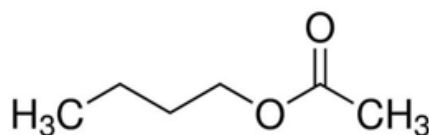
RDKit

RDKit ist ein Open-Source-Cheminformatik-Toolkit, das umfassende Werkzeuge zur Darstellung, Analyse und Verarbeitung chemischer Moleküle bereitstellt [40]. Es ermöglicht die Repräsenta-

tion chemischer Strukturen in Formaten wie SMILES und SMILES Arbitrary Target Specification (SMARTS) und unterstützt die Konvertierung zwischen diesen Formaten. Darüber hinaus beinhaltet RDKit Algorithmen zur Bestimmung der Aromatizität, die sich an chemischen Regeln wie der Hückel-Regel orientieren. Eine weitere Funktion ist die grafische Visualisierung chemischer Strukturen, die eine intuitive Interpretation ermöglicht. Ein zentrales Feature von RDKit ist die Substruktursuche, die es ermöglicht, spezifische funktionelle Gruppen oder strukturelle Muster innerhalb komplexer Moleküle zu identifizieren. Mithilfe von Graphen-Matching-Algorithmen können durch Nutzung der RD-Kit Funktion `GetSubstructMatches(substructure)` Teilstrukturen innerhalb eines Moleküls effizient gefunden werden, indem nach bestimmten Atom- und Bindungskonfigurationen gesucht wird. Diese Methode wird beispielsweise zur Erkennung funktioneller Gruppen, zur Klassifikation von Molekülen oder zur Filterung großer chemischer Datenbanken eingesetzt und ist daher auch im Kontext dieser Arbeit von entscheidender Relevanz [40, 41].

3.2.3 InChI

Der International Chemical Identifier (InChI) ist ein standardisiertes, maschinenlesbares Format, das von der IUPAC entwickelt wurde, um chemische Strukturen eindeutig zu beschreiben. Es ermöglicht eine konsistente Darstellung chemischer Informationen, was den Austausch und die Suche nach chemischen Verbindungen in Datenbanken erleichtert. Die Entwicklung begann im Jahr 2000 in Zusammenarbeit mit dem National Institute of Standards and Technology (NIST) und wurde 2005 abgeschlossen. Seitdem wird der Standard vom InChI-Trust gepflegt und weiterentwickelt, wobei die IUPAC eine führende Rolle spielt [42, 43]. Eine Besonderheit ist die Eindeutigkeit des InChI-Codes. Durch den Kanonisierungsprozess ist der InChI-Code für jede Struktur vollständig individuell. Die Abbildung 3.1 zeigt exemplarisch die Strukturformel sowie den sich daraus ergebenden InChI-Code für das Essigsäure-Molekül:



InChI = 1S/C2H4O2/c1-2(3)4/h1H3,(H,3,4)

Abbildung 3.1: InChI-Code und Strukturformel von Essigsäure [44]

Ein InChI-Code besteht aus mehreren hierarchisch aufgebauten Schichten (Layers), die verschiedene molekulare Eigenschaften erfassen. Abbildung 3.1 zeigt das Hauptlayer, welches in jedem

InChI-Code vorhanden ist. Dieses enthält die Summenformel sowie Informationen über die atomare Bindung und die Position der Wasserstoffatome. Weitere Schichten kodieren zusätzliche chemische Details wie elektrische Ladungen, Isotopenverhältnisse und Stereochemie, die im Rahmen dieser Arbeit allerdings nicht weiter betrachtet wurden [42, 43].

3.2.4 SMILES

Eine weitere vielfach angewandte maschinenlesbare Notation chemischer Strukturen ist das SMILES. SMILES wurde in den achtziger Jahren von Arthur und David Weininger entwickelt und erlaubt die Speicherung und den Austausch chemischer Informationen in Form von Molekülketten [45]. Die Notation basiert auf einer graphenbasierten Darstellung von Molekülen, bei der Atome durch ihre Elementsymbole und Bindungen durch einfache Zeichen repräsentiert werden [45].

Die Grundregeln der SMILES-Syntax sind in der Tabelle 3.1 zusammengefasst. Einzelne Atome werden durch ihr Elementsymbol angegeben, wobei bestimmte Elemente in eckige Klammern [] gesetzt werden müssen. Chemische Bindungen werden durch -, = und # für Einfach-, Doppel- und Dreifachbindungen dargestellt. Ringstrukturen werden durch Zahlen und Verzweigungen durch runde Klammern () gekennzeichnet [45].

Tabelle 3.1: SMILES-Spezifikationen [45]

Regel	Syntax	Beispiel
Atome	Elementsymbol [] für nicht-organische	C, [Fe]
Einfachbindung	- oder weggelassen	CC, C-C
Doppelbindung	=	C=C
Dreifachbindung	#	C#C
Verzweigungen	()	CC(C)C
aromatische Ringe	durch kleingestellte "c" dargestellt	c1ccccc1
Getrennte Moleküle	.	[Na+].[Cl-]

Eine weitere Spezifikation der SMILES ist die implizite Darstellung der Wasserstoffatome. Für Atome der organischen Teilmenge (B, C, O, P, S, F, Cl, Br und I) werden die Wasserstoffatome abhängig vom jeweiligen Valenzzustand innerhalb der Notation zugewiesen und erkannt, sodass keine explizite Angabe der Wasserstoffatome erforderlich ist. Im Gegensatz zu InChI-Codes sind SMILES im Allgemeinen degeneriert, d. h. ein Molekül kann durch mehrere unterschiedliche, aber äquivalente Zeichenketten dargestellt werden. Um eine eindeutige Notation zu gewährleisten, wurden kanonische SMILES entwickelt, bei denen Algorithmen eine konsistente Reihenfolge zur Generierung eines eindeutigen Codes verwenden. Hierbei ist allerdings zu beachten, dass verschiedene Algorithmen zur Identifikation von kanonischen SMILES existieren und Konsistenz und Vergleichbarkeit nur bei Verwendung des selben Algorithmus sichergestellt werden kann [43, 45].

3.2.5 SMARTS

Das SMARTS ist eine Erweiterung der SMILES-Notation, die speziell zur Substruktursuche in chemischen Datenbanken dient. Im Gegensatz zu SMILES, das komplette Molekülstrukturen beschreibt, erlaubt SMARTS die Definition von chemischen Mustern durch flexible Spezifikationen von Atomen und Bindungen. Eine erweiterte Syntax erlaubt das Suchmuster zur Substruktursuche durch Definition von Freiheitsgraden und Restriktionen gezielter anzupassen. Darunter zählen topologische Eigenschaften wie die Anzahl der Bindungen ($D<n>$), aber auch logische Operatoren wie $!$, was eine Negation darstellt, oder $;$, das ein „und“ darstellt. Einige dieser erweiterten SMARTS-Spezifikation und logischen Operatoren, die im Rahmen dieser Arbeit von Relevanz sind, sind in Tabelle 3.2 dargestellt [46].

Die Funktionalität zur gezielten Anpassung der Substruktursuche durch diese erweiterte Syntax bietet eine solide Grundlage, um die automatische Klassifikation von Substanzen und Reaktionen im Rahmen dieser Arbeit zu ermöglichen und somit eine strukturierte und effiziente Analyse chemischer Systeme zu gewährleisten.

Tabelle 3.2: SMARTS-Spezifikationen [46]

Symbol / Syntax	Bedeutung
$*$	Beliebiges Atom (Wildcard)
$D<n>$	Atom mit genau $<n>$ expliziten Bindungen
$H<n>$	Atom mit genau $<n>$ gebundenen Wasserstoffatomen
$h<n>$	Atom mit genau $<n>$ impliziten Wasserstoffatomen
$v<n>$	Atom mit einer Gesamtbindungsordnung von $<n>$
$X<n>$	Atom mit insgesamt $<n>$ Verbindungen
$-<n>$	Atom mit einer negativen Ladung von $<n>$
$+<n>$	Atom mit einer positiven Ladung von $<n>$
$;$	Logisches UND zwischen zwei Bedingungen
$,$	Logisches ODER zwischen zwei Bedingungen
$\$(...)$	Eingebettete SMART-Muster, die vorrangig ausgewertet werden
$!$	Logische Negation einer Bedingung

3.2.6 pyEnzyme

Das Package pyEnzyme stellt eine Python-Bibliothek zur Erstellung, Bearbeitung und Analyse von EnzymeML-Dokumenten dar und ist folglich ein wichtiges Werkzeug zur Arbeit mit EnzymeML. Durch die Bereitstellung einer Programmierschnittstelle für den Import, die Validierung und den Export von EnzymeML-Dateien erleichtert pyEnzyme die automatisierte Verarbeitung enzymkinetischer Daten. pyEnzyme erweist sich insbesondere bei der Extraktion kinetischer Parameter, der Modellierung von Reaktionsmechanismen und der nahtlosen Integration von EnzymeML-Daten in bestehende Analysepipelines als außerordentlich hilfreich. In der Folge leistet pyEnzyme einen we-

sentlichen Beitrag zur Nutzung von EnzymeML in computationalen Biochemie-Anwendungen [47].

4 Methodischer Ansatz

Die methodische Vorgehensweise dieser Arbeit umfasst mehrere Schritte zur Erweiterung einer Ontologie sowie der Entwicklung eines Workflows, der den Import, die Klassifizierung sowie auch weitergehende Analysemöglichkeiten von chemischen Reaktionsdaten aus unterschiedlichen Quellen ermöglicht. Der während dieser Arbeit entstandene Python-Code wird im Verlaufe dieses Kapitels an relevanten Stellen ausschnittsweise entweder unmittelbar oder unter entsprechendem Verweis im Anhang dargestellt. Der vollständige Code ist unter <https://hdl.handle.net/21.11165/4cat/2d9b-emem> zu finden [48]. Das Prozedere ist in Abbildung 4.1 visualisiert dargestellt.

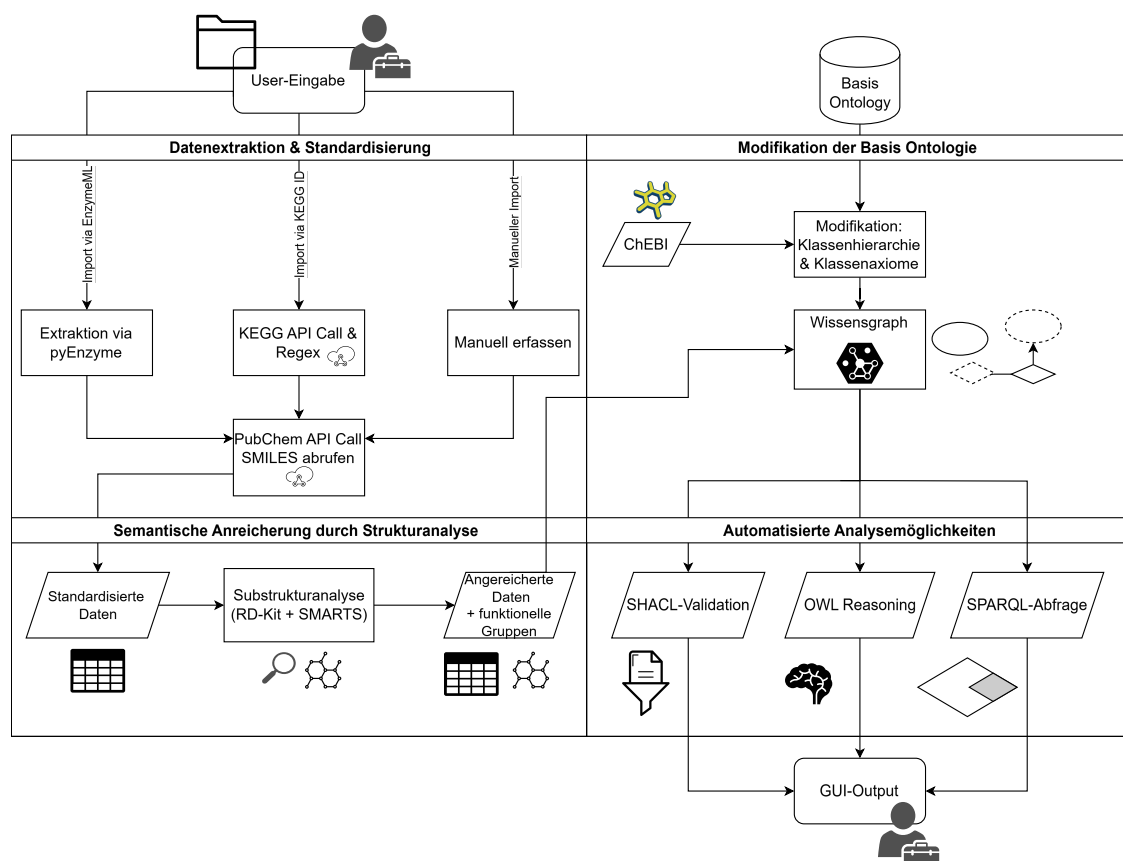


Abbildung 4.1: Workflow der Datenintegration und -analyse

Der Prozess beginnt (siehe Abbildung 4.1, oben links/Mitte) mit der Bereitstellung von Reaktions-

daten durch den Nutzer. Diese können auf verschiedenen Wegen in die Pipeline eingespeist werden: Entweder liegen die experimentellen Daten bereits strukturiert in Form von EnzymeML-Dateien vor, oder es handelt sich um Referenzdaten (z. B. aus der Literatur oder Datenbanken wie KEGG), die der Nutzer entweder manuell über die entwickelte grafische Benutzeroberfläche (GUI) eingibt oder durch Angabe einer KEGG-Reaktions-ID für den automatisierten Abruf über die KEGG API spezifiziert.

Die entwickelte Pipeline verarbeitet diese unterschiedlichen Eingaben dann weitgehend automatisiert, um sie in einen einheitlichen, semantisch angereicherten Wissensgraphen zu überführen. Im ersten Schritt erfolgt die Datenerfassung und Standardisierung: EnzymeML-Dateien werden mit pyEnzyme geparst; KEGG-Daten werden über die KEGG REST API abgerufen und mit regulären Ausdrücken analysiert; bei manuellen Eingaben oder fehlenden Strukturinformationen werden Substanznamen über die PubChem API abgefragt, um kanonische SMILES-Codes und IRIs zu erhalten. Anschließend erfolgt die semantische Anreicherung, bei der die gültigen SMILES-Codes mittels RDKit auf relevante funktionelle Gruppen analysiert werden.

Im zentralen Schritt der Pipeline werden diese aufbereiteten und angereicherten Informationen verwendet, um den Wissensgraphen zu füllen. Dies geschieht mit Hilfe von Owlready2, wobei Instanzen für Reaktionen, Stoffe und funktionelle Gruppen in einer zuvor erweiterten Ontologie (basierend auf RXNO) angelegt und durch semantische Relationen miteinander verknüpft werden. Ein wesentlicher Aspekt vor der Pipeline-Entwicklung ist daher die Modifikation dieser Basisontologie, insbesondere die Definition spezifischer Klassen (z. B. für funktionelle Gruppen nach Anzahl oder für bestimmte Reaktionstypen wie Fettverseifung) und die Implementierung logischer Axiome (Klassenrestriktionen), um die Grundlage für das spätere automatische Reasoning zu schaffen.

Der resultierende Wissensgraph ermöglicht schließlich die automatisierte Analyse, die wiederum vom Benutzer über die GUI gestartet wird: OWL2-Reasoning (HermiT) klassifiziert die Reaktionen, während die SHACL-Regeln zur Validierung der Datenstruktur eingesetzt werden können. Spezifische SPARQL-Abfragen, wie die entwickelte semantische Ähnlichkeitsanalyse, erlauben gezielte Untersuchungen und Vergleiche von Reaktionen. Die GUI dient als Schnittstelle, um diese Analysen durchzuführen und die Ergebnisse darzustellen, ohne dass der Benutzer direkt mit Ontologiewerkzeugen wie Protégé interagieren muss.

Im Folgenden werden die einzelnen methodischen Schritte - beginnend mit der Ontologiemodifikation, gefolgt von der detaillierten Implementierung der Import- und Analysepipeline sowie der Formulierung der SHACL/SPARQL-Logik - näher erläutert und die dabei getroffenen Entscheidungen begründet.

4.1 Auswahl und Modifikation der Basis Ontologie

4.1.1 Wahl der Basis-Ontologie

Für die Modellierung chemischer Reaktionen in dieser Arbeit wurde die RXNO als Basisontologie gewählt. RXNO bietet eine strukturierte Taxonomie für chemische Reaktionen, die besonders für die systematische Klassifizierung und das Reasoning über Reaktionen geeignet ist. Da diese Arbeit eine Ontologie für chemische Reaktionen erweitert, insbesondere im Hinblick auf funktionelle Gruppen und semantische Ähnlichkeitsanalysen, bietet RXNO eine besonders geeignete Basis.

Ein zentraler Aspekt ist die bestehende Hierarchie für Reaktionsklassen und funktionelle Gruppen, die eine präzise Modellierung chemischer Umsetzungen ermöglicht. RXNO erlaubt es, bestehende Reaktionskategorien zu verfeinern und durch restriktive Klassenregeln gezielt einzuschränken, um logische Rückschlüsse auf chemische Reaktionsmechanismen zu ermöglichen. Dies erleichtert die Identifikation ähnlicher Reaktionen sowie deren semantische Anreicherung. Ein besonders wichtiger Aspekt ist die Implementierung in OWL2, die eine flexible Erweiterung der Ontologie durch eigene Klassen, Relationen und semantische Regeln erlaubt [26]. Diese Struktur ermöglicht es, SPARQL-Anfragen zur Ähnlichkeitsanalyse chemischer Reaktionen zu formulieren und Beziehungen zwischen importierten und existierenden Reaktionen systematisch zu analysieren. Dies stellt einen entscheidenden Vorteil gegenüber relationalen Datenbanken wie KEGG dar, die primär tabellenbasierte Strukturen verwenden und keine semantische Modellierung chemischer Beziehungen erlauben [49]. Ein weiterer Vorteil ist die enge Verknüpfung mit ChEBI, wodurch Substanzen und funktionelle Gruppen präzise erfasst werden können. Die Oberklasse der Substanzen „molecular entity“ innerhalb von RXNO entstammt vollständig der ChEBI-Ontologie [50]. Durch diese Integration kann die etablierte ChEBI-Taxonomie genutzt werden, um chemische Substanzen einschließlich funktioneller Gruppen und generischer Klassen systematisch zu klassifizieren und für die Reaktionsmodellierung zu verwenden [17, 51].

4.1.2 Modifikation von RXNO

Die Erweiterung und Anpassung von RXNO wurde mit dem Ziel durchgeführt, eine genauere semantische Modellierung chemischer Reaktionen zu ermöglichen. Zu diesem Zweck wurden gezielt neue Klassen erstellt, um spezifische Reaktionsmechanismen, funktionelle Gruppen und strukturelle Eigenschaften abzubilden. Dieser Abschnitt erläutert, warum bestimmte Klassen erstellt wurden, wo sie in die bestehende Hierarchie integriert wurden und welche Vorteile diese Anpassungen bieten.

Abbildung 4.2 stellt einige der in RXNO vorkommenden Oberklassen dar. So enthält die RXNO beispielsweise Superklassen aus der Information Artifact Ontology (IAO). Die IAO ist Teil der OBO-Foundry und mit der BFO abgestimmt und liefert ein standardisiertes Vokabular zur Beschreibung von Meta-Daten, unabhängig vom jeweiligen Fachbereich der Ontologie. Von besonderer Relevanz

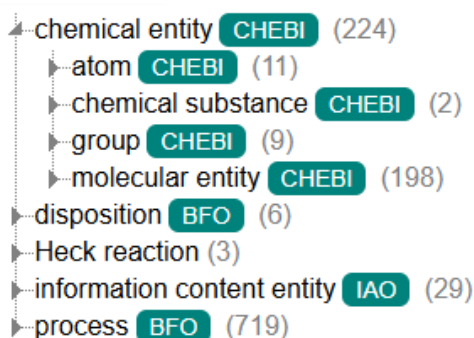


Abbildung 4.2: RXNO Superklassen [50]

sind im Rahmen dieser Arbeit die Superklassen `molecular entity`, `group` sowie `molecular process`. Diese werden genutzt, um die zur Klassifikation nötigen Klassenrestriktionen zu definieren.

4.1.3 Einführung der Beispielreaktion: Verseifung von Triacetin

Um die semantische Modellierung chemischer Reaktionen in RXNO anhand eines konkreten Beispiels zu veranschaulichen, wird im Folgenden die basisch katalysierte Verseifung von Triacetin (Glycerintriacetat) betrachtet. Diese Reaktion stellt eine typische Hydrolysereaktion von Estern dar, bei der unter Einwirkung einer starken Base die Esterbindungen gespalten und in die entsprechenden Alkohol- und Salzprodukte überführt werden. Die zugehörige Reaktionsgleichung ist in Abbildung 4.3 dargestellt [52, 53].

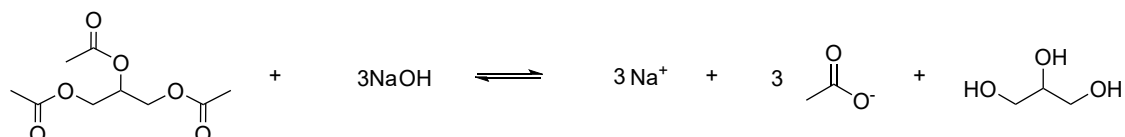


Abbildung 4.3: Beispielreaktion „Basische Hydrolyse von Triacetin“ für die repräsentative Modellierung von Reaktionen in Ontologien

Triacetin, ein Dreifachester des Glycerins mit Essigsäure, reagiert in Anwesenheit einer wässrigen Natriumhydroxid-Lösung (NaOH) unter Hydrolyse der Acetat-Gruppen. Dabei entstehen Glycerin und Natriumacetat als Reaktionsprodukte. Die Reaktion folgt einem typischen nukleophilen Angriffsmechanismus, bei dem die Hydroxid-Ionen die Carbonylgruppen der Ester angreifen, wodurch nach einer Umlagerung die Carboxylat-Ionen der Essigsäure sowie Glycerin freigesetzt werden [54]. Die Verseifung verläuft dabei über zwei Zwischenstufen, Diacetin (Diacetylglycerol) und Monoacetin (Monoacetylglycerol), bevor schließlich Glycerol als Endprodukt entsteht. Diese Mehrstufigkeit der Reaktion wird zur vereinfachten semantischen Modellierung außer Acht gelassen, indem die Umwandlung von Triacetin zu Glycerin vereinfacht als ein einzelner Reaktionsschritt betrachtet

wird.

Eine weitere Modellannahme besteht darin, dass das auf der Produktseite entstehende Natriumacetat nicht als einzelnes Salz betrachtet wird, sondern die Acetat-Anionen und die Metall-Kationen jeweils als einzelne Reaktionsprodukte betrachtet werden. Dies erlaubt eine höhere Flexibilität während der Modellierung und verringert die absolut anfallende Anzahl an zu definierenden Klassen, da nicht für jedes Salz eine weitere Substanzklasse definiert werden muss.

Parallel dazu wird die Verseifung von Butylacetat (Essigsäurebutylester) modelliert, um aufzuzeigen, dass eine differenzierte Klassifizierung von Esterspaltungen möglich ist (s. Reaktionsgleichung im Anhang Abbildung A.2). Dass eine semantische Trennschärfe zwischen verschiedenen Esterhydrolysen erreicht werden kann, kann durch den Vergleich dieser beiden Reaktionen gezeigt werden. Während die Verseifung von Triacetin als Beispiel für die Hydrolyse eines dreiwertigen Alkohols mit drei Estergruppen dient, auch als Fettverseifung bekannt, stellt die Hydrolyse von Butylacetat eine typische Spaltung eines einfachen Carbonsäureesters dar. Die Modellierung beider Reaktionen unterstreicht somit die Möglichkeit, durch gezielte Anpassungen der Ontologie und spezifische Klasseinschränkungen eine präzisere Unterscheidung zwischen verschiedenen Reaktionsmechanismen zu ermöglichen.

Alkohole

Eine der wesentlichsten Produktklassen im Kontext dieser Reaktion sind Alkohole, die durch die Hydrolyse der Esterbindungen entstehen. Bei Glycerin handelt es sich dabei insbesondere um einen mehrwertigen Alkohol. Die RXNO enthält bereits eine auf ChEBI basierende Oberklasse für Alkohole. Abbildung 4.4 zeigt die standardmäßige Hierarchie der Alkoholklassen innerhalb von RXNO.

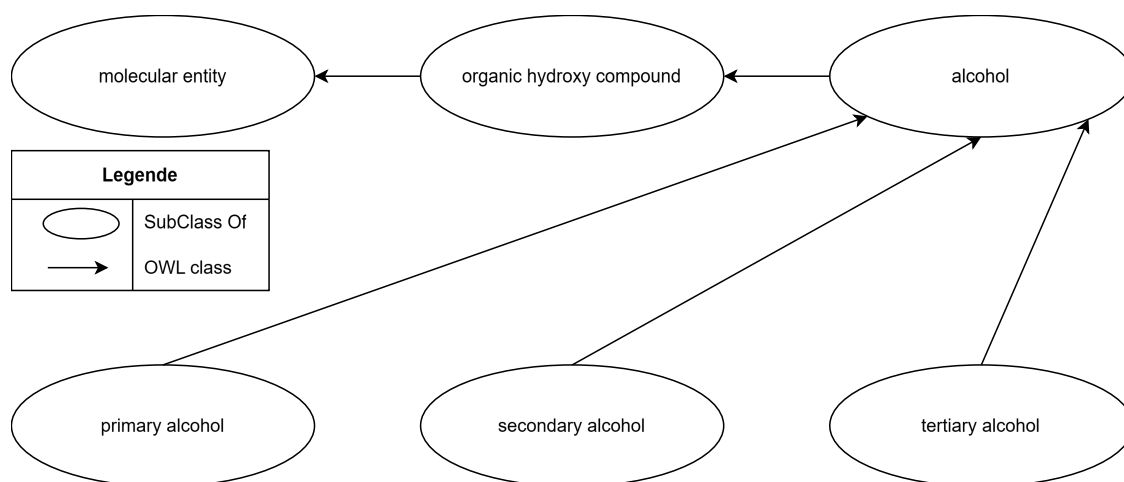


Abbildung 4.4: Klassenhierarchie der Klasse der Alkohole in RXNO

Wie anhand von Abbildung 4.4 zu erkennen ist, hat die Klasse `alcohol` bereits drei Subklassen für primäre, sekundäre als auch tertiäre Alkohole. Im vorliegenden Fall ist allerdings die Wertigkeit des Alkohols von Relevanz, um auch zwischen Triglyceridspaltungen und Esterspaltungen unterscheiden zu können. Daher werden an dieser Stelle drei neue Subklassen definiert, jeweils für ein-, zwei-, und dreiwertige Alkohole. Da in der ChEBI keine generischen Einträge zu speziell mehrwertigen Alkoholen existieren, werden die Klassen für die einwertigen Alkohole mit keiner IRI hinterlegt, und die Klassen zu zwei- bzw. dreiwertigen Alkoholen mit den ChEBI-Einträgen zu Diolen bzw. Triolen als Datenpunkte hinterlegt. Zwar können unter den Begriff „Diol“ bzw. „Triol“ auch weitere Moleküle fallen, die zusätzlich zu den Hydroxygruppen noch weitere funktionelle Gruppen aufweisen, doch das ist im Rahmen der gewählten Eingangsbeispiele als unproblematisch zu bewerten [55, 56]. Abbildung 4.5 zeigt die modifizierte Klassenhierarchie, wobei Klassen, welche nicht ursprünglich in der RXNO vorhanden waren, mit einer gestrichelten Umrandung markiert sind.

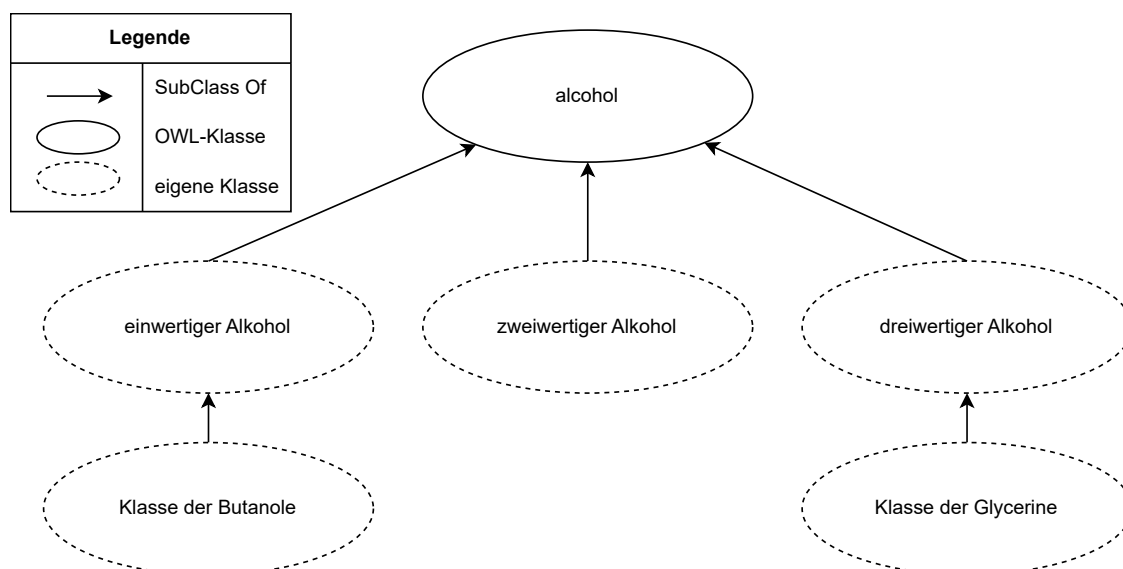


Abbildung 4.5: Modifizierte Klassenhierarchie der Klasse der Alkohole in RXNO

Neben den Klassen für ein- bis mehrwertige Alkohole, kommen nun noch die Oberklassen für explizite Substanzen, wie `Klasse der Butanole` und `Klasse der Glycerine` hinzu. Auch diese Klassen werden entsprechend mit dem zugehörigen ChEBI-Eintrag als IRI hinterlegt [57, 58]. Die zum Reasoning erforderlichen Klassenrestriktionen bauen grundlegend auf der automatischen Erkennung und Zuweisung von funktionellen Gruppen während des Importprozesses auf und werden daher erst im Anschluss daran erläutert.

Ester

Die RXNO enthält auch die Oberklasse der **carbonyl compounds** und die Subklasse der **carboxylic ester**. Die Klassen können in diesem Anwendungsfall so genutzt werden und müssen lediglich durch weitere Subklassen ergänzt werden. Die verwendeten Klassen sowie die selbst erstellten „custom classes“ für die Stoffgruppe der Ester sind in nachfolgend in Abbildung 4.6 dargestellt:

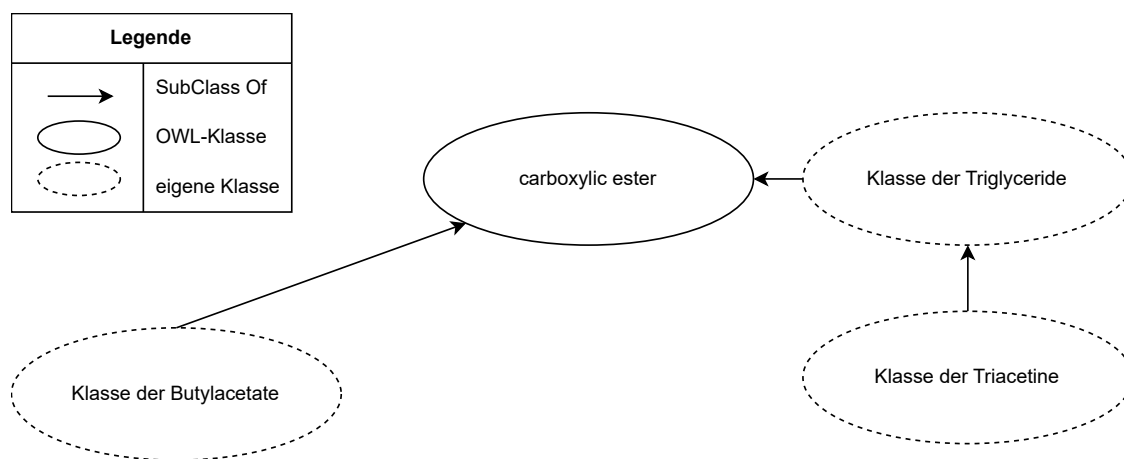


Abbildung 4.6: Modifizierte Klassenhierarchie der Klasse der Ester

Basen & Kationen

Die RXNO enthält zwar bereits eine Superklasse für Basen, diese erfolgt jedoch ausschließlich über die Kategorie der Disposition, wodurch Basen nur als abstrakte chemische Eigenschaft und nicht als konkrete molekulare Entitäten erfasst werden. Dies führt im weiteren Verlauf zu mehreren Einschränkungen im Rahmen der Festlegung der Klassenrestriktionen. So können unter anderem die Basen nicht explizit als Reaktionspartner definiert werden, da die konkrete Präsentation von Basen als Substanz fehlt. Der Reaktionsmechanismus kann dadurch nur eingeschränkt dargestellt werden, da die Hydrolysereaktion nicht mit einer spezifischen Base verknüpft wird, sondern nur mit der generellen Fähigkeit zur Protonenaufnahme. Aus diesem Grund wurde die neue Klasse **inorganic compounds** als Subklasse von **molecular entity** definiert. Darunter wurden die spezifischeren Subklassen für **Metall-Kationen**, **Wasser**, **Basen** sowie **Brønsted Basen** festgelegt, wie in Abbildung 4.7 dargestellt.

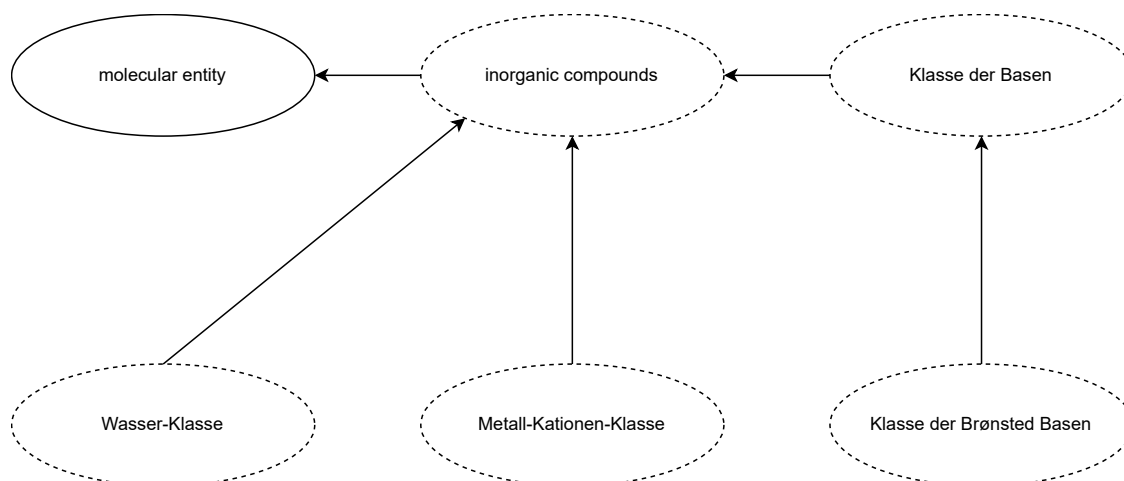


Abbildung 4.7: Modifizierte Klassenhierarchie für anorganische Komponenten

Funktionelle Gruppen & Acetat-Ionen

Die Klassifikation der funktionellen Gruppen ist für das Reasoning der Stoffe und damit der Reaktionen von entscheidender Bedeutung. Auch hier kann auf die bestehende Klassenhierarchie der RXNO zurückgegriffen werden.

Die RXNO enthält die Klasse `groups`, die bereits einige für diese Arbeit relevante Unterklassen enthält. Dazu zählen beispielsweise die Klassen `alkyl group` sowie `hydroxy group`, die für die semantische Modellierung von Edukten und Produkten in verschiedenen Reaktionsmechanismen essentiell sind. Allerdings fehlen auch noch einige Subklassen. So fehlt beispielsweise eine Klasse für die funktionelle Gruppe der Ester und eine Klasse für das Carboxylate-Anion. Des Weiteren ist zur Differenzierung weiterhin notwendig weitere Subklassen für Alkylreste verschiedener Gestalt und Kettenlänge zu definieren. Die folgende Abbildung 4.8 zeigt die modifizierte Klassenhierarchie der Superklasse `groups`:



Abbildung 4.8: Modifizierte Klassenhierarchie für funktionelle Gruppen

Wie der Abbildung 4.8 zu entnehmen ist, wurden weitere Unterklassen für unterschiedliche Häufigkeiten des Vorkommens der jeweiligen funktionellen Gruppe definiert. Diese Differenzierung ist notwendig, um z. B. die Klasseneinschränkungen für Alkohole so trennscharf formulieren zu können, dass verschiedenwertige Alkohole voneinander unterschieden werden können, oder auch für die semantische Trennung zwischen Triglyceriden und einfachen Carbonsäureestern. Ein Verzicht auf diese die Häufigkeit beschreibenden Subklassen wäre unter Nutzung erweiterter Kardinalitätsrestriktionen, wie sie im Abschnitt 2.2.2 beschrieben worden sind, in OWL grundsätzlich möglich, wurde im Rahmen dieser Arbeit allerdings nicht umgesetzt. Gründe für die Wahl dieser indirekten Modellierung werden im weiteren Verlauf erläutert.

Definition der Reaktionsklassen

Für die semantische Modellierung von Esterhydrolysen in RXNO wurden im Rahmen dieser Arbeit insgesamt drei spezifische Reaktionsklassen definiert: die Fettverseifung (Saponifikation), die basische Hydrolyse eines einfachen Carbonsäureesters sowie die neutrale Esterhydrolyse, wobei im Folgenden exemplarisch die beiden basischen Esterspaltungen erläutert werden. Alle hier besprochenen Reaktionsklassen werden als Subklassen der bereits bestehenden Klasse `hydrolysis` eingeordnet, da es sich um Hydrolysereaktionen handelt. Die Hierarchie sowie die Definition der Reaktionsklassen ist in Abbildung 4.9 dargestellt:

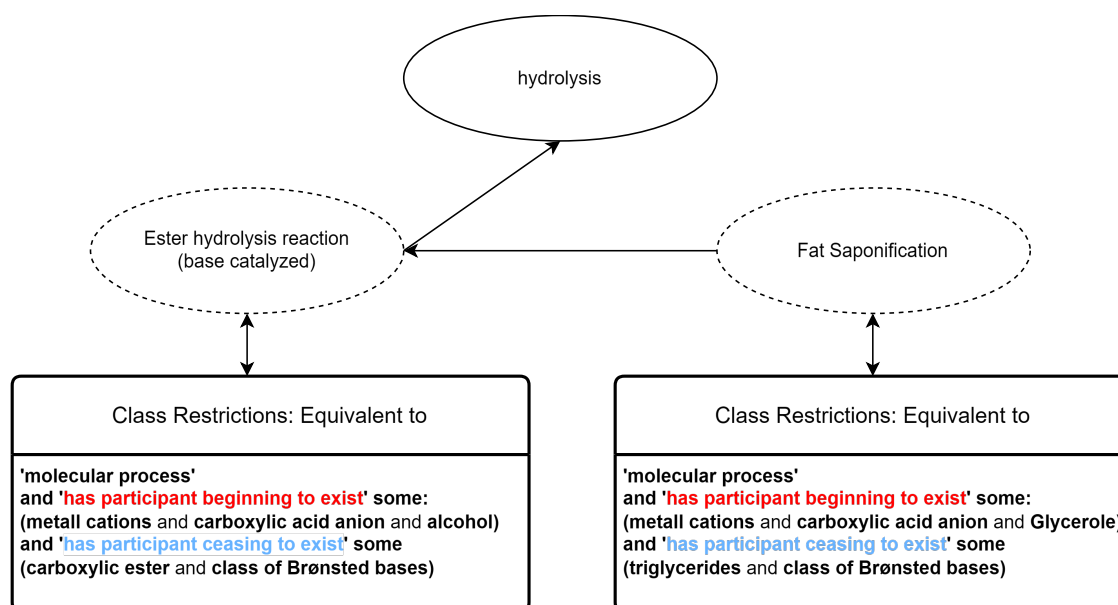


Abbildung 4.9: Klassenhierarchie und - restriktionen der Reaktionsklassen

Um die Stoffumwandlung innerhalb dieser Reaktionsklassen präzise abzubilden, wurden die bereits in RXNO vorhandenen Subproperties `participant beginning to exist` und `participant ceasing to exist` verwendet. Diese ermöglichen eine eindeutige semantische Trennung zwischen den Edukten und Produkten einer Reaktion. Für die basische Hydrolyse eines einfachen Carbonsäureesters wurde definiert, dass Metallkationen, Carboxylatanionen und Alkohole als `has participant beginning to exist` zugeordnet werden, da sie im Verlauf der Reaktion entstehen. Gleichzeitig werden Carbonsäureester und Brønsted-Basen als `participant ceasing to exist` zugeordnet, da sie während der Reaktion aufgebraucht werden. Für die Fettverseifung wurde eine vergleichbare semantische Modellierung vorgenommen, wobei sich die entstehenden Produkte von denen der einfachen Hydrolyse unterscheiden. In diesem Fall wurden ebenfalls Metallkationen und Carboxylatanionen als `participant beginning to exist` definiert, wobei anstelle eines einfachen Alkohols Glycerol als Produkt auftritt. Triglyceride und Brønsted-Basen werden entsprechend als `participant ceasing to exist` erfasst. Die Nutzung der `has participant`-Properties ermöglicht nicht nur eine korrekte Beschreibung der Stoff-

umwandlung, sondern auch die Schaffung einer Grundlage für das automatische Reasoning über Reaktionsmechanismen. Des Weiteren wurden beide Reaktionsklassen als definierte Klassen modelliert. Dies impliziert, dass sie nicht allein durch ihre Position in der Hierarchie determiniert werden, sondern über notwendige und hinreichende Bedingungen charakterisiert sind. Dem Reasoner ist es somit möglich, Reaktionen automatisch zu klassifizieren, sofern alle definierten Bedingungen erfüllt sind. Eine detaillierte Erläuterung dieses Konzepts findet sich in Abschnitt 2.2.2.

4.2 Import und Erkennung funktioneller Gruppen

Die automatisierte Klassifikation chemischer Reaktionen erfordert eine strukturierte Import-Pipeline, um Reaktionsdaten aus verschiedenen Quellen in das System zu überführen. Aktuell existieren zwei primäre Importwege: der automatisierte Import aus EnzymeML sowie der manuelle Import von Referenzdaten aus der Literatur oder externen Datenbanken. Eine Visualisierung der verfügbaren Importwege ist im Folgenden in Abbildung 4.10 dargestellt.

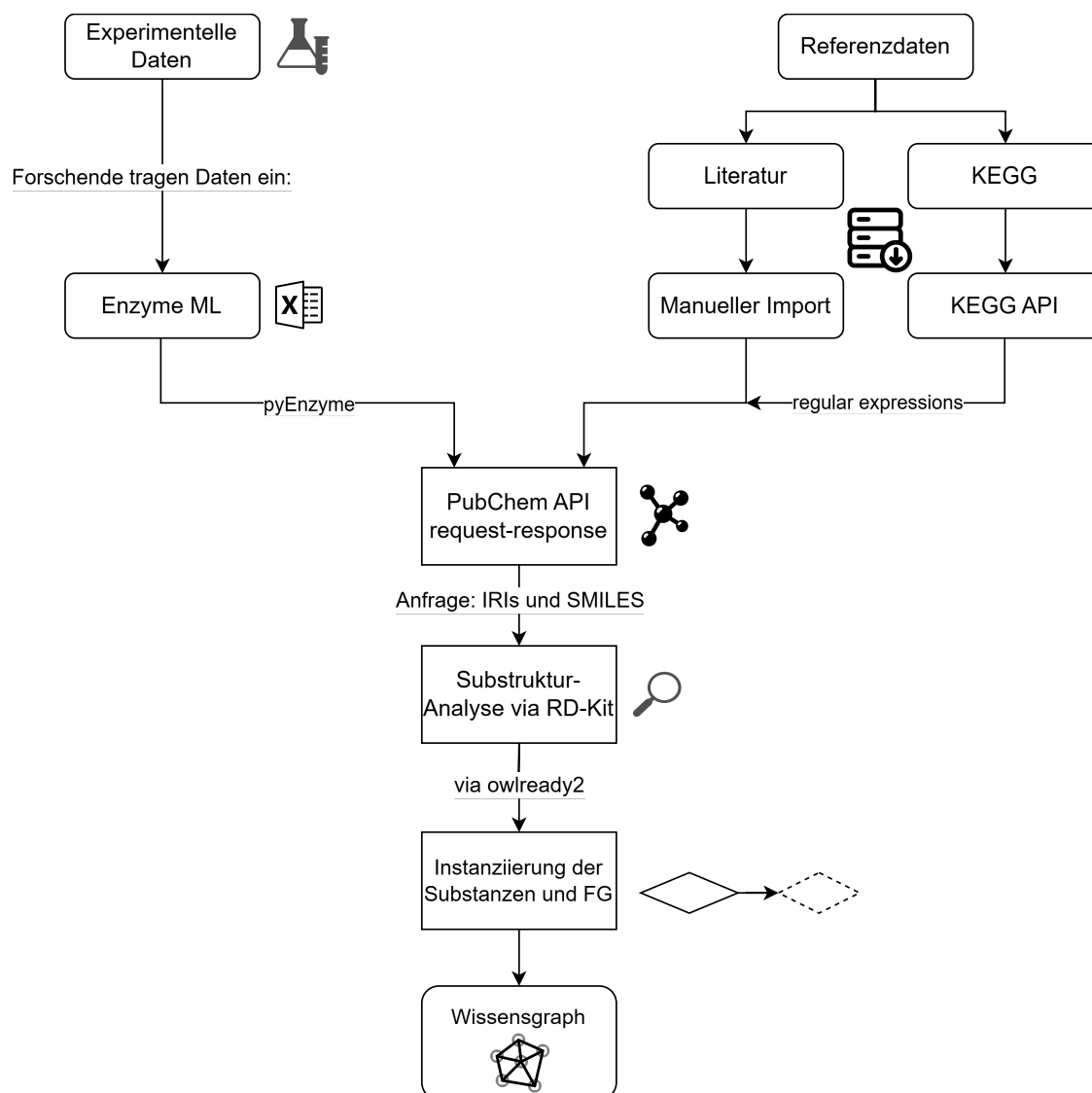


Abbildung 4.10: Import-Pipelines von exp. sowie kuratierten bzw. idealisierten Reaktionsdaten aus Literatur und Datenbanken in den Wissensgraphen

Die Substruktursuche sowie Zuweisung und Erkennung von funktionellen Gruppen verläuft unabhängig vom Importweg jeweils identisch und wird daher im Anschluss daran erläutert.

4.2.1 Import-Pipelines

Import aus EnzymeML

Forschende haben die Möglichkeit, ihre experimentellen Daten in Form von EnzymeML-Dateien hochzuladen. Diese Dateien beinhalten standardisierte Informationen zu enzymatischen Reaktionen, wie beispielsweise Edukte, Produkte, Katalysatoren und kinetische Parameter. Die Datenex-

traktion aus EnzymeML erfolgt dabei durch die Python-Erweiterung pyEnzyme. Der Importprozess verläuft in mehreren Schritten: Zunächst wählen die Forschenden die zu importierende Datei aus und laden diese hoch. Hierbei handelt es sich um eine .xslm-Datei, die einer Bereinigung und Validierung unterzogen wird. Im Anschluss wird das EnzymeML-Dokument mit `pe.EnzymeML-Document.fromTemplate(file_path)` geladen, um alle enthaltenen Reaktionen zu iterieren und die Reaktanden zu extrahieren. Die dafür relevanten Sheets innerhalb der EnzymeML-Datei sind im Anhang unter Abbildung A.1 dargestellt. Die Spezies-IDs der Edukte und Produkte werden aus den `reaction.educts-` und `reaction.products-` Attributen abgerufen und mit den entsprechenden Molekülnamen sowie SMILES-Codes aus dem `reactant dict` verknüpft. Zur Erkennung funktioneller Gruppen erfolgt eine automatische Validierung der SMILES-Codes mittels `Chem.MolFromSmiles(smiles)`, wodurch fehlerhafte oder ungültige Strukturen frühzeitig ausgeschlossen werden. Im Anschluss an den Extraktionsprozess werden die gewonnenen Reaktionsinformationen in einem Dictionary `reaction data` gespeichert, das alle relevanten Daten wie Edukte, Produkte und SMILES-Codes für die Klassifikation enthält. Schließlich werden die verarbeiteten Reaktionsdaten über die `execute import`-Funktion in das System überführt.

Im Falle des Imports aus dem EnzymeML Sheet dient der SMILES-Code als Input für die PubChem API. Zum Abruf der PubChem-Einträge und der dazugehörigen IRIs wird mithilfe der `requests`-Erweiterung in Python folgende Funktion in Codeabschnitt 4.1 definiert:

```
1 def get_pubchem_iri(smiles):
2
3     base_url = "https://pubchem.ncbi.nlm.nih.gov/rest/pug"
4
5     #PubChem benötigt URL-encodierte SMILES
6     encoded_smiles = requests.utils.quote(str(smiles), safe='')
7
8     # Abruf der CID (Compound ID) aus PubChem
9     cid_url = f"{base_url}/compound/smiles/{encoded_smiles}/cids/JSON"
10    cid_response = requests.get(cid_url)
11    print(cid_response)
12
13    #Fehlerhandling, falls die Anfrage fehlschlägt...
14
15
```

Codeabschnitt 4.1: PubChem API request

Da die SMILES-Darstellungen als Input dienen, müssen diese vorerst in eine URL-encodierte Schreibweise überführt werden, was durch die `requests` Erweiterung ermöglicht wird [59]. Daraufhin wird diese encodierte SMILES Darstellung zu einer CID-URL zusammengesetzt, welche den PubChem-Eintrag der entsprechenden Substanz darstellt. Auf Grundlage dieser PubChem-Einträge werden nun über `owlready2` die Instanzen erstellt und mit dem entsprechenden PubChem

Eintrag als IRI im Wissensgraphen hinterlegt. Der entsprechende Codeabschnitt ist nachfolgend in Codeabschnitt 4.2 dargestellt:

```
1 for name, smiles in zip(products, reine_produkt_smiles): # Richtige Zuordnung: Name
   zuerst
2 pubchem_iri = get_pubchem_iri(smiles)
3 existing_instance = onto.search_one(iri=pubchem_iri) if pubchem_iri else None
4
5 if existing_instance:
6     substance_instance = existing_instance
7     print(f" Bestehende Instanz gefunden: {name} (IRI: {pubchem_iri})")
8 else:
9     safe_name = name.replace(" ", "_") # Name ohne Leerzeichen
10    substance_instance = molecular_entity(safe_name)
11    substance_instance.iri = pubchem_iri if pubchem_iri else f"
    Unknown_IRI_for_{safe_name}"
12    print(f" Neue Instanz erstellt: {name}, IRI: {substance_instance.iri}")
13
14    substance_instance.label = name
15
16
```

Codeabschnitt 4.2: Erstellung der Instanzen via owlready2

Wie dem Codeabschnitt 4.2 zu entnehmen ist, wird für jeden Reaktanden der PubChem-Eintrag über die Funktion `get_pubchem_iri` abgerufen. Die Instanzen werden dabei von vornherein zur Klasse `molecular entity` zugeordnet, damit sie innerhalb der Ontologie auch als Substanzen auffindbar sind. Nachdem nun die Instanzen für die jeweiligen Reaktionspartner automatisiert erstellt worden sind, ist zur Klassifizierung von Reaktionen noch die Definition der Reaktionsinstanzen notwendig. Dazu wird pro Reaktion jeweils eine neue Instanz erstellt, welche dann über die oben bereits beschriebenen Eigenschaftsbeziehungen `has participant ceasing to exist` und `has participant beginning to exist` mit den Edukten bzw. Produkten in eine Relation gesetzt wird. Der folgende Codeabschnitt 4.3 zeigt die Zuweisung der Reaktanden zu den Reaktionen via owlready2, wobei die Zuweisung für die Produkte entsprechend analog verläuft.

```

1 # Instanz für die Reaktion erstellen
2 reaction_instance = molecular_process_class(f"Reaction_{reaction_id}")
3 reaction_instance.label = f"Reaktion {reaction_id}"
4 print(f"Deine Reaktion wurde erfolgreich importiert.")
5
6 # Eigenschaften für Reaktanten und Produkte suchen oder erstellen
7 has_participant_ending_to_exist = onto.search_one(iri="http://purl.obolibrary.org/obo/BFO_0000078")
8 # Reaktanten der Reaktion hinzufügen
9 for edukt in educts:
10     edukt_instance = onto.search_one(label=edukt)
11     if edukt_instance:
12         has_participant_ending_to_exist[reaction_instance].append(edukt_instance)
13         print(f"Assigned {edukt} as participant ending to exist to Reaktion {reaction_id}")
14
15

```

Codeabschnitt 4.3: Erstellung der Reaktionsinstanzen via owlready2

Auch hier werden die Instanzen a priori einer Oberklasse zugeordnet, in diesem Fall die Klasse `molecular process`, welcher alle Reaktionen angehören.

Manueller Import

Da aus der EnzymeML heraus nur experimentelle Daten von Forschenden importiert werden können, muss ein separater Importweg für Referenzreaktionen etabliert werden, welche entweder aus kuratierten Datenbanken wie der KEGG stammen oder aus anderer Fachliteratur. Im Falle von Referenzreaktionen aus der Literatur oder anderen Datenbanken kann man, anders als in der EnzymeML, nicht davon ausgehen, dass SMILES Codes in jedem Fall mit angegeben werden. Daher wird die PubChem API Anfrage für diesen Zweck abgeändert, sodass auf Basis der offiziellen Bezeichnungen der Substanzen die entsprechenden Datenbankeinträge abgerufen werden.

```

1 def get_smiles_from_pubchem(molecule_name):
2     url = f"https://pubchem.ncbi.nlm.nih.gov/rest/pug/compound/name/{molecule_name}/property/CanonicalSMILES/JSON"
3     response = requests.get(url)
4     if response.status_code == 200:
5         data = response.json()
6         smiles = data['PropertyTable']['Properties'][0]['CanonicalSMILES']
7         return smiles
8     else:
9         print(f"Error fetching SMILES for {molecule_name}: {response.status_code}")
10        return None

```

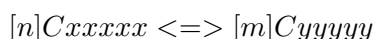
Codeabschnitt 4.4: PubChem API SMILES Request

Wie dem Codeabschnitt 4.4 zu entnehmen ist, wird mithilfe des Substanznamen als Input die PubChem URL abgerufen. Daraufhin wird ebenso die zugehörige SMILES Darstellung aus PubChem aufgerufen und in Listen gespeichert. Erwähnenswert ist an dieser Stelle weiterhin, dass explizit die zuvor bereits in Abschnitt 3.2.4 beschriebenen kanonischen SMILES abgerufen werden, um die Eindeutigkeit der Moleküldarstellungen zu gewährleisten. Die auf den Import folgende Suche nach Substrukturen mittels RDKit erfolgt unabhängig vom Importweg, da hierzu ausschließlich die SMILES Darstellungen der Reaktanden nötig sind.

Import aus KEGG mithilfe der KEGG API

Neben dem Import von experimentellen Daten aus EnzymeML-Dateien und der manuellen Eingabe wurde eine weitere Pipeline implementiert, um Referenzreaktionen direkt aus der KEGG Datenbank über deren REST API zu beziehen und in den Wissensgraphen zu integrieren. Der Importprozess für eine spezifische KEGG-Reaktion wird durch die Eingabe ihrer eindeutigen KEGG-Reaktions-ID (z. B. R12516) in die entwickelte GUI ausgelöst und durch die Python-Funktion `process_single_kegg_reaction(reaction_id)` orchestriert (s. Codeabschnitt A.2 im Anhang). Diese Funktion kommuniziert zunächst mit der KEGG REST API über die requests-Bibliothek mittels einer HTTP GET-Anfrage, um den vollständigen Texteintrag für die angegebene Reaktions-ID abzurufen. Aus dieser Rohdateneingabe wird dann die für die Reaktion relevante EQUATION-Zeile extrahiert. Um die darin enthaltenen Informationen – stöchiometrische Koeffizienten und KEGG Compound IDs (CIDs) der Reaktanten – strukturiert zu erfassen, wird die Funktion `parse_kegg_equation` verwendet (s. Anhang Codeabschnitt A.1).

Die vorliegende Funktion veranschaulicht exemplarisch den mehrfachen Einsatz regulärer Ausdrücke (s. Abschnitt 3.2.2) zur semantischen Analyse von Datenbankeinträgen innerhalb der hier entwickelten Pipeline. Da die EQUATION-Zeile eines KEGG-Eintrags für eine Reaktion die Teilnehmer nur in einem semi-strukturierten Format wie



zur Verfügung stellt, wie Abbildung 4.11 zeigt, nutzt `parse_kegg_equation` Pythons re-Modul. Ein solcher KEGG-Eintrag ist exemplarisch im Folgenden abgebildet:

 REACTION: R12516 Help	
Entry	R12516 Reaction
Name	3-methylbutyl acetate acetohydrolase
Definition	Isoamyl acetate + H2O <=> 3-Methylbutanol + Acetate
Equation	C12296 + C00001 <=> C07328 + C00033
[n] Cxxxxx + [m] Cyyyyy <=> [k] Czxxxx + [i] Cwwwww	

Abbildung 4.11: Exemplarischer Datenbankeintrag für eine Reaktion aus der KEGG

Speziell wird u. a. die Funktion `re.findall()` mit dem auf die KEGG Compounds-IDs (CIDs) angepassten Muster `r"(\d*)\s*(C\d{5})"` auf die Edukt- bzw. Produktseite der Gleichung angewendet und liefert als Output so die CIDs sowie die stöchiometrischen Koeffizienten der Reaktanden.

Nach der Identifizierung der beteiligten CIDs werden deren Details (insbesondere Namen und SMILES-Codes) benötigt. Dies geschieht mit der Funktion `get_kegg_compound_details(list_of_cids)`. Analog zur Abfrage der Reaktionsdaten erfolgt ein weiterer API-Aufruf (`requests.get`) an die KEGG REST API, um die Einträge für alle beteiligten CIDs gesammelt abzurufen. Da KEGG-Compound-Einträge oft keine direkten SMILES-Informationen enthalten, wird hier, ähnlich wie beim manuellen Import für fehlende SMILES, auf die PubChem PUG REST API zurückgegriffen, um einen kanonischen SMILES-Code zu erhalten.

Um die Nachverfolgbarkeit und Distinktion zwischen Reaktionen aus verschiedenen Quellen zu gewährleisten, wird die Herkunft (`source_info`) als `rdfs:comment` gespeichert und der experimentelle Status über eine dedizierte Eigenschaft festgehalten. Diese Eigenschaft, `isExperimental`, ist als `owl:DatatypeProperty` mit dem Wertebereich (Range) `xsd:boolean` definiert. Der Wert dieser Eigenschaft wird beim Import anhand der übergebenen Quelleninformation (`source_info`) bestimmt: Beginnt dieser String mit „EnzymeML:“, wird der Wert auf 'true' gesetzt, andernfalls (bei KEGG- oder manuellen Importen) auf 'false'. Diese explizite, typisierte Markierung erlaubt eine zuverlässige Filterung der Reaktionen in SPARQL-Abfragen.

Überprüfung auf stöchiometrische Korrektheit bei manuellem Import von Reaktionsdaten

Zur Gewährleistung der Konsistenz und Korrektheit manuell importierter Referenzreaktionen (beispielsweise aus der Literatur oder anderen Datenbanken ohne standardisiertes Format) wurde eine Funktion zur Überprüfung der stöchiometrischen Ausgeglichenheit als Teil der Import-Pipeline implementiert. Während Daten aus EnzymeML oft keine vollständigen stöchiometrischen Angaben enthalten und KEGG-Gleichungen einer eigenen Kuratierung unterliegen, besteht bei der manuellen Eingabe ein höheres Risiko für Fehler bei der Angabe der Koeffizienten oder der Reaktanten selbst.

4.2.2 Substrukturanalyse mittels RDKit

Wahl der Moleküldarstellungen

Zu den gängigsten maschinenlesbaren Molekülnotationen gehören SMILES, SMARTS und InChI. Im Folgenden wird die Entscheidung für eine dieser Notationsarten abgewägt und gefällt. Ein signifikanter Vorteil von InChI besteht darin, dass es im Gegensatz zu SMILES immer eine konsistente und kanonisierte Darstellung bietet, was den Austausch zwischen verschiedenen Datenbanken erleichtert [43, 45]. Allerdings ist InChI nicht für Substruktursuchen optimiert. So ist es beispielsweise nicht möglich, gezielt nach Teilstrukturen zu suchen, da InChI eine vollständige Beschreibung des Moleküls liefert, aber keine Teilmuster unterstützt [60]. Ein weiterer Nachteil ist, dass die Zeichenkette oft lang und schwer für Menschen lesbar ist, was die direkte Anwendung in cheminformatischen Workflows erschwert [60]. Auch die Verarbeitung von InChI ist rechenaufwendiger als bei SMILES, da die Struktur erst dekodiert werden muss, bevor sie weiterverarbeitet werden kann, was insbesondere bei einer großen Anzahl an zu importierenden und zu analysierenden Reaktionen ins Gewicht fällt. Ein entscheidender Vorteil von SMILES gegenüber InChI liegt in seiner guten Lesbarkeit, welche das Formulieren von Substrukturen entscheidend vereinfacht. Zudem bietet die SMILES-Notation die Möglichkeit zur Substruktursuche, wobei ein weiterer großer Vorteil die Kompatibilität mit SMARTS ist, wodurch bei Bedarf eine detailliertere und spezifischere Substruktursuche durch erweiterte Syntax mit komplexeren Mustern möglich – jedoch nicht zwingend erforderlich ist. Dadurch vereint SMILES Flexibilität, Effizienz und Lesbarkeit, was es zur idealen Wahl für die Substruktursuchen im Rahmen dieser Arbeit macht.

Erkennung und Zuweisung der funktionellen Gruppen

Mithilfe von RDKit werden die zuvor gespeicherten SMILES einer Substrukturanalyse unterzogen, wobei zwei verschiedene Verfahren zum Einsatz kommen, die sich in Abhängigkeit von der zu suchenden Substruktur bzw. der molekularen Entität unterscheiden. Während bei sauerstoffhaltigen organischen Verbindungen (z. B. Ester, Alkohole und Carbonsäuren) gezielt nach funktionellen Gruppen oder Alkylresten innerhalb des Moleküls gesucht wird, ist dieses Verfahren für anorganische Verbindungen in den hier betrachteten Beispielen nicht sinnvoll anwendbar. Daher erfolgt die Identifikation von Wasser, Kationen und Basen durch direkte explizite Abfragen nach den jeweiligen Stoffen. Eine detaillierte Diskussion dieser Vorgehensweise findet sich im Abschnitt 4.2.2. Nachfolgend ist in Abbildung 4.5 die Funktion zur Analyse von organischen Substanzen auf Substrukturen dargestellt unter Nutzung der RD-Kit Funktion `GetSubstructMatches()`.

```

1 def get_functional_groups(smiles):
2     mol = Chem.MolFromSmiles(smiles)
3     patterns = {
4         "hydroxy": Chem.MolFromSmarts("[CX4][OH]"),
5         "ester": Chem.MolFromSmarts("C(=O)O[C;!$(C=[O,N,S]))"),
6         "carboxylic_acid": Chem.MolFromSmarts("C(=O)[OH]"),
7         "carboxylate_anion": Chem.MolFromSmarts("C(=O)[O-]"),
8         "glycerylrest": Chem.MolFromSmarts("[CH2][CH](O)[CH2]"),
9         "butylrest": Chem.MolFromSmarts("[CH2][CH2][CH2][CH3]"),
10        "ethylrest": Chem.MolFromSmarts("[CH2][CH3]"),
11        "3-methyl-butylrest": Chem.MolFromSmarts(
12            "[CH3][CH]([CH3])[CH2][C;D2]"),
13        "Acetat-Anteil": Chem.MolFromSmarts("[O][C](=O)[CH3]"),
14        "methylrest": Chem.MolFromSmarts(
15            "[CH3][C](=O)[O]"),
16        "Formylgruppe": Chem.MolFromSmarts(
17            "[CH](=O)[#6]"),
18    }
19    # Übereinstimmungen für jedes Muster finden
20    groups = {name: mol.GetSubstructMatches(pattern) for name, pattern in patterns
21               .items()}
22    return groups

```

Codeabschnitt 4.5: Substruktursuche für organische Verbindungen

Das Nutzen von `GetSubstructMatches()` innerhalb des oben gezeigten Codeabschnitts 4.5 ermöglicht eine gezielte Untersuchung von SMILES-Strukturen auf vordefinierte funktionelle Gruppen. Auf dieser Grundlage wurde die Funktion `get_functional_groups()` entwickelt. Die in 4.5 dargestellte Funktion analysiert alle SMILES der an einer Reaktion beteiligten Reaktanden systematisch. Hierbei werden vordefinierte Muster für funktionelle Gruppen wie Hydroxygruppen, Ester-, Carbonsäure- und Alkylreste genutzt. Die RDKit-Funktion `GetSubstructMatches()` ermittelt, welche der vordefinierten Musterstrukturen (*patterns*) in einer Struktur enthalten sind, und wie oft sie vorkommen. Die Wahl der zu identifizierenden Musterstrukturen basiert auf den exemplarisch herangezogenen Beispielreaktionen. Der Output der Funktion liefert eine strukturierte Übersicht über die Art und Häufigkeit der funktionellen Gruppen in jedem Reaktanden und speichert diese in der „groups“-Liste.

Zur Formulierung der Substrukturen ist, wie eingangs bereits erörtert, die SMILES bzw. SMARTS Notation als Moleküldarstellung gewählt worden. Bei den formulierten Mustern handelt es sich um Strukturen, die im Rahmen der hier gewählten Beispielreaktionen im Kontext der Esterhydrolysen zur Klassifizierung notwendig sind. Im Folgenden wird die angewendete Syntax exemplarisch anhand einiger Musterstrukturen erläutert. So lautet die für die `hydroxy-group` formulierte Substruktur beispielsweise `'[CX4][OH]'`. Die Verwendung der SMARTS-Restriktion `'[CX4]'` gewährleis-

tet, dass die Hydroxygruppe ausschließlich dann identifiziert wird, wenn sie an ein Kohlenstoffatom mit vier Einfachbindungen gebunden ist (sp^3 -hybridisiert, s. dazu Tabelle 3.2). Diese Einschränkung ist notwendig, um ausschließlich Alkohole zu erfassen und OH-Gruppen in Carbonsäuren oder Phenolen auszuschließen, da diese an sp^2 -hybridisierte Kohlenstoffatome gebunden sind [61].

Auch bei der für die Estergruppe gewählten Darstellung `'C(=O)O[C;!$(C=[O,N,S])]`' musste erweiterte SMARTS Syntax genutzt werden. Die so gewählte Substruktur sorgt dafür, dass es sich bei dem an den Sauerstoff gebundenen Atom um ein Kohlenstoff handeln muss. Damit wird ebenfalls sichergestellt, dass diese Substruktur nicht fälschlicherweise in einem Carbonsäure Molekül erkannt wird, da dort an gleicher Stelle eine Hydroxygruppe anhängt. Zusätzlich dazu wird verboten, dass der an den Ester-Sauerstoff gebundene Kohlenstoff eine Doppelbindung ausbildet. Dies ist notwendig, um Carbonsäurederivate wie beispielsweise das Ameisensäureanhydrid mit der SMILES Darstellung `C(=O)OC=O` aus den Ergebnissen der Suche auszuschließen [62].

Die Formulierung der Alkylreste hingegen ist in der Regel mit weniger Modellierungsaufwand verbunden. Der Butylrest wird beispielsweise durch die Struktur `'[CH2][CH2][CH2][CH3]'` dargestellt. Kettenbeginn- sowie -länge können dabei durch die Angabe der Anzahl an gebundenen Wasserstoffatomen pro Kohlenstoff festgelegt werden. Der Glycerylrest stellt unter den Alkylresten eine Besonderheit dar, da er im Rahmen der hier gewählten Beispielreaktionen sowohl im dreiwertigen Alkohol Glycerin, als auch im Triglycerid Triacetin erkannt werden sollte. Das Muster `'[CH2][CH](O)[CH2]'` enthält explizite Anforderungen an die mittlere CH-Gruppe, welche sicherstellen, dass dort eine Hydroxy- oder Estergruppe vorhanden ist.

Nachdem die funktionellen Anteile nun erkannt worden sind, müssen diese den entsprechenden Reaktionskomponenten noch zugeordnet und in den Wissensgraph importiert werden. Dazu werden jeweils für Edukte und Produkte die maximale Anzahl an funktionalen Anteilen identischer Gestalt gezählt. Daraufhin werden über owlready2 für jeden funktionalen Anteil die Instanzen in der Häufigkeit des maximalen Vorkommens innerhalb eines Moleküls erstellt. Im Falle von Glycerin würden so also die drei Instanzen `single hydroxy group` bis `triple hydroxy group` erstellt werden. Weiterhin werden die erstellten Instanzen, die die funktionalen Anteile beschreiben direkt bei Import ihrer gleichnamigen Subklasse zugeordnet werden, um im weiteren Verlauf trennscharfes Reasoning zu ermöglichen. Die dafür relevante `for`-Schleife ist in Codeabschnitt 4.6 dargestellt.

```

1 # Create instances for functional groups in the maximum frequency
2     for group_name, count in max_functional_groups.items():
3         for i in range(1, count + 1):
4             group_instance_name = f"{group_name}_{i}fach"
5             group_instance = group_sub(group_instance_name)
6             print(f"Created instance: {group_instance_name}")
7
8         # Zuordnung der hydroxy_ifach Instanz zu den entsprechenden Subklassen
9         if group_name == "hydroxy":
10             sub_class_name = f"OH_{['einfach', 'zweifach', 'dreifach'][i-1]}"
11             sub_class = onto.search_one(iri=f"*{sub_class_name}")
12             if sub_class:
13                 group_instance.is_a.append(sub_class)
14                 print(f"Assigned {group_instance_name} to {sub_class_name}")
15             else:
16                 print(f"Sub-class {sub_class_name} not found.
17                       Please check the ontology.")
18
19

```

Codeabschnitt 4.6: Erstellung der Instanzen für funktionale Anteile

In einem letzten entscheidenden Schritt müssen die identifizierten funktionalen Anteile innerhalb des Wissensgraphen in eine Eigenschaftsbeziehung mit den Instanzen für die Reaktionspartner gebracht werden. Auch das geschieht unter Nutzung von `owlready2`. Um die Relation zwischen Substanzen und ihren funktionalen Anteilen innerhalb der Ontologie semantisch zu beschreiben, wird die neue Objekteigenschaft `has_functional_part` definiert, deren Domäne alle Substanzen sind, also die Klasse der `molecular entity` und deren Range die Subklasse `groups` ist, der alle funktionalen Anteile angehören. Es findet eine Iteration über alle Substanzen und für jede Substanz jeweils über alle funktionellen Anteile statt, wobei die funktionellen Anteile in entsprechender Häufigkeit des Auftretens über `has_functional_part` mit den Substanzen in Beziehung gesetzt werden. Damit ist eine weitere Voraussetzung für das automatische Reasoning erfüllt.

Identifizierung und Klassifizierung der anorganischen Reaktanden

Wie zuvor bereits erläutert, ist das Identifizieren von mehreren funktionalen Anteilen innerhalb des Moleküls nicht bei jedem Reaktanden sinnvoll. Darunter fallen beispielsweise Wasser, Metall-Kationen, Basen sowie molekularer Sauerstoff. Um diese Reaktanden trotzdem automatisiert klassifizieren zu können, wurden Hilfsfunktionen definiert, die die SMILES Darstellung prüfen und die Instanzen ggf. zur korrekten Klasse zuordnen. Diese Vorgehensweise wird im folgenden Codeabschnitt 4.7 beispielhaft anhand der Prüfung auf Sauerstoff erläutert:

```

1 def is_oxygen(smiles, oxygen_smarts_list=["[#8]=[#8]"]):
2     mol = Chem.MolFromSmiles(smiles)
3     if not mol:
4         return False
5     # Prüfe auf exakt 2 Atome
6     if mol.GetNumAtoms() != 2:
7         return False
8     # Prüfe auf O=O SMARTS
9     if check_substructure(smiles, oxygen_smarts_list[0]):
10        return True
11    return False
12

```

Codeabschnitt 4.7: Automatische Klassifizierung der anorganischen Reaktanden

Die oben dargestellte Funktion `is_oxygen` prüft direkt den vollständigen SMILES auf Übereinstimmung mit der SMARTS Darstellung von molekularem Sauerstoff, während die Darstellung zuvor auf einzelne Substrukturen hin untersucht wurde. Weiterhin wird die Anzahl der Atome geprüft, um falsch positive Ergebnisse zu verhindern. Diese Funktion wird während des Importvorgangs in folgendem Codeabschnitt 4.8 abgerufen:

```

1 if oxygen_class and isinstance(oxygen_class, ThingClass):
2     if is_oxygen(smiles): # is_oxygen Funktion von oben verwenden
3         if oxygen_class not in substance_instance.is_a:
4             substance_instance.is_a.append(oxygen_class)
5             print(
6                 f"      [OK] '{name}' als Sauerstoff (CHEBI_15379) klassifiziert.")
7

```

Codeabschnitt 4.8: Automatische Zuweisung der Klassen für anorganische Reaktanden

In Rahmen der oben dargestellten `for`-Schleife, werden bei erfolgreicher Prüfung durch die Funktion `is_oxygen` die entsprechenden Instanzen mithilfe von `owlready2` der entsprechenden Klasse (in diesem Fall Sauerstoffklasse) zugeordnet, sofern diese in der Ontologie aufgefunden wird. Somit ist gewährleistet, dass auch die anorganischen Reaktanden im Rahmen dieser Import-Pipeline automatisch klassifiziert werden, sodass auf Basis dieser Erkenntnisse Reasoning betrieben werden kann.

Definition der Substanzklassen

Ausstehend sind bislang die Klassendefinitionen für die jeweiligen Substanzen, da diese unmittelbar von der Fähigkeit zur Substrukturanalyse und der expliziten Gestalt der dazugehörigen Analyseergebnisse abhängig sind. Im Folgenden wird anhand einiger Beispielklassen im Kontext der Beispielreaktion der Verseifung von Triacetin dargelegt, wie die aus der Substrukturanalyse stammenden

Eigenschaftsbeziehungen zwischen den Instanzen nun zur Definition der Klassenrestriktionen für die jeweiligen Substanzen verwendet werden können, um im Ergebnis eine erfolgreiche Zuordnung der Substanzen zu ermöglichen. Die folgende Abbildung 4.12 zeigt die Klassenrestriktionen für die Klassen der Glycerine sowie der Triacetine.

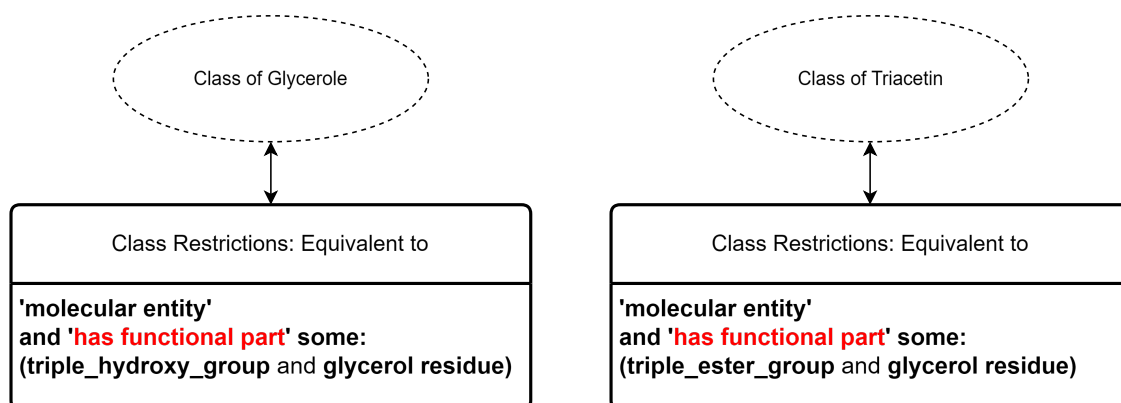


Abbildung 4.12: Klassenrestriktionen für Glycerin und Triacetin

Wie Abbildung 4.12 zu entnehmen ist, werden für die Klasse der Glycerine nun einerseits das Vorhandensein von drei Hydroxygruppen, und andererseits das Vorhandensein eines Glycerylrests als distinkte Eigenschaften des dreiwertigen Alkohols definiert. Analog dazu wird für das Triacetin das Vorhandensein von drei Estergruppen und dem Glycerylrest als strukturbestimmende Eigenschaft definiert. Nach dem gleichen Prinzip, das bei den Reaktionsklassen Anwendung findet, werden auch diese Klassen als definiert statt primitiv festgelegt, um das Reasoning bei Erfüllen aller Bedingungen zu ermöglichen.

4.2.3 SHACL-SPARQL-Abfrage

Nachdem nun der Import sowie die Analyse der Reaktionsdaten aus verschiedenen Quellen erläutert wurde, wird nun das Vorgehen der angedachten SHACL-SPARQL-Abfrage erläutert, die Relationen zwischen ähnlichen Reaktionen identifizieren soll. In den folgenden Code-Beispielen und Abfragen werden häufig abgekürzte Identifikatoren wie `obo:MOP_0000543` verwendet, deren Bedeutung im Folgenden kurz erläutert wird. Es handelt sich hierbei um sogenannte CURIEs (Compact URIs/IRIs). Diese Methode stellt eine Standardisierung dar, um ausgedehnte Web-Identifikatoren (IRIs) mittels der Kombination eines definierten Präfixes (beispielsweise `obo` für den Namespace `http://purl.obolibrary.org/obo/`) mit einem lokalen Namen (beispielsweise `MOP_0000543`) abzukürzen [63]. Dies resultiert in einer signifikanten Verbesserung der Lesbarkeit von Code und Abfragen.

Zur Sicherstellung der Qualität und strukturellen Konsistenz der Reaktionsdaten im Wissensgra-

phen wurde vorerst eine Validierungslogik mittels SHACL implementiert. Ziel dieser Validierung ist es, ausschließlich jene Reaktionen zu identifizieren, die über mindestens ein Edukt und ein Produkt verfügen und der Oberklasse `molecular process` (MOP_0000543) zugeordnet sind. Diese Prüfung ist essenziell, um sicherzustellen, dass ausschließlich vollständig definierte Reaktionen in den nachfolgenden semantischen Analysen berücksichtigt werden. Die Validierung erfolgt unter Verwendung der Bibliothek pySHACL, wobei zwei RDF-Graphen erzeugt werden: der Daten-Graph, bestehend aus den idealisierten Ontologie-Reaktionen sowie den experimentellen Reaktionen, und der SHACL-Shape-Graph, der die zu überprüfenden Regeln definiert. Das verwendete NodeShape, dargestellt in Codeabschnitt 4.9, adressiert die Zielklasse `molecular process` und spezifiziert mittels `sh:minCount`, dass sowohl mindestens ein Edukt (`obo:BFO_0000078`) als auch mindestens ein Produkt (`obo:BFO_0000077`) vorhanden sein muss.

```
1 # SHACL-Shape zur Identifikation relevanter Reaktionen
2 ex:ReactionIdentificationShape
3     a sh:NodeShape ;
4     sh:targetClass obo:MOP_0000543 ; # Ziel: Reaktionen der Klasse "molecular process"
5
6     # Bedingung: Mindestens ein Edukt vorhanden
7     sh:property [
8         sh:path obo:BFO_0000078 ; # Edukte
9         sh:minCount 1 ; # Mindestens ein Edukt erforderlich
10    ] ;
11
12    # Bedingung: Mindestens ein Produkt vorhanden
13    sh:property [
14        sh:path obo:BFO_0000077 ; # Produkte
15        sh:minCount 1 ; # Mindestens ein Produkt erforderlich
16    ] .
17 # SHACL-Graph erstellen (Regeln)
18 shacl_graph = Graph()
19 shacl_graph.parse(data=Valid, format="turtle")
20
21 # SHACL-Validierung durchführen
22 conforms, results_graph, results_text = validate(
23     data_graph=data_graph,
24     shacl_graph=shacl_graph,
25     inference='rdfs',
26     advanced=True
27 )
28
```

Codeabschnitt 4.9: SHACL-Validierung

Nach erfolgreicher Validierung ermöglicht es die im Folgenden dargestellte simple SPARQL-SELECT-

Abfrage 4.10, auf sämtliche konformen Reaktionen zuzugreifen. Die Verwendung der **SELECT**-Klausel ermöglicht es, die URIs aller Reaktionen abzufragen, die den im SHACL-Shape definierten Bedingungen entsprechen:

```
1 SELECT ?reaction
2 WHERE {
3     ?reaction a obo:MOP_0000543 ;
4               obo:BF0_0000078 ?educt ;
5               obo:BF0_0000077 ?product .
6 }
7
8 results = data_graph.query(query)
9
```

Codeabschnitt 4.10: Ausgeben der validen Reaktionen

Im Anschluss wird eine gezielte semantische Analyse durchgeführt, bei der experimentelle Reaktionen mit idealisierten Referenzreaktionen verglichen werden. Ziel ist die Identifizierung von Reaktionspaaren, bei denen die experimentelle Reaktion alle Edukte und Produkte der idealen Reaktion enthält. Wie im Abschnitt 4.2.1 erläutert, werden die Reaktionsinstanzen bereits im Zuge des Imports durch die boolsche Eigenschaft **isExperimental**: [true / false] markiert, um eine Unterscheidung zwischen den zu vergleichenden Reaktionsinstanzen zu ermöglichen. Um fälschliche Zuordnungen oder Selbstreferenzen zu vermeiden, wird die SPARQL-Query mittels mehrerer **FILTER**- und **FILTER NOT EXISTS**-Konstrukte angereichert, wie im folgenden Codeauszug 4.11 dargestellt wird:

```

1 SELECT DISTINCT ?ExpReaktion ?idealReaction ?expLabel ?idealLabel
2     WHERE {{
3         # "Ideale" Reaktionen (explizit als false markiert)
4         ?idealReaction a obo:MOP_0000543 ;
5             ex:isExperimental false ;
6         OPTIONAL {{ ?idealReaction rdfs:label ?idealLabel }}
7
8         # Experimentelle Reaktionen (als true markiert)
9         ?ExpReaktion a obo:MOP_0000543 ;
10            ex:isExperimental true ;
11        OPTIONAL {{ ?ExpReaktion rdfs:label ?expLabel }}
12
13        # Vermeide Selbstreferenzen
14        FILTER(?ExpReaktion != ?idealReaction)
15
16        # Bedingung: ALLE Edukte der idealen (false) Reaktion müssen auch Edukte
17        der experimentellen (true) sein.
18        FILTER NOT EXISTS {{
19            ?idealReaction obo:BF0_0000078 ?idealEduct .
20            FILTER NOT EXISTS {{ ?ExpReaktion obo:BF0_0000078 ?idealEduct . }}
21        }}
22
23        # Bedingung: ALLE Produkte der idealen (false) Reaktion müssen auch
24        Produkte der experimentellen (true) sein.
25        FILTER NOT EXISTS {{
26            ?idealReaction obo:BF0_0000077 ?idealProduct .
27            FILTER NOT EXISTS {{ ?ExpReaktion obo:BF0_0000077 ?idealProduct . }}
28        }}
29        ORDER BY ?ExpReaktion ?idealReaction
30
31

```

Codeabschnitt 4.11: SPARQL-Abfrage zur Ähnlichkeitsanalyse

Die in Codeabschnitt 4.11 dargestellte **FILTER**-Klausel ist ein wesentlicher Bestandteil von SPARQL-Abfragen, und dient dazu, die Ergebnismenge solcher Triple-Muster einzuschränken, die spezifische Bedingungen erfüllen. Im vorliegenden Fall gewährleistet sie, dass experimentelle Reaktionen nicht mit sich selbst verglichen werden (`FILTER(?ExpReaktion != ?idealReaction)`) und dass die idealen Reaktionen nicht experimentell markiert sind. Die zentrale Logik zur Überprüfung, ob alle Edukte und Produkte der Referenzreaktion auch in der experimentellen Reaktion vorhanden sind, basiert auf verschachtelten **FILTER NOT EXISTS**-Konstrukten. Diese Konstrukte ermöglichen die Negation innerhalb der semantischen Logik und dienen dazu, Reaktionspaare herauszufiltern, bei denen keine der idealen Komponenten fehlt. Die implementierte verschachtelte Struktur ermöglicht

somit eine universelle Mengenprüfung („für alle Edukte/Produkte“), die mit einer reinen **FILTER**-Anwendung nicht realisierbar wäre. Dieses Modell gewährleistet, dass sämtliche Komponenten der idealen Reaktion auch in der experimentellen Reaktion enthalten sind. Sollte auch nur ein Edukt oder Produkt nicht gefunden werden, so wird das entsprechende Reaktionspaar vom Resultat ausgeschlossen.

Die Verwendung einer **DISTINCT**-Klausel im Query-Header garantiert, dass jede gefundene Reaktionspaarung nur einmal im Ergebnis auftritt. Dies ist auch dann gewährleistet, wenn sich innerhalb des RDF-Datenmodells mehrere redundante Informationen in den Tripelpaaren befinden.

Die finale Abfrage generiert eine strukturierte Liste valider Reaktionspaarungen, die als Grundlage für weiterführende Analysen dient.

Zusammenfassend wurden in diesem Kapitel die entwickelten Importwege zur Erfassung heterogener Reaktionsdaten detailliert beschrieben und die Methoden zur Strukturierung und semantischen Anreicherung dieser Informationen innerhalb der erweiterten Ontologie dargestellt. Damit ist die methodische Grundlage geschaffen, um die im Ergebnisteil folgenden domänenspezifischen Analysen und Auswertungen, insbesondere die automatische Reaktionsklassifikation mittels Reasoning sowie Validierungen und Ähnlichkeitssuchen mittels SHACL und SPARQL durchzuführen und deren Mehrwert für die Datenintegration anhand von Beispielreaktionen zu demonstrieren.

5 Ergebnisse

In diesem Kapitel werden die zentralen Ergebnisse der Arbeit präsentiert. Die in dieser Arbeit entwickelten Funktionalitäten, wie der Import von Reaktionsdaten (aus EnzymeML, KEGG oder manuell) und die nachfolgenden Analysen, stehen den Anwendern über eine entwickelte grafische Benutzeroberfläche (GUI) zur Verfügung. Zunächst werden die Resultate der funktionellen Gruppenanalyse mittels RDKit präsentiert, deren korrekte Zuweisung exemplarisch validiert und tabellarisch veranschaulicht wird. Darauf aufbauend folgen die Ergebnisse des automatisierten Reasonings, welches eine korrekte Klassifizierung der Reaktionen basierend auf ihren Merkmalen ermöglicht. Abschließend wird die semantische Validierung und Abfrage mittels SHACL und SPARQL demonstriert.

5.0.1 Aufzeigen des Imports

Für den Anwender besteht die Möglichkeit Reaktionsdaten auf verschiedene Weisen in den Wissensgraph zu importieren. Einerseits über das Hochladen einer EnzymeML Datei, aber auch durch manuelle Angabe der Reaktionsdaten aus Literatur oder Angabe einer Reaktions-ID für eine aus der KEGG stammende Reaktion. Die Eingabemaske für die verschiedenen Import-Varianten ist im Folgenden in Abbildung 5.1 dargestellt:

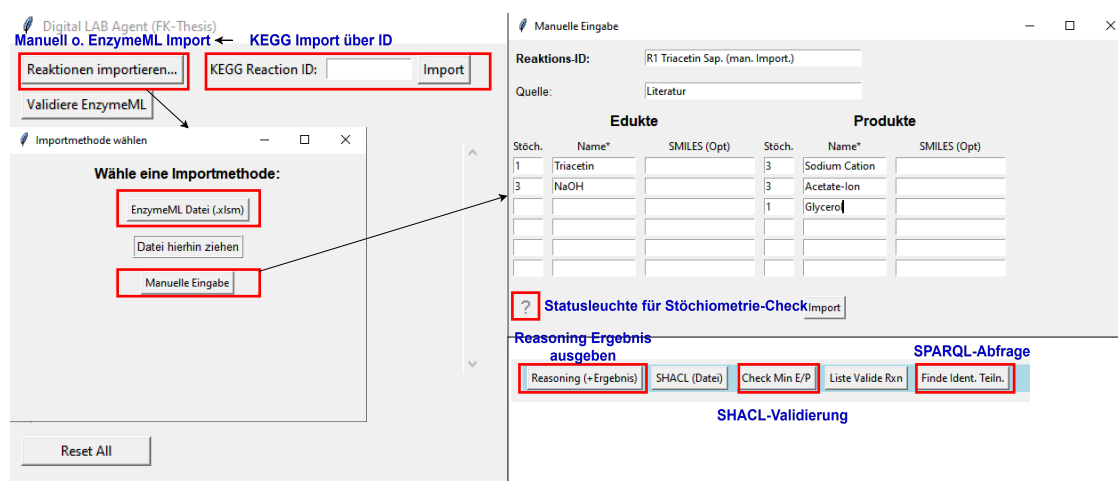


Abbildung 5.1: GUI-Eingabefenster für verschiedene Import-Optionen

Neben den Importpipelines sind in Abbildung 5.1 ebenso die Schaltflächen zum Anstoß der verschiedenen Analysemethoden durch den Anwender für Reasoning, SHACL-Validation sowie der SPARQL-Abfrage dargestellt.

Nach dem in Abbildung 4.10 beschriebenen Vorgehen werden die Instanzen erstellt und zueinander in Beziehung gebracht. Die folgende Abbildung 5.2 zeigt die Ergebnisse des Importvorgangs der beiden Beispielreaktionen R1 (Triacetin-Verseifung) und R2 (Butylacetat-Spaltung) mithilfe der OntoGraf-Funktion Protege:

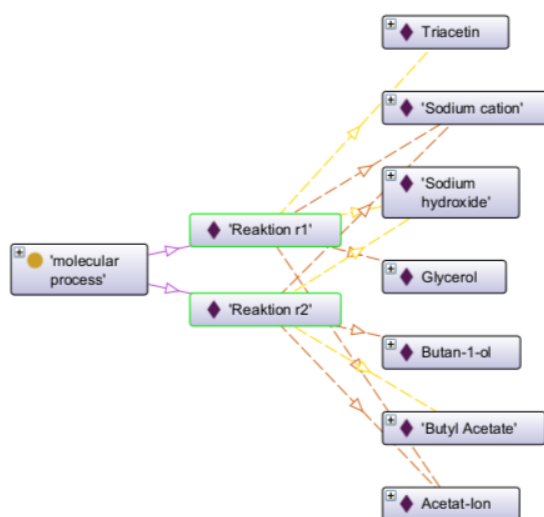


Abbildung 5.2: OntoGraf für importierte Beispielreaktionen

Im Fokus der Darstellung befinden sich die beiden Instanzen **R1** und **R2**, welche die importierten Reaktionen repräsentieren. Wie durch die Verbindung zur Klasse `molecular process` ersichtlich ist, wurden diese korrekt als Instanzen der entsprechenden Reaktionsklasse (`obo:MOP_0000543`) im Wissensgraphen angelegt. Die gestrichelten Pfeile, die von den Reaktionsinstanzen ausgehen, zeigen die ebenfalls während des Imports erstellten oder gefundenen Instanzen der beteiligten Substanzen. Diese Verknüpfungen repräsentieren die in der Ontologie etablierten Beziehungen zwischen einer Reaktion und ihren Teilnehmern, welche über die BFO-Objekteigenschaften `obo:BFO_0000078` (`has participant ceasing to exist` für Edukte, in gelb dargestellt) und `obo:BFO_0000077` (`has participant beginning to exist` für Produkte, in orange dargestellt) modelliert wurden. Die Abbildung 5.2

demonstriert somit, dass die Kernentitäten der Reaktionen, die Prozesse selbst sowie die beteiligten molekularen Entitäten, durch die Importpipeline korrekt instanziiert und grundlegend miteinander verknüpft werden. Dies bildet die notwendige Basis für die nachfolgenden Analysen, wie die detailliertere Substrukturanalyse und das Reasoning.

Eine zusätzliche Funktion während des manuellen Imports ist die Überprüfung auf Korrektheit der Stöchiometrie. Anwender haben im GUI die Möglichkeit Reaktionen manuell auf die in Abbildung 5.1 dargestellte Weise anzugeben. Die stöchiometrische Überprüfung nutzt primär die Funktion `get_atom_counts`, welche mit RDKit die Anzahl jedes Atoms (inklusive Wasserstoff durch `Chem.AddHs`, da diese normalerweise implizit vorhanden sind) für einen gegebenen SMILES-String ermittelt. Die Funktion `check_manual_stoichiometry` ruft `get_atom_counts` für alle Edukte und Produkte auf, summiert die Atombilanzen pro Seite und vergleicht diese. Für den Anwender ergibt sich innerhalb der GUI ein Output für das Ergebnis der Prüfung der Art: „[INFO] Verarbeite 'Triacetin Verseifung (man. Import)'... Starte Stöchiometrie-Prüfung für 'Triacetin Verseifung (man. Import)'... -> Ergebnis: Stöchiometrie OK für 'Triacetin Verseifung (man. Import)'“

5.0.2 Ergebnisse der Substrukturanalyse

Nachdem die Import-Pipelines die Reaktionsdaten erfolgreich in den Wissensgraphen überführt haben, fokussiert dieser Abschnitt auf die Ergebnisse der automatisierten Substrukturanalyse. Wie in Kapitel 4.2.2 dargelegt, wurde RDKit mit spezifischen SMARTS-Mustern eingesetzt, um die funktionellen Gruppen und strukturellen Merkmale der an den Beispielreaktionen (Verseifung von Triacetin und Butylacetat) beteiligten Substanzen zu identifizieren. Zusätzlich wird eine aus der KEGG Datenbank stammende Reaktion zur Hydrolyse von Isoamylacetat herangezogen, um im weiteren Verlauf die Funktionalitäten der SHACL-SPARQL-Abfrage aufzuzeigen. Diese Analyse ist von entscheidender Bedeutung für die spätere semantische Klassifizierung der Reaktionen durch Reasoning. Die detaillierten Erkennungsergebnisse werden zunächst anhand der nachfolgenden Abbildung 5.3 dargestellt. Im Anschluss wird dargelegt, wie diese Informationen zur Erstellung und Verknüpfung entsprechender Instanzen im Wissensgraphen genutzt wurden.

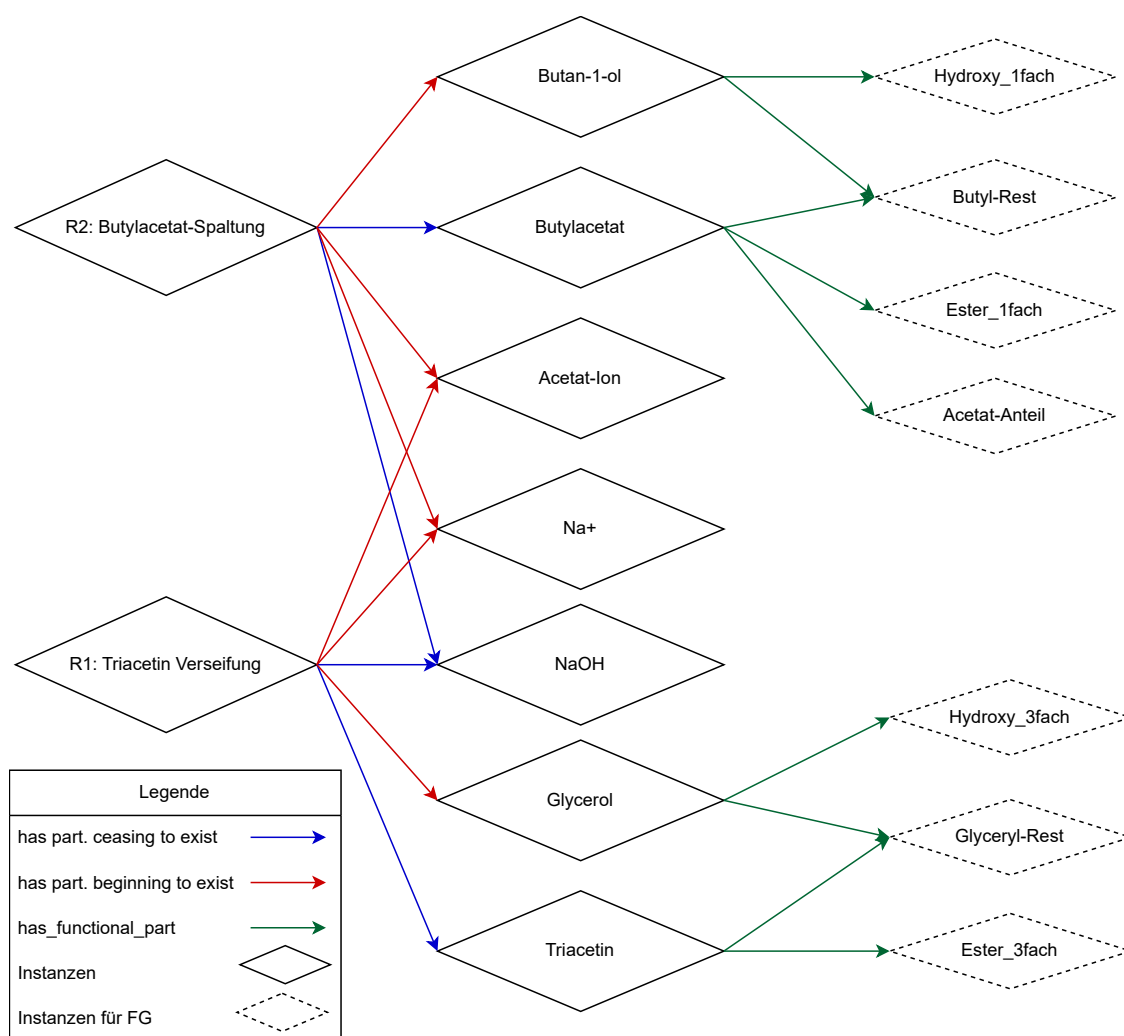


Abbildung 5.3: Substrukturanalyse-Ergebnisse und Verknüpfung der Instanzen

Wie der Abbildung 5.3 zu entnehmen ist, werden die Substanz-Instanzen den Reaktionsinstanzen korrekt über die vorher definierten Eigenschaften für Edukte und Produkte zugewiesen. Ebenso werden die vorher definierten Substrukturen in der korrekten Häufigkeit erkannt und den jeweiligen Substanzen zugewiesen. Für die Verseifung von Triacetin (R1) wurden für das Edukt Triacetin beispielsweise drei Estergruppen (**Ester_3fach**) sowie der **Glyceryl-Rest** identifiziert. Wie in der Abbildung 5.3 durch die grünen Pfeile (**has_functional_part**) dargestellt, werden diese erkannten Anteile als Instanzen modelliert (mit gestricheltem Rand dargestellt) und mit der entsprechenden Substanz-Instanz verknüpft. Wie im Methodik-Teil bereits angedeutet, werden anorganische bzw. ionische Komponenten, wie das Natrium-Kation oder NaOH, keiner Substrukturanalyse unterzogen, sondern bereits im Zuge des Imports ihrer jeweiligen Zielklasse zugeordnet.

5.0.3 Reasoningergebnisse

Aufbauend auf den importierten und mit funktionellen Gruppen verknüpften Reaktionsdaten wird in diesem Abschnitt die Fähigkeit des entwickelten Systems zur automatischen Klassifikation durch logisches Rückschließen (Reasoning) demonstriert. Wie in der Methodik beschrieben, wurden für spezifische Substanz- und Reaktionstypen wie die Fettverseifung und die einfache Esterhydrolyse OWL2-Klassen mit Äquivalenzaxiomen (**Equivalent-To**) definiert, die auf den beteiligten Reaktandenklassen basieren. Nach Ausführung des HermiT-Reasoners (**sync_reasoner_hermit**), der über die Funktion **perform_reasoning_and_display_results** (siehe GUI) aufgerufen wird, werden die Ergebnisse dieser automatischen Klassifikation für die Beispielreaktionen dargestellt.

Der Ausgangszustand vor dem Reasoning ist in Abbildung 5.2 dargestellt. Bis auf wenige bereits diskutierte Ausnahmen für anorganische Komponenten, sind sowohl die Reaktanden, als auch die Reaktionsinstanzen nach Import nur den generischen Subklassen **molecular process** bzw. **molecular entity** zugeordnet. Voraussetzung für das Reasoning der Reaktionen ist vorerst die korrekte Klassifizierung der Substanzen. Auf Basis der zugeordneten funktionalen Anteile, wie sie in Abbildung 5.2 dargestellt sind, liefert das Reasoning für die Reaktanden der Beispielreaktionen folgende Ergebnisse:

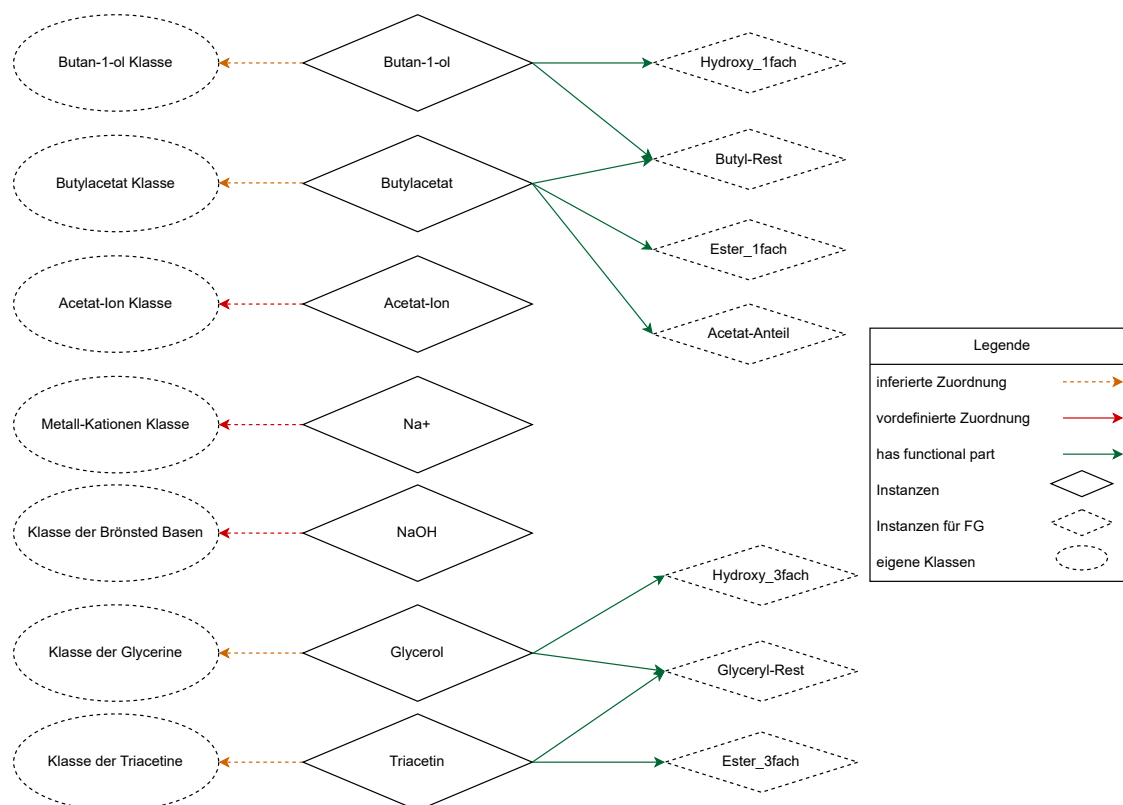


Abbildung 5.4: Klassenzuordnung der Substanzen durch Reasoning

Dieses inferierte Wissen über die spezifischen Typen der Substanzen bildet das Fundament für das anschließende Reasoning der Reaktionsinstanzen. Der Reasoner kann nun die Äquivalenzaxiome der spezifischen Reaktionsklassen (s. Abschnitt 4.1.3) evaluieren, da diese Axiome auf den nun bekannten Typen der über obo:BFO_0000078 (has participant ceasing to exist) und obo:BFO_0000077 (has participant beginning to exist) verknüpften Teilnehmer basieren.

Ergebnis-Output für Reasoning durch HermiT im GUI

```

🔧 Starte Reasoner (HermiT)...
✅ Reasoning abgeschlossen.
📊 Vergleiche Klassen vor und nach Reasoning...

--- Ergebnisse für Reaktion R12516 (Reaction_R12516) ---
✳️ Neu inferierte Klassen:
  - Ester_Hydrolysis_Reaction_(neutral) (Ester_Hydrolysis_Reaction_(neutral))

--- Ergebnisse für Reaktion r0 (Reaction_r0) ---
✳️ Neu inferierte Klassen:
  - Ester_Hydrolysis_Reaction_(neutral) (Ester_Hydrolysis_Reaction_(neutral))

--- Ergebnisse für Reaktion r1 (Reaction_r1) ---
✳️ Neu inferierte Klassen:
  - Fat Saponification (Fat_Saponification)

--- Ergebnisse für Reaktion r2 (Reaction_r2) ---
✳️ Neu inferierte Klassen:
  - Ester Hydrolysis Reaction (base catalyzed) (EsterHydrolysis)
  
```

Abbildung 5.5: Ergebnisse für durch HermiT Reasoner neu inferierte Klassen

Wie Abbildung 5.5 zeigt, werden die Beispielreaktionen R1 und R2 erfolgreich vom HermiT Reasoner als Fettverseifung bzw. als basenkatalysierte Esterhydrolyse klassifiziert. Dies demonstriert die Fähigkeit des Systems, die komplexen, in OWL definierten Klasseneinschränkungen korrekt auszuwerten und die Reaktionen allein auf Basis der zuvor importierten und angereicherten Teilnehmerinformationen automatisch der passenden semantischen Klasse zuzuordnen.

5.0.4 SHACL-SPARQL-Abfrage

Dieser Abschnitt demonstriert die kombinierte Anwendung von SHACL zur Datenvalidierung und SPARQL zur Identifikation semantisch ähnlicher Reaktionen, wie in der Methodik (s. Abschnitt 4.2.3) beschrieben. Ziel ist es zu zeigen, wie zunächst die grundsätzliche Korrektheit der importierten Reaktionen sichergestellt werden kann und anschließend gezielt nach Übereinstimmungen zwischen experimentellen und idealisierten Reaktionen gesucht werden kann. Die hier beschriebenen Funktionalitäten sind in die graphische Nutzeroberfläche integriert worden, allerdings bleibt die Funktionsweise identisch zu dem im Methodikteil beschriebenen Vorgehen. Als Beispielszena-

rio wurde die Referenzreaktion R12516 aus der KEGG-Datenbank über die entwickelte KEGG-Importpipeline in den Wissensgraphen geladen, welche folgend in Abbildung 5.6 dargestellt ist:

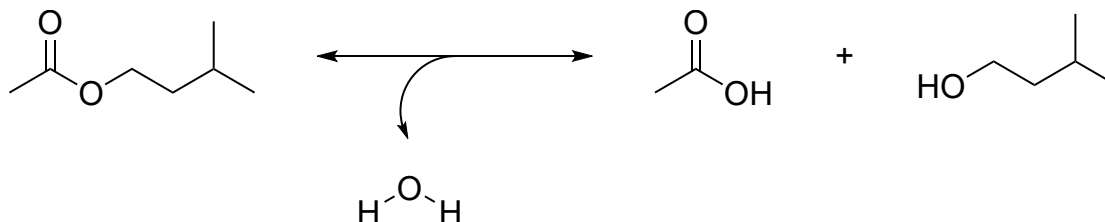


Abbildung 5.6: KEGG-Reaction R12516: Hydrolyse von Isoamylacetat [64]

Wie Abbildung 5.6 zeigt, handelt es sich hierbei, genau wie bei den bisher eingeführten Beispielreaktionen, um eine weitere Esterspaltung. Entsprechend der im Methodikteil beschriebenen Implementierung wurde dieser Reaktionsinstanz automatisch die Eigenschaft `isExperimental = false` zugewiesen. Zusätzlich wurde eine hypothetische experimentelle Durchführung dieser Reaktion aus EnzymeML importiert, welche im Folgenden abgebildet ist:

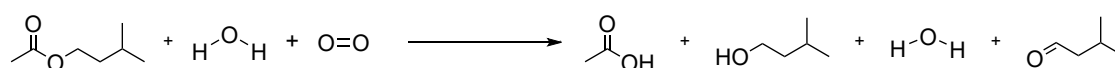


Abbildung 5.7: Experimentelle Durchführung der Isoamylacetat-Hydrolyse mit zusätzlichen Produkten

Bei Durchführung der in Abbildung 5.7 dargestellte Reaktion wird Isoamylaldehyd als wesentliches zusätzliches Produkt beobachtet, welches durch Oxidation des Hauptproduktes Isoamylol entsteht. Die Gleichung beschreibt lediglich die beobachtete Gesamttransformation sowie Reaktanden und enthält keinerlei Informationen über stöchiometrische Verhältnisse oder differenziert zwischen Haupt- und Neben- bzw. Folgereaktion. Diese Instanz wurde entsprechend ihrer Herkunft mit `isExperimental = true` gekennzeichnet. Nach erfolgreichem Import der experimentellen und Referenzdaten über die bereits erläuterten Pipelines, beinhaltet der Wissensgraph nun folgende in Abbildung 5.8 dargelegte Informationen und Relationen:

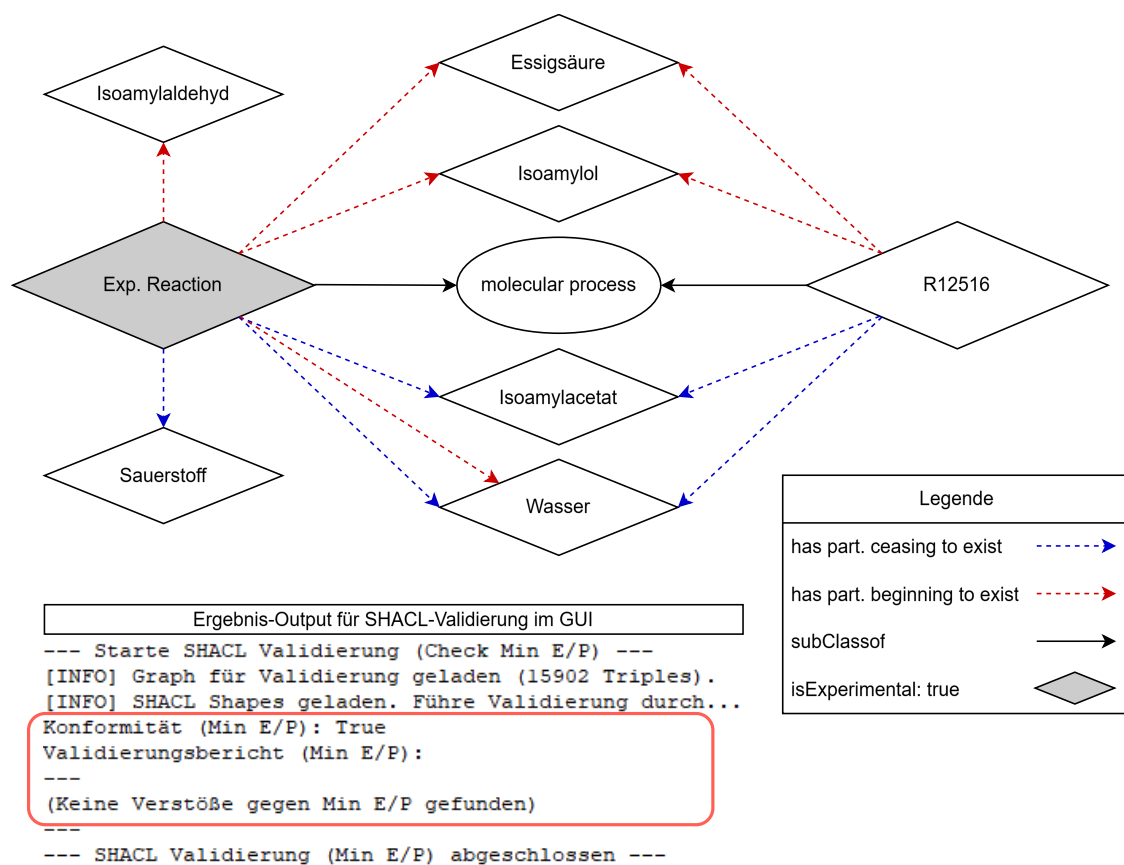


Abbildung 5.8: Reaktionsinstanzen und Relationen innerhalb des KG nach Import von Reaktionsdaten

Wie Abbildung 5.8 zu entnehmen ist, sind die beiden Reaktionsinstanzen erfolgreich der Superklasse `molecular process` zugeordnet worden und sind ebenfalls durch die entsprechende Eigenschaftsbeziehungen mit ihren jeweiligen Reaktionsteilnehmern in Relation gesetzt worden. Zuerst gibt es nun die Möglichkeit die strukturelle Validität aller importierten Reaktionsinstanzen mittels der fest codierten SHACL-Regeln (s. Codeabschnitt 4.9) zu überprüfen. Abgerufen wird diese SHACL-Abfrage ebenfalls über die GUI mithilfe der Funktion `run_hardcoded_shacl_validation` (s. Github). Wie Abbildung 5.8 zeigt, sind die in der Abfrage definierten Bedingungen des Vorhandenseins von jeweils mind. einem Edukt und Produkt sowie die Zugehörigkeit zur Klasse `molecular process` erfüllt. Dem Ausschnitt des sich für den Nutzer ergebenden Outputs in der obigen Abbildung kann man daher entnehmen, dass keinerlei Verstöße gegen die definierten Bedingungen vorliegen, sodass die grundlegende semantische Sinnhaftigkeit und Eignung zur weiteren Analyse der zu untersuchenden Instanzen gewährleistet ist („Konformität: true“). Im nächsten Schritt wird nun die SPARQL-Abfrage zur Identifikation semantisch ähnlicher Reaktionen ausgeführt, welche ebenfalls durch den Nutzer im Rahmen der GUI angestoßen werden kann (s. Codeabschnitt 4.11). Diese Abfrage implementiert die Logik ($Ideal \subseteq Experimental$), um Paare von experimentellen

Reaktionen (`isExperimental = true`) und idealen Reaktionen (`isExperimental = false`) zu finden, wobei alle Teilnehmer der idealen Reaktion auch Teilnehmer der experimentellen Reaktion sind. Bei Anwendung auf das definierte Szenario mit der zusätzlichen Anwesenheit von Isoamylaldehyd in der experimentellen Reaktion „R0“ liefert die Abfrage erfolgreich das erwartete positive Ergebnis, wie der folgende Auszug aus der GUI-Ausgabe in Abbildung 5.9 zeigt:

Ergebnis-Output für SPARQL-Ähnlichkeitsanalyse im GUI
<pre>Gefundene Matches (Exp(T) enthält Ideal(F)): - Exp (T): 'Reaktion r0' (http://example.org/my-local-ontology#Reaction_r0) enthält Ideal (F): 'Reaktion R12516' (http://example.org/my-local-ontology#Reaction_R12516)</pre>

Abbildung 5.9: Ergebnis der SPARQL-Ähnlichkeitsanalyse

6 Diskussion

Aufbauend auf der entwickelten Methode zur Ontologieerweiterung und den Ergebnissen der automatisierten Datenintegration und -analyse werden zunächst die Stärken und Potenziale des erstellten wissensbasierten Systems diskutiert. Anschließend werden die bei der Entwicklung und Anwendung aufgetretenen Herausforderungen sowie die inhärenten Limitationen des gewählten Ansatzes und der eingesetzten Software-Tools ausführlich diskutiert. Ziel ist es, die Leistungsfähigkeit der entwickelten Datenstruktur und der Analysepipelines realistisch einzuschätzen und Ansatzpunkte für zukünftige Verbesserungen aufzuzeigen.

6.1 Bewertung des entwickelten Systems und der Ontologie

Ein zentrales Ergebnis dieser Arbeit ist die erfolgreiche Entwicklung und Implementierung einer weitgehend automatisierten Pipeline zur Integration und Klassifikation von Reaktionsdaten. Durch die Verwendung einer zentralen Ontologie als Wissensbasis sowie der Entwicklung von entsprechenden Schnittstellen gelingt es, Datensilos zwischen unterschiedlichen Quellen wie experimentellen Daten aus EnzymeML, kuratierten Daten aus der KEGG und manuell eingegebenen Referenzdaten aufzubrechen. Dieser Schritt leistet einen Beitrag zur Interoperabilität der Ontologie mit bestehenden chemischen Datenformaten. Die Pipeline standardisiert die eingehenden Informationen, indem sie z. B. über die PubChem API Namen in SMILES-Codes oder SMILES-Codes in kanonische IRIs auflöst und die Daten so in einheitlicher, semantisch angereicherter Form im Wissensgraphen ablegt. Die Eignung von KEGG als Referenzdatenquelle wurde durch die gute Datenstruktur und die Verfügbarkeit von Reaktandeninformationen über die KEGG Rest API bestätigt. Die entwickelte graphische Benutzeroberfläche (GUI) macht diese komplexe Funktionalität auch Forschenden ohne vertiefte Ontologiekenntnisse zugänglich. Darüber hinaus konnte eine automatische Klassifizierungsfunktion erfolgreich implementiert werden. Die Kombination aus Substrukturanalyse mittels RDKit zur Identifikation funktioneller Gruppen und anschließendem OWL2-Reasoning (HermiT) auf Grundlage vordefinierter Klassenaxiome ermöglicht die korrekte Zuordnung der importierten Reaktionen zu bestimmten, semantisch definierten Reaktionsklassen (z. B. Fettverseifung vs. einfache Esterhydrolyse), wie in Abschnitt 5.0.3 gezeigt wird. Entscheidend ist dabei nicht nur die Identifikation, die durch gezielt angepasste SMARTS-Muster erfolgt, sondern auch die Quantifizierung der funktionellen Gruppen, d. h. die Zählung ihres Vorkommens pro Molekül. Diese Zählung ermög-

licht eine verfeinerte automatische Klassifizierung der Substanzen (z. B. die Zuordnung von Glycerin zur Klasse der Glycerole aufgrund von drei Hydroxygruppen), die die Voraussetzung schafft, um anschließend chemisch verwandte Reaktionstypen wie die Fettverseifung (mit Triglyceriden/-Glycerin) und die einfache Esterhydrolyse (mit einfachen Estern/Alkoholen) durch den Reasoner semantisch eindeutig voneinander abzugrenzen. Dieser wissensbasierte Ansatz bietet das Potenzial, auch größere Reaktionsdatensätze automatisiert zu verarbeiten und zu klassifizieren, was über die Möglichkeiten reiner Datenbankabfragen hinausgeht, da durch das integrierte OWL-Reasoning implizites Wissen – wie eben die spezifische Klassenzugehörigkeit von Reaktionen basierend auf ihren Teilnehmern und deren Eigenschaften – logisch abgeleitet und somit neues Wissen generiert werden kann, welches über die explizit importierten oder direkt abfragbaren Daten hinausgeht. Ergänzend zur OWL-Logik erweitert die Verwendung von SPARQL-Abfragen und SHACL-Validierungen die Analysemöglichkeiten. Während SHACL die strukturelle Integrität der Daten sicherstellen kann, erlauben SPARQL-Abfragen flexible Suchen nach komplexen Mustern im Graphen. Die Werkzeuge ergänzen die OWA-basierte Logik des OWL-Reasoners um graphbasierte Validierungs- und Abfragemechanismen, die teilweise einer Closed World Assumption (CWA) folgen, wie in Abschnitt 2.2.4 erläutert. Insbesondere zur Validierung struktureller Integrität (z. B. Prüfung auf minimale Teilnehmerzahl pro Reaktion mittels SHACL) oder zur Identifikation spezifischer Beziehungsmuster (z. B. Suche nach experimentellen Reaktionen, die eine ideale Reaktion als Teilmenge enthalten, via SPARQL) können diese Ansätze eine direktere und potenziell effizientere Alternative zur alleinigen Modellierung über komplexe OWL-Axiome darstellen. Sie stellen somit eine wertvolle Erweiterung des logischen Werkzeugkastens zur Analyse des Wissensgraphen dar.

6.2 Herausforderungen, Limitationen und Modellierungsentscheidungen

Trotz der erzielten Erfolge zeigten sich während der Entwicklung und Anwendung auch Herausforderungen und Limitationen, die im Folgenden diskutiert werden.

6.2.1 Modellierungsentscheidungen und ihre Grenzen

Eine grundlegende Herausforderung liegt in der statischen Natur der Reaktionsdarstellung. Die Ontologie modelliert die gesamte Umwandlung, aber der Zeitpunkt der Betrachtung oder das Erreichen eines Gleichgewichts wird nicht explizit erfasst, was die eindeutige Definition von Edukt (obo:BFO_0000078) und Produkt (obo:BFO_0000077) erschwert und die Berücksichtigung von nicht vollständig abgelaufenen Reaktionen mit Zwischenprodukten verhindert. Es sei darauf hingewiesen, dass dem Datensatz bzw. den darin enthaltenen Reaktionen durch diese Modellierung mit einem fixen Set an Edukten und Produkten implizit ein statischer Zustand oder „Zeitpunkt“ zugeordnet wird. Dieser Modellzustand ist in diesem Fall der Endpunkt der betrachteten Gesamttransformation. Infolge der Diskrepanz zwischen dem modellierten Zustand und dem realen dynamischen

Prozess können fehlerhafte oder unvollständige Angaben entstehen.

Des Weiteren ist ursprünglich angedacht gewesen, Substanzklassen über Kardinalitätsrestriktionen auf die `has_functional_part`-Eigenschaft zu definieren (z. B. „hat genau 3 Hydroxygruppen“). Tests mit dem HermiT-Reasoner zeigen jedoch, dass Äquivalenzaxiome mit solchen Kardinalitäten nicht zuverlässig zur erwarteten Klassifizierung der Instanzen führen, vermutlich aufgrund von Beschränkungen des Tools Reasoner [65]. Als praktikabler Workaround sind daher explizite Klassen und Instanzen für die jeweilige Anzahl der funktionellen Gruppen gebildet worden (z. B. `hydroxy_3fach`) und die Stoffklassen über eine Relation zu diesen „Zähl“-Instanzen definiert. Dieser Ansatz ermöglicht eine erfolgreiche automatische Klassifizierung, erhöht jedoch die Anzahl der Klassen und Instanzen in der Ontologie.

Eine weitere Limitation des auf funktionellen Gruppen basierenden Klassifizierungsansatzes zeigt sich bei anorganischen Substanzen oder einfachen Ionen wie Wasser, Sauerstoff, Metallkationen oder spezifischen Basen wie Natriumhydroxid. Da diese keine komplexen funktionellen Gruppen besitzen, die mittels SMARTS effektiv erfasst werden können, oder (wie bei Basen) nicht durch ein einziges Strukturmuster definiert sind, versagt hier die allgemeine Substrukturanalyse. Für die korrekte Identifizierung und Klassifizierung dieser Elemente im Rahmen der Reaktionsmodellierung (etwa zur Erfüllung der `Equivalent-To`-Axiome) wurden spezifische Python-Hilfsfunktionen (z. B. `is_oxygen`) und explizite Namensprüfungen implementiert. Diese erkennen die in der Praxis häufig auftretenden Fälle direkt. Dieser pragmatische Ansatz mag das unmittelbare Problem zwar lösen, jedoch resultiert daraus eine weniger homogene Klassifizierungslogik. Zudem sind potenziell manuelle Code-Anpassungen erforderlich, um weitere spezifische anorganische Teilnehmer oder Basentypen zu unterstützen.

Ebenso ist die Nützlichkeit des Stöchiometriechecks kontextabhängig: Während er für idealisierte Referenzreaktionen sinnvoll ist, kann er bei experimentellen Daten aus EnzymeML ohnehin nicht durchgeführt werden, da das EnzymeML-Sheet keine Informationen über die stöchiometrischen Verhältnisse der Reaktionen bereitstellt. Darüber hinaus muss die Natur der KEGG-Daten berücksichtigt werden, die eine Mischung aus spezifischen (oft enzymatischen) Reaktionen und konzeptuell-allgemeineren Reaktionen enthalten. Die Verwendung dieser als „ideale“ Referenz ist eine pragmatische Wahl, erfordert aber ein Bewusstsein für diesen Kontext.

6.2.2 Herausforderungen der chemischen Repräsentation und Analyse

Abgesehen von den o. g. grundlegenden Entscheidungen bezüglich der Modellierung ergaben sich spezifische Herausforderungen bei der Auswahl und Anwendung chemischer Repräsentationsformate sowie der darauf aufbauenden, automatisierten Analyse funktioneller Merkmale.

Die Wahl von SMILES/SMARTS als chemische Darstellung hat im Gegensatz zu InChI Nachtei-

le in der Eindeutigkeit aufgrund fehlender generischer Algorithmen zur Erstellung kanonisierter SMILES. Allerdings eignet sich die InChI Notation nicht für die Formulierung von Substrukturen, während die Erweiterung der SMILES-Notation, SMARTS, gezielt für den Zweck der Substruktursuche entwickelt wurde und aufgrund der erweiterten Syntax eine sehr distinkte Suche nach funktionalen Anteilen ermöglicht. Daher ist die Wahl der SMILES bzw. SMARTS Notation in diesem Kontext alternativlos.

Darüber hinaus erfordert die Definition präziser SMARTS-Muster Sorgfalt und chemisches Verständnis, wie am Beispiel des 3-Methylbutyl-Restes (Isoamyl) deutlich wird. Ziel ist die Erkennung dieses Restes in Alkoholen, Estern aber auch Aldehyden. Ein zunächst naheliegender SMARTS-Ansatz wie '[CH3][CH]([CH3])[CH2][CH2]' beschreibt zwar die Basiskette, scheitert aber an der Erkennung in Isoamylaldehyd (CC(C)CC=O). Der Grund dafür ist das strenge Matching der RDKit-Erweiterung, das zusätzlich die Bindungsumgebung und die Hybridisierung berücksichtigt: Die letzte '[CH2]'-Gruppe wird nicht gematcht, wenn sie an den sp^2 -hybridisierten Carbonylkohlenstoff in der funktionalen Endgruppe des Isoamylaldehyd gebunden ist. Um eine robuste Erkennung des Restes auch in solch unterschiedlichen funktionellen Umgebungen (Alkohole, Ester, Aldehyde) zu gewährleisten, musste das Suchmuster zu '[CH3][CH]([CH3])[CH2][C;D2]' verfeinert werden. Der Deskriptor '[C;D2]' spezifiziert hier ein Kohlenstoffatom mit genau zwei Nachbarn, unabhängig von deren Hybridisierung oder Bindungsordnung, was die Erkennung über verschiedene funktionelle Gruppen hinweg ermöglicht. Dieses Beispiel verdeutlicht den Aufwand und den Detaillierungsgrad, der bei der Formulierung von SMARTS-Mustern für eine zuverlässige, automatisierte Substrukturanalyse erforderlich ist.

Ein weiteres Problemfeld stellt die semantische Mehrdeutigkeit von Substanznamen in externen Datenbanken dar. Ein Beispiel hierfür ist das von PubChem als Acetat-Anion interpretierte „Acetate“ aus der KEGG-Reaktion, welches in diesem Kontext Essigsäure meint. Die implementierte Override-Liste (`KEGG_SMILES_OVERRIDES`) stellt eine pragmatische, jedoch manuell zu pflegende Lösung für solche bekannten Fälle dar.

Schließlich ist die Abdeckung funktioneller Gruppen limitiert. Es ist nicht möglich, jede denkbare Gruppe oder jeden Alkylrest (deren Vielfalt enorm ist) einer spezifischen Klasse mit ChEBI-IRI zuzuordnen, weswegen zu hinterlegende Datenbankeinträge manuell ergänzt werden müssten.

6.2.3 Grenzen des ontologischen Modells & Datenqualität

Die entwickelte Ontologie fokussiert bewusst auf für die Klassifizierung und Verkettung relevante Informationen und repräsentiert nicht sämtliche potenziellen Details chemischer oder prozesstechnischer Natur. Die Qualität der Resultate ist dabei maßgeblich von der Qualität der Eingangsdaten abhängig. Inkonsistente oder fehlerhafte Daten aus den Quellen führen unweigerlich zu Fehlern in der Analyse und Klassifikation. In diesem Kontext ist speziell der manuelle Import von Reakti-

onsdaten als kritisch zu erachten, da die korrekte Analyse maßgeblich von der korrekten Eingabe von Substanznamen und ggf. SMILES Darstellungen durch den Nutzer abhängt. Auch hier ist die semantische Mehrdeutigkeit wieder ein potenzielles Problem. Der erfolgreiche manuelle Import ist davon abhängig, dass der Nutzer einen Substanznamen eingibt, der in explizit dieser Form auch in der PubChem-Datenbank hinterlegt ist.

Zusammenfassend zeigt die Diskussion, dass die entwickelte Ontologie und Pipeline erfolgreich als Proof-of-Concept für die automatisierte Klassifikation dienen. Das Aufbauen einer detaillierteren Reaktionsontologie erscheint erst sinnvoll, wenn die komplexe, zugrundeliegende Modellierung der Substanzen und ihrer funktionellen Gruppen – mit ihren Abhängigkeiten von Basis-Ontologien und Limitationen – weiter verfeinert wurde. Die vorliegende Arbeit bildet somit eine wesentliche Grundlage für automatisierte Klassifikation von Reaktionsdaten, beleuchtet kritische Punkte bestehender Systeme und definiert klare Ansatzpunkte für zukünftige Entwicklungen in der semantischen Reaktionsanalyse.

7 Ausblick

Die Implementierung eines „On-Demand“-Abrufs von Referenzreaktionen, beispielsweise aus der KEGG-Datenbank, birgt ein vielversprechendes Potenzial für zukünftige Erweiterungen. Anstatt eine Vielzahl von KEGG-Reaktionen vorab oder manuell zu importieren, könnte das System automatisch nach relevanten Vergleichsreaktionen suchen, sobald eine neue experimentelle Reaktion aus der EnzymeML-Datei in den lokalen Wissensgraphen aufgenommen wird. Die Umsetzung dieses Ansatzes könnte auf zwei verschiedene Arten erfolgen, je nachdem, ob eine lokale Speicherung der relevanten KEGG-Datenbankeinträge auf eigenen Server erwünscht ist oder nicht.

Einerseits könnte die Realisierung dieses Vorhabens durch eine direkte Kommunikation mit dem öffentlichen SPARQL-Endpunkt der KEGG-Datenbank (<https://www.genome.jp/sparql/kegg>) erfolgen, wodurch das lokale Speichern in einer eigenen Infrastruktur (sei es auf lokalen Rechnern oder Privatservern) der gesamten KEGG-Reaktionsdatenbank umgangen wird. Der generelle Ablauf gestaltet sich wie folgt: Nach dem erfolgreichen Import einer experimentellen Reaktion (z. B. aus einer EnzymeML Datei) in die lokale Ontologie, extrahiert das System deren zentrale Merkmale, insbesondere die Identifikatoren (CIDs oder IRIs) der beteiligten Edukte und Produkte, wobei diese Funktionalität bereits implementiert ist. Auf Basis dieser Merkmale muss eine spezifische SPARQL-Abfrage dynamisch konstruiert werden – eine sogenannte „Discovery Query“. Im Gegensatz zu den bisherigen lokalen Abfragen zielt diese darauf ab, unmittelbar auf dem externen KEGG-Graphen nach KEGG-Reaktionen zu suchen, die eine teilnehmerbasierte Ähnlichkeit zur importierten experimentellen Reaktion aufweisen. Es empfiehlt sich, die Suchbedingungen bewusst wenig restriktiv zu gestalten, beispielsweise durch die Anforderung von mindestens einem übereinstimmenden Edukt oder Produkt. Auf diese Weise lässt sich eine möglichst umfassende Liste potenzieller Kandidaten für den anschließenden lokalen Import ermitteln. Die Kommunikation mit dem Endpunkt, also das Senden dieser Abfrage und der Empfang der Ergebnisse, würde unter Verwendung von HTTP-Anfragen erfolgen, die über etablierte Python-Bibliotheken wie `requests` abgewickelt würden. Dies entspricht der bereits implementierten Kommunikation mit der PubChem-API. Die von der Abfrage an den KEGG-Endpunkt zurückgelieferten Ergebnisse wären primär Listen von KEGG Reaction IDs, die als potenziell relevante Vergleichsreaktionen identifiziert wurden. Diese identifizierten Reaktionen könnten anschließend (ggf. nach einer Filterung oder Auswahl durch den Nutzer) automatisiert über die bereits bestehende Import-Pipeline für Reaktionen aus der KEGG-Datenbank in den lokalen Wissensgraphen importiert werden. Denkbar wäre

zudem dem Nutzer das Formulieren von individuellen Filterbedingungen zu ermöglichen, die seinem Anwendungskontext entsprechen, beispielsweise durch Angabe von verpflichtend auftretenden Reaktanden in den Suchergebnissen. Dadurch erfolgt eine bedarfsgesteuerte, lokale Anreicherung der Ontologie mit relevanten Referenzdaten, auf welcher wiederum die bereits entwickelten lokalen Vergleichs-SPARQL-Abfragen basierend agieren. Dieser Mechanismus stellt somit eine signifikante Weiterentwicklung dar, um externe Datenbanken intelligent zur Kontextualisierung neu gewonnener experimenteller Daten zu nutzen. Dabei ist die Formulierung effizienter Ähnlichkeitsabfragen für den externen Endpunkt von zentraler Bedeutung. Ein Nachteil dieses Workflows würde allerdings in der hohen Anzahl benötigter Anfragen und der damit auftretenden Rechenleistung bestehen. Weiterhin würde die lokale Speicherung des Wissensgraphen einen signifikanten Mehraufwand bezüglich Aufbau und Wartung bedeuten und unter Umständen sehr ressourcenintensiv sein, was gerade auf kleinen Systemen zu einem überhöhten Leistungsbedarf führen kann.

Eine alternative und potenziell robustere Weiterentwicklungsperspektive stellt die Transformation des Systems in einen zentralisierten, webbasierten Service dar. Anstatt die gesamte Logik und die Ontologie auf dem lokalen Rechner des Nutzers auszuführen, könnten die Kernkomponenten auf einem dedizierten Server betrieben werden. Die Notwendigkeit von Echtzeit-Abfragen an externe, öffentliche SPARQL-Endpunkte während der Analyse würde somit entfallen oder auf ein Minimum reduziert werden (z. B. nur für initiale Updates oder sehr spezielle Daten), was die Abhängigkeit von externen Diensten und potenzielle Performance-Engpässe verringert. Für den Endnutzer ergeben sich ebenfalls Vorteile: So würde er weder die Ontologie noch spezielle Software wie Protégé lokal installieren müssen. Der Zugriff könnte über eine rein webbasierte Oberfläche oder eine einfache Client-Anwendung erfolgen, die mit dem Backend-Service kommuniziert. Die Nutzer würden lediglich ihre experimentellen Daten (z. B. EnzymeML) hochladen oder Reaktionen manuell über die Weboberfläche eingeben. Die Verarbeitung der Daten (Import, Substrukturanalyse, Reasoning, SPARQL-Abfragen gegen die lokale KEGG-Kopie und die Nutzerdaten) würde auf dem Server stattfinden. Dieser Ansatz erfordert zwar einen initial höheren Aufwand für die Einrichtung und Wartung der Server-Infrastruktur. Langfristig bietet sich jedoch der Vorteil, dass Performance, Skalierbarkeit, zentrale Datenverwaltung und einfachere Zugänglichkeit für eine größere Nutzergruppe innerhalb einer Forschungseinrichtung ermöglicht werden. Zudem müssen Aspekte der Datensicherheit und des Datenschutzes bei der Speicherung von Nutzerdaten zentral adressiert werden.

Literatur

- [1] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg: „The FAIR Guiding Principles for scientific data management and stewardship“, *Scientific Data* 3, 2016, DOI 10.1038/sdata.2016.18, 16–18.
- [2] N. T. Garabedian, P. J. Schreiber, N. Brandt, P. Zschumme, I. L. Blatter, A. Dollmann, C. Haug, D. Kümmel, Y. Li, F. Meyer: „Generating FAIR research data in experimental tribology“, *Scientific Data* 9, 2022, DOI 10.1038/s41597-022-01416-x, 315.
- [3] R. Studer, V. R. Benjamins, D. Fensel: „Knowledge engineering: Principles and methods“, *Data & Knowl. Eng* 25, 1998, DOI 10.1016/S0169-023X(97)00056-6, 161–197.
- [4] Technische Universität Wien, Interoperabilität, **2024**, <https://www.tuwien.at/forschung/fti-support/forschungsdaten/fdm-infos-tipps/aufbewahren-und-publizieren/interoperabilitaet> (besucht am 03.05.2024).
- [5] J. Bao, D. Calvanese, B. C. Grau, M. Dzbor, A. Fokoue, OWL 2 Web Ontology Language Document Overview (Second Edition), <https://www.w3.org/TR/2012/REC-owl2-overview-20121211/>, W3C, **2012**.
- [6] G. Schreiber, Y. Raimond, RDF 1.1 Primer, W3C Note, <https://www.w3.org/TR/2014/NOTE-rdf11-primer-20140624/>, W3C, **2014**.
- [7] R. Cyganiak, M. Lanthaler, D. Wood, RDF 1.1 Concepts and Abstract Syntax, W3C Recommendation, <https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>, W3C, **2014**.
- [8] M. Krötzsch, S. Rudolph, B. Parsia, P. Patel-Schneider, P. Hitzler, OWL 2 Web Ontology Language Primer (Second Edition), W3C Recommendation, <https://www.w3.org/TR/2012/REC-owl2-primer-20121211/>, W3C, **2012**.
- [9] J. Sequeda, Introduction to Open World Assumption vs. Closed World Assumption, DATAVERSITY, **2025**, <https://www.dataversity.net/introduction-to-open-world-assumption-vs-closed-world-assumption/> (besucht am 02.02.2025).
- [10] M. Debellis, A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins, **2021**, <https://www.michaeldebellis.com/post/owl-tutorial-protege-5-5> (besucht am 10.02.2025).
- [11] M. DeBellis, Synonyms, IRIs, and Labels in OWL and SKOS, **2023**, <https://www.michaeldebellis.com/post/owlsynonyms> (besucht am 03.03.2025).

- [12] C. B. Aranda, O. Corby, S. Das, L. Feigenbaum, P. Gearon, SPARQL 1.1 Overview, W3C Recommendation, <https://www.w3.org/TR/2013/REC-sparql11-overview-20130321/>, W3C, **2013**.
- [13] S. Harris, A. Seaborne, SPARQL 1.1 Query Language, W3C Recommendation, <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>, W3C, **2013**.
- [14] H. Knublauch, D. Kontokostas, Shapes Constraint Language (SHACL), W3C Recommendation, <https://www.w3.org/TR/2017/REC-shacl-20170720/>, W3C, **2017**.
- [15] P. Strömert, J. Hunold, A. Castro, S. Neumann, O. Koepler: „Ontologies4Chem: the landscape of ontologies in chemistry“, *Pure Appl. Chem.* *94*, 2022, DOI 10.1515/pac-2021-2007, 605–622.
- [16] National Center for Biotechnology Information, About PubChem, **2025**, <https://pubchem.ncbi.nlm.nih.gov/docs/about> (besucht am 06.03.2025).
- [17] OBO Foundry / Ontobee, Chemical Entities of Biological Interest (ChEBI) Ontology, **2025**, <https://ontobee.org/ontology/CHEBI> (besucht am 07.03.2025).
- [18] M. Kanehisa, S. Goto: „KEGG: kyoto encyclopedia of genes and genomes“, *Nucleic Acids Res.* *28*, 2000, DOI 10.1093/nar/28.1.27, 27–30.
- [19] R. Alcántara, K. B. Axelsen, A. Morgat, E. Belda, E. Coudert, A. Bridge, H. Cao, P. de Matos, M. Ennis, S. Turner, G. Owen, L. Bougueleret, I. Xenarios, C. Steinbeck: „Rhea—a manually curated resource of biochemical reactions“, *Nucleic Acids Res.* *40*, 2011, DOI 10.1093/nar/gkr1126, D754–D760.
- [20] SIB Swiss Institute of Bioinformatics, Rhea Statistics, Rhea - annotated reactions database, **2025**, <https://www.rhea-db.org/statistics> (besucht am 07.03.2025).
- [21] Basic Formal Ontology, Basic Formal Ontology (BFO) 2020, **2025**, <https://basic-formal-ontology.org/bfo-2020.html> (besucht am 07.03.2025).
- [22] OBO Foundry, About OBO Foundry, **2025**, <http://dashboard.obofoundry.org/dashboard/index.html> (besucht am 07.03.2025).
- [23] OBO Foundry / Ontobee, Basic Formal Ontology (BFO), **2025**, <https://ontobee.org/ontology/BFO> (besucht am 07.03.2025).
- [24] C. Batchelor, Molecular Process Ontology (MOP), **2012**, <https://github.com/rsc-ontologies/rxno> (besucht am 07.03.2025).
- [25] C. Batchelor, Chemical reactions ontology (RXNO), **2012**, <https://github.com/rsc-ontologies/rxno> (besucht am 07.03.2025).
- [26] OBO Foundry, RXNO - The Reaction Ontology, **2025**, <https://obofoundry.org/ontology/rxno.html> (besucht am 07.03.2025).
- [27] NCBO BioPortal, RXNO - The Reaction Ontology, **2025**, <https://bioportal.bioontology.org/ontologies/RXNO> (besucht am 07.03.2025).
- [28] EnzymeML Initiative, EnzymeML - A Data Exchange Format for Enzymatic Kinetics, EnzymeML Initiative, **2025**, <https://realenzymeml.org/> (besucht am 14.03.2025).

- [29] J. Range, C. Halupczok, J. Lohmann, N. Swainston, C. Kettner, F. T. Bergmann, A. Weidemann, U. Wittig, S. Schnell, J. Pleiss: „EnzymeML—a data exchange format for biocatalysis and enzymology“, *FEBS J.* 289, 2022, DOI 10.1111/febs.16318, 5864–5874.
- [30] Stanford Center for Biomedical Informatics Research, Protégé - Ontology Editor and Knowledge Management System, Stanford University, **2025**, <https://protege.stanford.edu/> (besucht am 07.03.2025).
- [31] M. A. Musen: „The protégé project: a look back and a look forward“, *AI Matters* 1, 2015, DOI 10.1145/2757001.2757003, 4–12.
- [32] Python Software Foundation, Welcome to Python.org, Python Software Foundation, **2025**, <https://www.python.org/> (besucht am 09.03.2025).
- [33] S. Janczura, Python – Müssen Ingenieure diese Programmiersprache auch können?, Ingenieur.de / VDI Verlag GmbH, **2025**, <https://www.ingenieur.de/technik/fachbereiche/itk/python-muessen-ingenieure-diese-programmiersprache-auch-koennen/> (besucht am 07.03.2025).
- [34] Python Software Foundation, The Python Standard Library, Techn. Ber., Python Software Foundation, **2025**.
- [35] Jean-Baptiste Lamy: „Ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies“, *Artificial Intelligence in Medicine* 80, 2017, DOI 10.1016/j.artmed.2017.07.002, 11–28.
- [36] RDKit Community, RDKit Documentation: Overview, **2025**, <https://www.rdkit.org/docs/Overview.html> (besucht am 07.03.2025).
- [37] A. Sommer, N. Car, pySHACL, Version 0.30.1, DOI 10.5281/zenodo.4750840, **2025**, <https://github.com/RDFLib/pySHACL>.
- [38] J. E. F. Friedl, *Mastering Regular Expressions*, 3. Aufl., O'Reilly Media, Sebastopol, CA, **2006**.
- [39] Python Software Foundation, re — Regular expression operations, <https://docs.python.org/3/library/re.html>, **2025**.
- [40] RDKit: Open-Source cheminformatics. Version 2024.03.6, DOI 10.5281/zenodo.13469390, **2025**, <https://www.rdkit.org>.
- [41] RDKit Project, RDKit Book - A Guide to Using RDKit in Chemistry and Data Science, Techn. Ber., RDKit, **2025**.
- [42] International Union of Pure and Applied Chemistry (IUPAC), What on Earth is InChI?, **2019**, <https://iupac.org/100/stories/what-on-earth-is-inchi/> (besucht am 07.03.2025).
- [43] J. M. Goodman, A. T. McNaught, S. R. Heller, I. Pletnev, I. T. Stein, D. Tchekhovskoi: „InChI, the IUPAC International Chemical Identifier“, *J. Cheminf.* 7, 2015, DOI 10.1186/s13321-015-0068-4, 23.

- [44] National Center for Biotechnology Information (NCBI), Acetic Acid - PubChem Compound Summary, **2025**, <https://pubchem.ncbi.nlm.nih.gov/compound/Acetic-acid#section=IUPAC-Name> (besucht am 07.03.2025).
- [45] D. Weininger: „SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules“, *J. Chem. Inf. Comput. Sci.* **28**, 1988, DOI 10.1021/ci00057a005, 31–36.
- [46] Daylight Chemical Information Systems, Inc., SMARTS - A Language for Describing Molecular Patterns, **2025**, <https://www.daylight.com/dayhtml/doc/theory/theory.smarts.html> (besucht am 07.03.2025).
- [47] E. Initiative, PyEnzyme - A Python Library for EnzymeML Processing, **2025**, <https://github.com/EnzymeML/PyEnzyme> (besucht am 11.03.2024).
- [48] F. Kitel, Code for Bachelor thesis Frederick Kitel, Version DRAFT VERSION, DOI 21.11165/4cat/2d9b-emem, **2025**, <https://hdl.handle.net/21.11165/4cat/2d9b-emem>.
- [49] Kanehisa Laboratories, KEGG: Kyoto Encyclopedia of Genes and Genomes, **2024**, <https://www.kegg.jp/kegg/kegg1a.html> (besucht am 11.03.2024).
- [50] European Bioinformatics Institute (EBI), RXNO - The Reaction Ontology, **2024**, <https://www.ebi.ac.uk/ols4/ontologies/rxno?tab=classes> (besucht am 11.03.2024).
- [51] J. Hastings, G. Owen, A. Dekker, M. Ennis, N. Kale, V. Muthukrishnan, S. Turner, N. Swainston, P. Mendes, C. Steinbeck: „ChEBI in 2016: Improved services and an expanding collection of metabolites“, *Nucleic Acids Res.* **44**, 2016, DOI 10.1093/nar/gkv1031, D1214–D1219.
- [52] E. Abel: „Notiz zur Theorie der Verseifung von Estern mehrwertiger Alkohole“, *Z. Elektrochem. Angew. Phys. Chem.* **12**, 1906, 681–696.
- [53] K. Schumann, K. Siekmann in *Ullmann's Encyclopedia of Industrial Chemistry*, Wiley-VCH Verlag GmbH & Co. KGaA, **2012**, S. 241–261.
- [54] O. Mumm: „Über den Mechanismus der Ester-Verseifung und Ester-Bildung“, *Ber. Dtsch. Chem. Ges. A/B* **72**, 1939, DOI 10.1002/cber.19390721012, 1874–1887.
- [55] European Bioinformatics Institute (EBI), Diol (ChEBI:23824), **2024**, <https://www.ebi.ac.uk/chebi/searchId.do?chebiId=ChEBI:23824> (besucht am 11.03.2024).
- [56] European Bioinformatics Institute (EBI), Triol (ChEBI:27136), **2024**, <https://www.ebi.ac.uk/chebi/searchId.do?chebiId=ChEBI:27136> (besucht am 11.03.2024).
- [57] European Bioinformatics Institute (EBI), Glycerol (ChEBI:17754), **2024**, <https://www.ebi.ac.uk/chebi/searchId.do?chebiId=ChEBI:17754> (besucht am 11.03.2024).
- [58] European Bioinformatics Institute (EBI), Butan-1-ol (ChEBI:28885), **2024**, <https://www.ebi.ac.uk/chebi/searchId.do?chebiId=ChEBI:28885> (besucht am 11.03.2024).
- [59] K. R. et al., Requests: HTTP for Humans, **2024**, <https://requests.readthedocs.io/en/latest/> (besucht am 17.03.2024).

- [60] InChI Trust, InChI Technical FAQ, **2024**, <https://www.inchi-trust.org/technical-faq/> (besucht am 17.03.2024).
- [61] J. Clayden, N. Greeves, S. Warren, *Organic Chemistry*, 2. Aufl., Oxford University Press, Oxford, **2012**.
- [62] National Center for Biotechnology Information, Formic anhydride, PubChem / NCBI, **2025**, <https://pubchem.ncbi.nlm.nih.gov/compound/9548680> (besucht am 18.03.2025).
- [63] M. Birbeck, S. McCarron, CURIE Syntax 1.0, Techn. Ber., <https://www.w3.org/TR/2010/NOTE-curie-20101216/>, W3C, **2010**.
- [64] Kanehisa Laboratories, KEGG REACTION entry R12516, **2025**, <https://www.genome.jp/entry/R12516> (besucht am 10.03.2025).
- [65] K. Dentler, R. Cornet, A. ten Teije, N. de Keizer: „Comparison of reasoners for large ontologies in the OWL 2 EL profile“, *Semantic Web 2*, 2011, DOI 10.3233/SW-2011-0034, 71–87.

Anhang

Die in Codeabschnitt A.1 dargestellten Funktionen extrahieren die Reaktanden sowie stöchiometrischen Koeffizienten aus EQUATION-Zeile des KEGG-Eintrags über Regex und speichert diese in entsprechenden Listen.

```
1 def parse_kegg_equation(equation_line):
2     educts_dict, products_dict = {}, {}
3     if not equation_line:
4         return educts_dict, products_dict
5     parts = re.split(r"\s*<?[-]>*\s*", equation_line.strip()) #Regex: <, =, > werden
        als Trennzeichen genutzt, Equation Line wird extrahiert und in 2 Parts aufgeteilt:
        Edukteliste und Produkteliste
6     if len(parts) != 2:
7         print(f"[WARNUNG] Ungültiges Gleichungsformat KEGG: {equation_line}")
8         return educts_dict, products_dict
9     reactants_side, products_side = parts
10
11     def parse_side(side_string):
12         compounds = {}
13         matches = re.findall(r"(\d*)\s*(C\d{5})", side_string)
14         for coeff_str, cid in matches:
15             compounds[cid] = int(coeff_str) if coeff_str else 1
16         return compounds
17
18     educts_dict = parse_side(reactants_side)
19     products_dict = parse_side(products_side)
20     return educts_dict, products_dict
```

Codeabschnitt A.1: Definition der parse_kegg_equation(equation_line) Funktion unter Nutzung von Regex

Die in Codeabschnitt A.2 dargestellte Funktion beschreibt die Kommunikation mit der KEGG API zum Abruf der Datenbankeinträge und Extraktion der darin enthaltenen relevanten Informationen wie z.B die EQUATION Zeile, aus der über die oben dargestellte Funktion die CIDs der beteiligten Substanzen abgerufen werden

```
1 def process_single_kegg_reaction(reaction_id):
```

```

2 # Kommunikaktion mit Rest API von KEGG: Abruf von Reaktionen über ihre ID: Input
  findet in GUI statt
3 output_text.insert(tk.END, f"[INFO] Verarbeite KEGG Reaktion {reaction_id}...\n")
4 reaction_url = f"https://rest.kegg.jp/get/{reaction_id}"
5 try:
6     response = requests.get(reaction_url, timeout=15)
7     response.raise_for_status()
8     reaction_data = response.text
9 except requests.exceptions.RequestException as e:
10     output_text.insert(tk.END, f"[FEHLER] Abrufen {reaction_id}: {e}\n")
11     return False
12 reaction_name, equation_line = f"KEGG {reaction_id}", None
13 for line in reaction_data.split("\n"):
14     if line.startswith("NAME") and reaction_name == f"KEGG {reaction_id}":
15         reaction_name = line[len("NAME ") :].strip()
16     elif line.startswith("EQUATION"): #Equation Line wird extrahiert (
  vorerst ganz)
17         equation_line = line[len("EQUATION ") :].strip()
18 if not equation_line:
19     output_text.insert(tk.END, f"[FEHLER] Keine EQUATION für {reaction_id}.\n")
20     return False
21 kegg_educts_coeffs, kegg_products_coeffs = parse_kegg_equation(equation_line)
22 if not kegg_educts_coeffs or not kegg_products_coeffs:
23     output_text.insert(
24         tk.END, f"[FEHLER] Parsen Gleichung {reaction_id}: {equation_line}\n"
25     )
26     return False
27 all_cids = list(kegg_educts_coeffs.keys() | kegg_products_coeffs.keys())
28 if not all_cids:
29     output_text.insert(tk.END, f"[FEHLER] Keine CIDs in Gleichung {reaction_id}.\n")
30     return False
31 compound_details = get_kegg_compound_details(all_cids)
32 educts, r_edu_s, products, r_prod_s = [], [], [], [] # Kürzer
33 missing_data = False
34 for cid in kegg_educts_coeffs.keys():
35     d = compound_details.get(cid)
36     if d and d.get("name") and d.get("smiles"):
37         if d["smiles"] not in r_edu_s:
38             educts.append(d["name"])
39             r_edu_s.append(d["smiles"])
40     else:
41         output_text.insert(
42             tk.END, f"[WARNUNG] Fehlende Daten Edukt CID {cid} in {reaction_id}.\n"
43         )
44     missing_data = True

```

```

45 for cid in kegg_products_coeffs.keys():
46     d = compound_details.get(cid)
47     if d and d.get("name") and d.get("smiles"):
48         if d["smiles"] not in r_prod_s:
49             products.append(d["name"])
50             r_prod_s.append(d["smiles"])
51     else:
52         output_text.insert(
53             tk.END,
54             f"[WARNUNG] Fehlende Daten Produkt CID {cid} in {reaction_id}.\n",
55         )
56         missing_data = True
57 if missing_data:
58     output_text.insert(
59         tk.END, f"[FEHLER] Import {reaction_id} abgebrochen: fehlende Daten.\n"
60     )
61     return False
62 if not educts or not products:
63     output_text.insert(
64         tk.END, f"[FEHLER] Keine gültigen Edukte/Produkte für {reaction_id}.\n"
65     )
66     return False
67 source_info = f"KEGG Reaction ID: {reaction_id}" # Quelle für die Reaktion
68 stöchio_data[reaction_id] = {
69     "reaction_name": reaction_name,
70     "educt_smiles": r_edu_s,
71     "product_smiles": r_prod_s,
72     "educt_names": educts,
73     "product_names": products,
74 }
75 print(f"[INFO] stöchio_data aktualisiert für {reaction_id}")
76 try:
77     execute_import(reaction_id, r_edu_s, r_prod_s, educts, products, source_info)
78     return True
79 except Exception as e:
80
81     output_text.insert(
82         tk.END, f"[FEHLER] execute_import für KEGG {reaction_id}: {e}\n"
83     )
84     traceback.print_exc() # Zeige Details in der Konsole
85     if reaction_id in stöchio_data:
86         del stöchio_data[reaction_id]
87     return False

```

Codeabschnitt A.2: Definition der process_single_kegg_reaction(reaction_id) Funktion

Abbildung A.1 zeigt beispielhaft die Ausschnitte der relevanten Tabellenblätter innerhalb des

EnzymeML-Sheets. Über pyEnzyme erfolgt die Extraktion der Reaktionsdaten vor allem aus den unten dargestellten Spalten für Substanznamen, SMILES Darstellung sowie Edukte und Produkte.

Reactions

ID	Name	Temperature value	Temperature unit	pH value	Reversible	Educts	Products	Proteins	Modifiers
r0									
r1	Triacetin Sap (Ref.)	25	°C	7	reversible	Triacetin, Sodium hydroxide	Acetat, Sodium cation, Glycerol	Pyruvate decarboxylase isozyme 1	
r2	Butyl Acetate Spaltung	25	°C	7	reversible	Butyl Acetate, Sodium hydroxide	Acetat, Sodium cation, Butan-1-ol	Pyruvate decarboxylase isozyme 1	

Reactants (species)

ID	Name	Vessel	Constant	SMILES	InCHI
s0	Butyl Acetate	CSTR	Constant	CCCCOC(=O)C	InChI=1S/C8H16O3/c1-2(4)3(5)6/h1H3,1H5,6H/p-1
s1	Butan-1-ol	CSTR	Constant	CCCCO	
s2	Glycerol	CSTR	Constant	C(C(CO)O)O	InChI=1S/C3H8O3/c2-1-3
s3	Triacetin	CSTR	Constant	CC(=O)OCC(OC(=O)C)OC(=O)C	
s4	Sodium hydroxide	CSTR	Constant	[OH-].[Na+]	
s5	Acetat	CSTR	Constant	CC(=O)[O-]	
s6	Sodium cation	CSTR	Constant	[Na+]	

Abbildung A.1: Exemplarische Darstellung von relevanten EnzymeML-Tabellenblätter für Reaktionen und Reaktanden

Nachfolgend ist in Abbildung A.2 die Reaktionsgleichung der zweiten Beispielreaktion, Hydrolyse von Essigsäurebutylester, dargestellt:

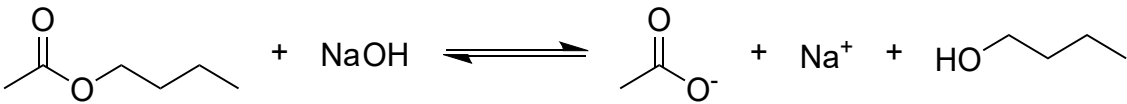


Abbildung A.2: Basische Esterhydrolyse von Essigsäurebutylester